

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Trabajo Terminal: 2026-A114

Simulación de sistemas complejos mediante autómatas celulares neuronales

Presenta:

Angeles Lopez Erick Jesse

eangelesl1700@alumno.ipn.mx

Director:

Dr. Genaro Juárez Martínez

Ciudad de México, 17 de noviembre de 2025

Índice general

1. Introducción **1**

1.1. Resumen 1

1.2. Planteamiento del problema 1

1.2.1. Preguntas de investigación 3

1.3. Objetivo 3

1.3.1. Objetivos específicos 4

1.4. Hipótesis 4

1.5. Justificación 4

1.6. Alcances y limitaciones 6

2. Marco teórico: Teoría de sistemas, caos y simulación **7**

2.1. Sistemas 7

2.1.1. Clasificación de sistemas 8

2.1.1.1. Sistemas naturales y artificiales 9

2.1.1.2. Sistemas continuos y discretos en el tiempo 9

2.1.1.3. Sistemas estáticos y dinámicos 9

2.1.1.4. Sistemas deterministas y estocásticos 10

2.1.1.5. Sistemas abiertos y cerrados 11

2.2. Propiedades fundamentales de los sistemas 11
2.2.1. Causalidad 12
2.2.2. Invertibilidad 12
2.2.3. Estabilidad 12
2.2.4. Invariante en el tiempo 13
2.3. Sistemas dinámicos 13
2.4. Teoría del caos 14
2.4.1. Exponentes de Lyapunov 15
2.4.2. Atractores 16
2.4.2.1. Tipos de atractores 16
2.5. Sistemas complejos 17
2.6. Modelado de sistemas complejos 19

3. Marco teórico: Autómatas celulares 21

3.1. Vecindad 22
3.2. Estados 22
3.3. Condiciones de frontera 25
3.4. Regla local 26
3.5. Ejemplos 29
3.5.1. John Von Neumann 29
3.5.2. El juego de la vida 30
3.5.3. Autómata celular elemental 34
3.6. Clasificación 38
3.7. Extensiones y cambios de la definición 41

4. Marco teórico: Inteligencia artificial y redes neuronales 43

4.1. Inteligencia artificial y aprendizaje supervisado 44

4.2. Neurona biológica 46

4.3. Neurona artificial 47

4.4. Redes neuronales 48

4.4.1. Funciones de activación 49

4.4.1.1. Binary step (escalón binario) 49

4.4.1.2. Lineal 50

4.4.1.3. Sigmoide 50

4.4.1.4. tanh (hiperbólica) 51

4.4.1.5. ReLU (Rectified Linear Unit) 52

4.4.2. Aprendizaje en redes neuronales 53

4.4.2.1. Función de pérdida 53

4.4.2.2. Optimización 54

4.4.2.3. Entrenamiento 55

4.5. Tipos de redes neuronales 56

4.5.1. Redes neuronales convolucionales 57

4.5.2. Redes neuronales recurrentes 58

5. Marco teórico: Autómatas celulares neuronales 59

5.1. Estado del arte, aplicaciones y lineas actuales de investigación . . . 62

5.1.1. Morfogénesis 63

5.1.1.1. Autómatas celulares neuronales isotrópicos 65

5.1.1.2. Autoclasificación 66

5.1.1.3. Representaciones tridimensionales 69

5.1.2. Autómata celular neuronal en malla 71
5.1.3. Modelos generativos 73
5.1.4. Representaciones y aplicaciones adicionales 76

6. Marco metodológico **79**

6.1. Diseño de la investigación 79
6.1.1. Diseño experimental 79
6.1.1.1. Diseño cuasiexperimental 80
6.2. Enfoque de la investigación 80
6.3. Población 81
6.4. Muestra y muestreo 81
6.5. Tipo de investigación 82
6.6. Técnicas e instrumentos de recolección de datos 82
6.6.1. Técnicas de recolección de datos 82
6.6.2. Instrumentos de recolección de datos 83
6.7. Procedimiento 83
6.8. Técnicas y herramientas de procesamiento de datos 84
6.8.1. Procesamiento de datos cuantitativos 84
6.9. Contextualización de la tecnología 85
6.9.1. Fundamentos y contexto de las tecnologías 85
6.9.1.1. C++ 86
6.9.1.2. CUDA 87
6.9.1.3. Python 87
6.9.2. Entornos de trabajo 90
6.9.2.1. Plataformas de desarrollo 90

7. Análisis de software **92**

7.1. Metodología de desarrollo 92

7.2. Reglas de negocio 94

7.3. Requerimientos 94

7.3.1. Requerimientos funcionales 94

7.3.2. Requerimientos no funcionales 95

7.3.3. Requerimientos de sistema 95

7.3.3.1. Recomendado 95

7.3.3.2. Mínimo 96

7.4. Análisis de riesgos 96

7.4.1. Identificación de riesgos 96

7.4.1.1. Riesgos técnicos 96

7.4.1.2. Riesgos operativos 97

7.4.1.3. Riesgos de requerimientos 97

7.4.1.4. Riesgos de estimación 97

7.4.1.5. Riesgos de alcance 97

7.4.2. Probabilidad e impacto 98

7.4.3. Plan de mitigación 100

7.4.4. Plan de contingencia 101

7.5. Estudio de factibilidad 103

7.6. Casos de uso 104

7.7. Diccionario 104

7.7.1. Entradas 104

7.7.2. Mensajes 105

7.7.3. Interfaces gráficas 105
7.8. Especificaciones de casos de uso 106
7.8.1. Controles de simulación 106
7.8.1.1. UC01 Iniciar simulación 106
7.8.1.2. UC02 Pausar simulación 107
7.8.1.3. UC03 Aumentar velocidad 108
7.8.1.4. UC04 Reducir velocidad 109
7.8.1.5. UC05 Simular un paso 110
7.8.1.6. UC06 Retroceder un paso 111
7.8.1.7. UC07 Reiniciar simulación 113
7.8.2. Configuración del autómata celular 114
7.8.2.1. UC08 Definir vecindad 114
7.8.2.2. UC09 Agregar estado 117
7.8.2.3. UC10 Definir condición de frontera 118
7.8.2.4. UC11 Definir numero de estados 119
7.8.2.5. UC12 Definir función de transición 121
7.8.3. Interacción con el espacio celular 122
7.8.3.1. UC13 Seleccionar color 123
7.8.3.2. UC14 Cambiar color de estado 124
7.8.3.3. UC15 Cambiar estado de una célula 125
7.9. Diagramas de secuencia 126

8. Diseño de software **129**

8.1. Arquitectura 129
8.1.1. Vista 129

8.1.2. Modelo 130
8.1.3. Controlador 130
8.2. Diagrama de clases 130
8.3. Interfaces de usuarios 131

Referencias **134**

Capítulo 1

Introducción

1.1. Resumen

En el presente documento se presenta la primera etapa de una investigación orientada a la simulación de sistemas complejos en espacios bidimensionales mediante autómatas celulares neuronales. El objetivo principal es utilizar una red neuronal para definir la función de transición de un autómata celular, con el fin de modelar y simular las representaciones de sistemas complejos sobre una red regular bidimensional.

Los resultados incluyen el desarrollo del marco teórico, que recopila el conjunto de definiciones formales necesarias; el estado del arte, donde se analizan investigaciones previas relacionadas con diferentes enfoques y configuraciones; la descripción del marco metodológico, en donde se especifican las técnicas, herramientas y procedimientos utilizados; y los avances obtenidos hasta el momento, que abarcan la implementación y documentación de herramientas para la simulación de autómatas celulares en dos dimensiones.

1.2. Planteamiento del problema

La simulación de sistemas complejos ha surgido como una herramienta indispensable para la comprensión y el análisis de fenómenos en diversas áreas del conocimiento, desde la física y la biología hasta las ciencias sociales y económicas. Tradicionalmente, los autómatas celulares han servido como un modelo matemáti-

co fundamental para estudiar estos sistemas mediante la representación de procesos dinámicos.

Los autómatas celulares son un modelo matemático definido como una 4-tupla que contiene:

- Un espacio de celdas en un dominio discreto.
- Un conjunto de estados posibles.
- Una vecindad que define las dependencias entre células.
- Una función de transición que determina cómo evolucionan simultáneamente las células [1, 2].

Ejemplos clásicos de autómatas celulares incluyen el Juego de la Vida de John Conway y los 256 autómatas elementales descritos por Stephen Wolfram. Estas herramientas han sido aplicadas en múltiples contextos como la simulación de incendios, modelado de distribuciones poblacionales, simulación de partículas y el análisis de ciclos climáticos [3]. No obstante, una limitación crítica de los modelos convencionales reside en la rigidez de sus funciones de transición; estas deben definirse de manera estática y anticipada, restringiendo la posibilidad de adaptarse a condiciones imprevistas o cambios en el entorno. Esta limitación motiva la búsqueda de nuevas metodologías capaces de incorporar mayor flexibilidad y adaptabilidad en la simulación de sistemas dinámicos.

En este contexto, se busca la integración de redes neuronales en el marco de los autómatas celulares. Las redes neuronales, inspiradas en el funcionamiento del cerebro y su capacidad de aprendizaje, ofrecen la posibilidad de redefinir las reglas de transición de forma dinámica. Esto permite que el autómata celular se ajuste de manera autónoma a partir de la experiencia y el entrenamiento [4].

El presente proyecto de investigación propone superar las limitaciones de los autómatas celulares mediante la integración de redes neuronales en la definición de las reglas de transición, dando origen a los *autómatas celulares neuronales*. En este enfoque, las reglas de actualización dejan de ser fijas y adquieren la capacidad de aprender y adaptarse a partir de los datos, lo que permite que el sistema se ajuste de forma dinámica ante comportamientos complejos y entornos variables [5, 6]. Esta aproximación no solo busca ampliar el campo de aplicación de los autómatas celulares, sino que también busca abrir nuevas oportunidades para modelar y simular fenómenos de formas realistas.

Dadas las infinitas configuraciones de autómatas celulares, este proyecto se limita a un espacio bidimensional finito (con condiciones de frontera) y un conjunto finito y numerable de estados. Adicionalmente, se busca experimentar con diferentes vecindades, incluyendo Moore y Neumann. Aunque el objetivo no es determinar las condiciones para la evolución de las células, sí se busca realizar pruebas bajo diferentes configuraciones que permitan mostrar los resultados sin comprometer su calidad.

Por otro lado, la red neuronal recibe como parámetros los estados de las celdas vecinas y devuelve el nuevo estado de la celda a la cual se aplique la función de transición. Este proceso es el mismo para todas las células.

Es por esto que surge la pregunta central de esta investigación: ¿Cómo los autómatas celulares neuronales pueden simular sistemas complejos en un espacio bidimensional? A partir de este cuestionamiento, surgen las siguientes preguntas de investigación.

1.2.1. Preguntas de investigación

- ¿Qué es un sistema complejo?
- ¿Qué es simular?
- ¿Cómo se simula un sistema complejo?
- ¿Qué es un autómata celular?
- ¿Qué es una red neuronal?
- ¿Qué es un autómata celular neuronal?
- ¿Cómo simular un sistema complejo?

1.3. Objetivo

Desarrollar un sistema de simulación basado en autómatas celulares neuronales bidimensionales que permita analizar la generación, mantenimiento, recuperación y propagación de patrones de información en entornos complejos y dinámicos.

1.3.1. Objetivos específicos

- Diseñar e implementar un simulador de espacios discretos bidimensionales que sirva de plataforma para estudiar los autómatas celulares neuronales.
- Entrenar y evaluar el desempeño de los autómatas celulares neuronales bajo diferentes configuraciones.
- Replicar configuraciones clásicas con comportamientos y soluciones conocidas.
- Analizar y comparar diferentes comportamientos con base en la red neuronal establecida.

1.4. Hipótesis

Si un sistema de simulación está basado en autómatas celulares neuronales bidimensionales, entonces es posible analizar y reproducir la generación, mantenimiento y propagación de patrones de información en entornos complejos y dinámicos.

1.5. Justificación

Los autómatas celulares son modelos lo bastante simples para permitir un análisis teórico riguroso, pero lo suficientemente complejos para exhibir fenómenos emergentes. Definir las reglas que describen comportamientos erráticos o no lineales de forma sincronizada resulta una tarea compleja [7].

Por esta razón, este proyecto de investigación nace de la necesidad de superar las limitaciones de los modelos tradicionales de autómatas celulares, que se basan en reglas de transición fijas y ya establecidas. Este enfoque limitado, aunque ha sido útil para entender algunos fenómenos, no siempre refleja la complejidad y la capacidad de adaptación de los sistemas dinámicos que encontramos en la naturaleza, en la sociedad y en la tecnología.

Cada autómata celular resuelve un problema particular; sin embargo, su definición matemática implica la existencia de una infinidad de posibles configuraciones y comportamientos. Por ello, llevar a cabo ajustes manuales hasta obtener el resultado deseado se convierte en un proceso computacionalmente inviable [8].

Se sugiere usar redes neuronales para definir las reglas de transición, con el fin de crear un modelo de autómata celular neuronal que pueda aprender y adaptarse por sí mismo a los cambios en su entorno. Esto no solo amplía el alcance y la aplicabilidad de los autómatas celulares, sino que también abre un novedoso campo de estudio en el que se combinan herramientas de inteligencia artificial y simulación de sistemas complejos.

La principal innovación de este enfoque reside en reemplazar las funciones de transición estáticas por modelos ajustables. Usando técnicas de aprendizaje automático, se puede entrenar la función de transición con datos representativos. Esto permite que el sistema se ajuste por sí solo a los cambios en el entorno [9].

La elección de redes neuronales para modelar la función de transición no es arbitraria. Varios estudios recientes han mostrado que las redes neuronales, sobre todo las convolucionales y recurrentes, son muy efectivas para aproximar funciones no lineales y entender relaciones complejas en el espacio y el tiempo [10, 11]. Estas características las hacen particularmente adecuadas para emular las dinámicas locales y temporales propias de los autómatas celulares. Este proyecto se basa en una línea de investigación establecida que intenta unir la capacidad expresiva de las redes neuronales con la estructura organizada y distribuida de los autómatas celulares. Así, se busca crear un modelo más flexible y que se pueda aplicar en diferentes situaciones.

Adicionalmente, este proyecto profundiza en técnicas de simulación, modelado matemático y redes neuronales, integrando conocimientos de ciencias de la computación, vida artificial y matemáticas.

La experiencia adquirida durante el desarrollo e implementación de estos modelos abrirá puertas a futuras investigaciones y proyectos en el campo de la inteligencia artificial. Además, la creación de un entorno de código abierto contribuirá al avance del estado del arte y facilitará que otros investigadores puedan extender y aplicar estos conceptos.

En resumen, este proyecto tiene como objetivo ayudar en el desarrollo de la simulación de sistemas complejos. Para lograrlo, se creará un modelo de autómatas celulares neuronales que sea flexible, adaptable y que se pueda ampliar. Esto unirá la teoría de sistemas dinámicos con las técnicas actuales de aprendizaje automático.

1.6. Alcances y limitaciones

Alcances

- El estudio se limita a la simulación en espacios bidimensionales finitos con condiciones de frontera establecidas en una red regular definida.
- Se emplean redes neuronales supervisadas para la definición de la función de transición.
- Se analizarán comportamientos de patrones conocidos y emergentes bajo distintas configuraciones de estados y vecindades.

Limitaciones

- No se contempla la generalización del modelo a diferentes dimensionalidades o topologías de red.
- El análisis se centra en la simulación de sistemas complejos, sin abordar la optimización computacional del entrenamiento.
- Los resultados se interpretan únicamente dentro del contexto de los parámetros y condiciones experimentales definidos.

Capítulo 2

Marco teórico: Teoría de sistemas, caos y simulación

La teoría general de sistemas es un enfoque interdisciplinario que se utiliza para comprender la complejidad bajo diferentes contextos y tiene dos pilares fundamentales: todo puede ser concebido como un sistema y las conexiones entre las partes son esenciales para su funcionamiento.

Independientemente de la naturaleza o su escala, cualquier entidad o fenómeno puede entenderse como un conjunto de elementos interconectados que trabajan en conjunto para lograr un objetivo. Por otro lado, las diferentes interacciones que se dan entre estos elementos, ya sea de materia, energía o información, son las que permiten que funcione un sistema [12].

Como no es práctico hacer experimentos en la naturaleza, que se define como todo proceso que se puede observar [13], necesitamos modelos que nos ayuden a probar cómo responde un proceso a ciertos estímulos que son importantes para el observador. En este capítulo se introducen elementos de la teoría general de sistemas y cuáles son los métodos y técnicas para la simulación de sistemas complejos.

2.1. Sistemas

Antes de adentrarnos en la definición de sistemas complejos y simulación, es necesario conocer qué es proceso, modelo y sistema.

Definición 1. *Un proceso es cualquier cosa que se encuentre en la naturaleza y*

que tiene un propósito que depende del observador.

Un proceso es un ente real que mapea diferentes estímulos o entradas en salidas o respuestas a esos estímulos.

Definición 2. *Un modelo es cualquier abstracción de un proceso que realiza un observador.*

Los modelos son inherentes al proceso original que los generó, es decir, se puede utilizar el mismo modelo para múltiples procesos que comparten ciertas propiedades. Por ejemplo, un modelo que examina las fuerzas que interactúan en un objeto que se encuentra en caída libre no depende de especificaciones del objeto como su color o su forma.

Los modelos permiten probar las respuestas de un proceso a estímulos de interés para el observador. Siguiendo el ejemplo de caída libre, no es necesario soltar múltiples objetos de diferentes alturas para identificar las fuerzas que están interactuando; únicamente es necesario probar las diferentes entradas dentro del mismo modelo. Además, la calidad de un modelo se determina si las salidas producidas son similares a las que produciría el proceso.

Finalmente, un sistema es un conjunto de partes interrelacionadas que actúan de manera coordinada y cumplen una función en específico. O dicho de otro modo:

Definición 3. *Un sistema es un modelo matemático que cumple con ciertos axiomas que lo caracterizan.*

De acuerdo con [14], los sistemas se caracterizan principalmente por

- Estar compuestos por elementos internos independientes que pueden organizarse en subsistemas.
- Operar de manera organizada para cumplir un objetivo.
- Tener elementos que se puedan distinguir como parte del sistema o del ambiente.
- Existir en un marco temporal.

2.1.1. Clasificación de sistemas

Los sistemas se pueden clasificar según su relación con el entorno, su origen, composición, comportamiento, etc. Estas son las clasificaciones que puede tener:

2.1.1.1. Sistemas naturales y artificiales

Un sistema es natural si no requiere de intervención humana para ser creado; son regidos por leyes físicas, químicas, biológicas y psicológicas. El sistema solar, el sistema respiratorio o el sistema cardiaco son algunos ejemplos.

Por otro lado, los sistemas artificiales son aquellos creados por los humanos y que cumplen un propósito. Estos tienen un dominio en el conocimiento, la ingeniería y las interacciones sociales. Por ejemplo, las calculadoras, el sistema educativo o las relaciones comerciales [15].

2.1.1.2. Sistemas continuos y discretos en el tiempo

Se dice que un sistema es continuo en el tiempo si sus entradas y salidas son señales continuas en esa variable.

Por otro lado, es discreto si transforma señales discretas en el tiempo en señales de salida discretas en el tiempo [16].

2.1.1.3. Sistemas estáticos y dinámicos

Un sistema estático o sin memoria es aquel que permanece en un estado constante o inmutable, es decir, no cambia con el tiempo. Se dice entonces que un sistema es estático si su salida en cualquier instante de tiempo depende solamente de entradas en ese mismo momento y, de acuerdo con [17], estos se caracterizan por:

- Inmutabilidad: No cambia con el tiempo y mantiene su estado constante.
- Estabilidad: Es sencillo de predecir y controlar su comportamiento.
- Simplicidad: Usualmente son más sencillos en diseño y operación, lo que facilita su implementación.

Por otro lado, un sistema dinámico o con memoria es aquel que cambia con el tiempo. Estos tienen la capacidad de adaptarse y responder ante diferentes condiciones de entrada. En otras palabras, es aquel cuya salida en un instante de tiempo depende de entradas en otros instantes y posiblemente de entradas en ese mismo instante. Este se caracteriza por:

- Cambio en el tiempo: Se adapta ante diferentes situaciones y entradas.
- Flexibilidad: Responde ante diferentes condiciones y requisitos.
- Complejidad: Son más complejos en diseño y operación debido a su capacidad de cambio.

Aunque en la naturaleza no podemos encontrar procesos estáticos, en ocasiones resulta impráctico modelarlos como sistemas dinámicos [12].

Este es un ejemplo de un sistema estático:

$$y[n] = 2x[n] + 2$$

Ya que en cualquier instante de tiempo particular n_0, el valor de $y[n]$ depende solamente del valor de $x[n]$ en ese mismo instante de tiempo n_0.

Por otro lado, este es un ejemplo de un sistema dinámico:

$$y[n] = x[n] + x[n-1]$$

Ya que el valor de $y[n]$ en el tiempo n_x depende del de $x[n]$ en n_1 y en n_{n-1}.

2.1.1.4. Sistemas deterministas y estocásticos

Un sistema determinista es un sistema en donde las mismas entradas producirán invariablemente las mismas salidas. No se contempla la existencia del azar, variables aleatorias ni principio de incertidumbre.

Por otro lado, un sistema estocástico es aquel que se rige por el azar. El resultado depende de probabilidades que cambian o pueden variar en función del tiempo [18].

No hay que confundir los sistemas estocásticos con los sistemas caóticos. En los sistemas estocásticos existe una variable aleatoria que impide que las mismas entradas tengan las mismas salidas. Los sistemas caóticos son deterministas, pero determinar el sistema en evoluciones futuras resulta complicado.

2.1.1.5. Sistemas abiertos y cerrados

Un sistema es abierto si puede intercambiar materia, energía o información con su entorno. Tienen una interacción activa con el mundo que los rodea o con su ambiente. Estas influencias externas pueden ajustar las respuestas del propio sistema y se caracterizan por:

- Adaptabilidad: Pueden ajustar el comportamiento y estructuras internas para mantener su funcionamiento y equilibrio en respuesta a perturbaciones externas.
- Homeostasis dinámica: Pueden mantener un equilibrio interno a lo largo del tiempo.
- Entradas y salidas: Sus entradas adquieren información del entorno y las salidas le devuelven una respuesta.

Por su parte, los sistemas cerrados se encuentran aislados de su entorno. Tiene límites que actúan como barreras para el intercambio de materia, energía o información. Estos operan de forma autocontenida y no tienen una interacción con el entorno que los rodea.

Siguiendo a [19], todas las fluctuaciones y cambios dependen únicamente de los elementos que se encuentren dentro del sistema. Estos sistemas tienden a mantener su estructura y equilibrio interno de manera constante; esto se debe principalmente a que:

- La frontera impide interacción significativa con el entorno, por lo que no se ve afectado el funcionamiento interno.
- Conservan la energía y la materia que los contienen, lo que contribuye a la estabilidad de la estructura y el equilibrio.

2.2. Propiedades fundamentales de los sistemas

Las propiedades de los sistemas dependen de su dominio. El dominio de un sistema es el campo sobre el cual se extiende su influencia [20]. A continuación se listan las propiedades fundamentales de los sistemas.

2.2.1. Causalidad

Un sistema es causal si su señal de salida en cualquier momento depende solamente de las entradas en ese instante específico o en instantes anteriores. Por ejemplo, el siguiente es un sistema que depende de entradas futuras:

$$y(t) = x(t+1)$$

Los sistemas anticausales tienen aplicaciones en imágenes, reconocimiento de voz, datos geológicos, etc. En la naturaleza, todos los procesos son causales [12].

2.2.2. Invertibilidad

Un sistema es invertible si a distintas entradas le corresponden distintas salidas, es decir, si podemos definir las entradas conociendo la salida del sistema.

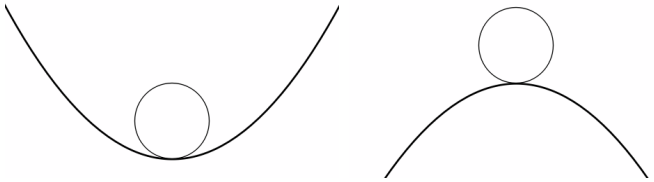
Si el sistema es invertible, podemos definir un sistema inverso que reciba como entrada la salida del sistema original y devuelva la entrada. Por lo que, si conectamos el sistema inverso a la salida del sistema original, obtendremos como resultado la entrada [12].

2.2.3. Estabilidad

Se denomina punto de equilibrio a un sistema dinámico a aquel en que la variable de interés permanece constante con todas sus derivadas iguales a cero.

Supóngase que se tiene un sistema dinámico autónomo, es decir, sin señales de entrada, y que este se encuentra en un punto de equilibrio. Se define como un punto de equilibrio estable si, al dejar evolucionar libremente el sistema a partir de una condición inicial en una vecindad del punto de equilibrio, el sistema regresa a dicho punto. Sí, por el contrario, el sistema se aleja, entonces es un punto de equilibrio inestable [12].

En la figura 2.1 se ilustran ambos puntos de equilibrio. Si una pelota se coloca en una vecindad del punto de equilibrio, en el caso 2.1a, la pelota regresará eventualmente al punto de equilibrio, mientras que en el caso 2.1b la pelota se alejará.



(a) Punto de equilibrio estable.

(b) Punto de equilibrio inestable

Figura 2.1: Puntos de equilibrio.

Se dice que un sistema es BIBO (*Bounded input bounded output*) estable si a entradas acotadas corresponden salidas acotadas.

Retomando el ejemplo de la figura 2.1, si se aplica un desplazamiento hacia la derecha, en el caso 2.1a, corresponde a una pequeña entrada a la que se le asigna una pequeña salida; por otro lado, si se le aplica una pequeña entrada al caso 2.1b, la diferencia de la respuesta del sistema es mucho más grande [12].

2.2.4. Invariante en el tiempo

Se dice que un sistema es invariante en el tiempo si un corrimiento en el tiempo en la señal de entrada causa el mismo corrimiento en el tiempo de la señal de salida del sistema. Si $y(t)$ es la salida de un sistema cuando la entrada es $x(n)$, entonces $y(t - t_0)$ es la salida cuando se aplica la señal de entrada $x(t - t_0)$ [12].

2.3. Sistemas dinámicos

Como ya se mencionó, un sistema dinámico es aquel en el cual existe un almacenamiento de energía, materia e información, es decir, es un sistema que posee memoria.

La evolución de un sistema se lleva a cabo con base en estados: el sistema comienza en un estado x_0 y la evolución al estado siguiente depende del estado anterior; además, existe una ley que rige su evolución, y, conociendo esta ley, es posible determinar cualquier estado posterior [21].

De acuerdo con [22], podemos definir a un sistema dinámico determinista de la siguiente manera.

Definición 4. *Un sistema dinámico determinista se define como una colección* $D =< T, U, \Omega, Y, Q, \Delta, \Lambda >$ *donde:*

- T *es la base del tiempo (el conjunto de valores posibles para la variable* t*).*
- U *es el conjunto de valores posibles de la entrada.*
- Ω *es el conjunto de segmentos admisibles de entrada (sus elementos son funciones* $\omega : [t_1, t_2] \subset T \to U$*).*
- Y *es el conjunto de valores de salida.*
- Q *es el conjunto de valores posibles de los estados.*
- $\Delta : Q \times \Omega \to Q$ *es la función de transición de estados.*
- $\Lambda : Q \to Y$ *es la función de salida.*

La función de transición Δ se caracteriza por:

- Dado un segmento de entrada admisible $\omega : [t_1, t_2] \subset T \to U$ y sabiendo que el estado en t_1 es $q(t_1) \in Q$, entonces $q(t_2) = \Delta(q(t_1), \omega)$. Es decir, Δ transforma el estado inicial y la entrada en el estado final.
- Dados dos segmentos de entrada admisibles consecutivos $\omega_1 : [t_1, t_2] \to U$ y $\omega_2 : [t_3, t_4] \to U$, y asumiendo que la concatenación $\omega_1 \circ \omega_2 \in \Omega$, debe cumplirse que $\Delta(q, \omega_1 \circ \omega_2) = \Delta(\Delta(q, \omega_1), \omega_2)$. Esta propiedad garantiza que la evolución del sistema sea compatible con el procesamiento secuencial de entradas.

2.4. Teoría del caos

De acuerdo con [23], el caos trata con sistemas cuyo comportamiento es impredecible a largo plazo. Suele ser asociado a sistemas impredecibles o que no tienen un orden, pero en teoría del caos se refiere a todos aquellos sistemas deterministas cuya predicción en instantes de tiempo futuro se vuelve impredecible.

Las condiciones iniciales suelen contener errores que incrementarán exponencialmente con el tiempo, lo que se conoce como sensibilidad a las condiciones iniciales, haciendo imposible la predicción a largo plazo. Estos se conocen como sistemas caóticos.

Definición 5. *Sea X un espacio métrico y sea $\phi : X \to X$ una aplicación continua. Se dice que un sistema es caótico si cumple las siguientes condiciones:*

- *La aplicación $\phi : X \to X$ es topológicamente transitiva.*
- *El conjunto de puntos de ϕ es denso en el atractor.*
- *La aplicación $\phi : X \to X$ tiene dependencias sensitivas a las condiciones iniciales.*

El hecho de que la aplicación sea topológicamente transitiva implica que el atractor caótico es irreducible, es decir, si iteramos la función ϕ un número determinado de veces (evolucionar el sistema durante un periodo de tiempo suficiente), se puede acceder a cualquier punto del atractor desde cualquier otro.

Si el conjunto de puntos es denso en el atractor, entonces cualquier punto en el espacio está arbitrariamente cerca de una órbita periódica.

Finalmente, la sensibilidad a las condiciones iniciales refleja el hecho de que en un sistema caótico, dos trayectorias que inician relativamente cerca acabarán comportándose de una forma muy diferente a largo plazo.

2.4.1. Exponentes de Lyapunov

Los exponentes de Lyapunov miden la sensibilidad de las condiciones iniciales y cuantifican la estabilidad o inestabilidad de las trayectorias en el espacio de fases al sufrir una perturbación ligera. Supongamos que tenemos dos trayectorias muy próximas separadas por una distancia $\delta\epsilon_0$; entonces la distancia entre ellas diverge de la forma:

$$\delta_\epsilon(t) \sim \delta\epsilon_0 e^{\lambda t} \tag{2.1}$$

Donde λ es el exponente de Lyapunov. Es decir, mide la tasa de separación entre dos trayectorias muy cercanas en el espacio de fases. En un espacio de fases n-dimensional, existen n exponentes de Lyapunov, lo que se conoce como espectro de Lyapunov.

2.4.2. Atractores

En un espacio disipado, la región del espacio de fases que atrae a las trayectorias vecinas se conoce como atractor. Es un subconjunto en el espacio de fases, es decir, un conjunto de estados a los que tiende a evolucionar el sistema en función de las condiciones iniciales.

Definición 6. *Sea X el espacio de fases de un sistema dinámico disipado y sea $\phi_t : X \to X$ la transformación que describe el movimiento. Se dice que $A \subset X$ es un atractor si verifica las siguientes condiciones:*

- *$\exists U$ entorno de A, llamado cuenca de atracción de A, tal que $\forall V$ abierto con $A \subset V$, existe un T_0 tal que $\phi_t(U) \subset V, \forall t > T_0$.*
- *$\phi_t(A) = A, \forall t > 0$*
- *$\nexists S \subset A$ subconjunto propio no vacío que satisfaga las dos propiedades anteriores.*

Es decir, un punto que se encuentra en el atractor no puede salir de ahí y, además, existe una órbita alrededor del atractor en donde, para cualquier tiempo, la trayectoria permanece en la órbita.

2.4.2.1. Tipos de atractores

Existen cuatro tipos de atractores:

1. Atractores de punto fijo: Se dice que un punto fijo es estable si las trayectorias próximas a él se mantienen próximas bajo la evolución. Y existe un atractor si dichas trayectorias acaban convergiendo a ese punto.
2. Atractores de ciclo límite: Las trayectorias vecinas se acercan o se alejan del ciclo límite en forma de espiral. Se dice que un ciclo es estable o inestable en función de si las trayectorias se acercan o se alejan, respectivamente. Existen las semiestables, en donde algunas trayectorias se acercan y otras se alejan.
3. Atractores cuasiperiódicos: Cuando dos ciclos límites con diferentes trayectorias se superponen, las trayectorias se enroscan en las direcciones longitudinales y latitudinales del toro con diferentes frecuencias.

4. Atractores extraños: Se dice que un atractor es extraño si tiene dimensión fractal (patrón de autosimilitud a diferentes escalas). Además, el atractor extraño es caótico si se verifica que las trayectorias que describe el sistema se ven atraídas por el atractor, pero son inestables en él, es decir, muestran sensibilidad a las condiciones iniciales.

2.5. Sistemas complejos

Son sistemas con muchas componentes que interactúan entre sí mediante el intercambio de materia, energía o información y que, eventualmente, sufren la acción de agentes externos [24]. Las colonias de hormigas, el mercado bursátil, el clima, el sistema nervioso, las células y los organismos vivos son algunos ejemplos de sistemas complejos. Estos se caracterizan por tener un conjunto de elementos que interactúan entre sí con el fin de alcanzar un objetivo [25, 26].

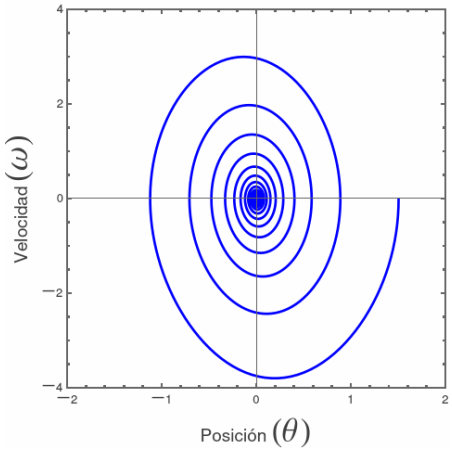
Los sistemas complejos tienen dos implicaciones fundamentales: (1) Existe una influencia mutua entre sus elementos, donde el cambio afecta inevitablemente al resto de elementos, y (2) persiguen un objetivo [26].

Definición 7. *Un sistema complejo es una entidad compuesta de partes interconectadas, de tal forma que el comportamiento colectivo es más que la suma de sus componentes individuales.*

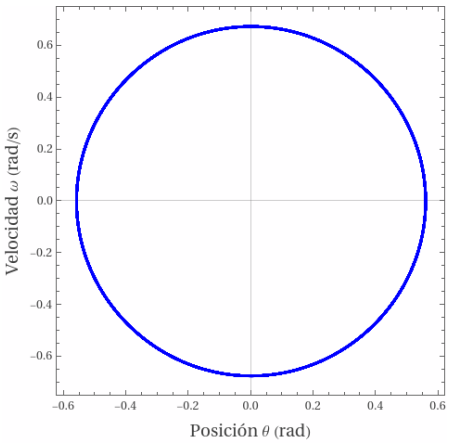
Para entender el comportamiento de un sistema es necesario conocer los elementos y sus interacciones. Es decir, se requiere conocer los estados, valores instantáneos de todos los elementos; y sus transiciones, cambios dinámicos en los estados.

Los sistemas complejos se caracterizan principalmente por:

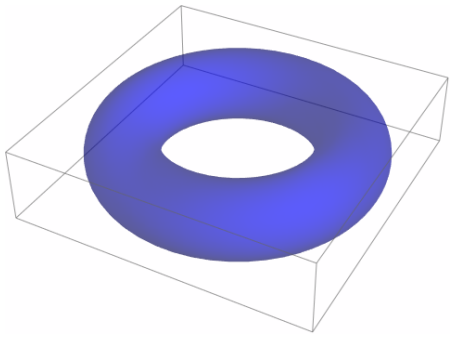
- **No linealidad**: No cumple el principio de superposición, es decir, el resultado no puede expresarse como la suma de las respuestas individuales a cada estímulo por separado. Resulta difícil predecir su evolución dinámica futura [26, 27].
- **Retroalimentación**: Las salidas pasadas influyen en las siguientes salidas. Este comportamiento hace alusión a la memoria del sistema [27].



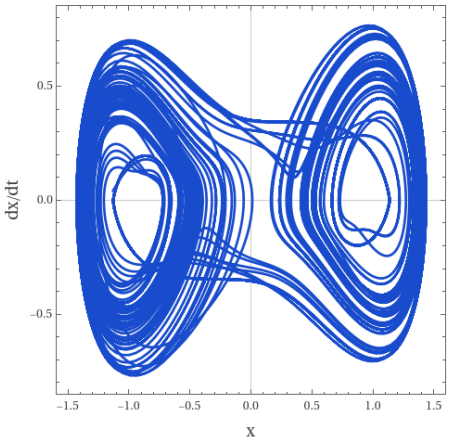
(a) Punto fijo: Relación entre posición y velocidad de un péndulo con fricción y resistencia al aire.



(b) Ciclo límite: Movimiento oscilante de un reloj de cuarzo.



(c) Cuasiperiódico: Junta dos atractores de ciclo límite formando un atractor de tipo toro.



(d) Extraño: Atractor de Duffing, describe el comportamiento complejo de un sistema dinámico no lineal.

Figura 2.2: Ejemplos de tipos de atractores.

- **Orden espontáneo**: Existe un orden emergente resultado de las interacciones locales no coordinadas, el cual no puede ser descrito mediante sus interacciones locales [27].

  Aparición (*emergence*): Entidades, patrones, comportamientos y propiedades emergen de la interacción de entidades más pequeñas, las cuales no exhiben estos atributos [27].

  Elementos múltiples: Compuesto de múltiples elementos relativamente idénticos con interacciones locales [26].

2.6. Modelado de sistemas complejos

Resulta evidente que predecir el comportamiento de estos sistemas es una tarea complicada. Una aproximación es el uso de modelos científicos y matemáticos, los cuales se construyen para desarrollar procesos de inferencia sobre aspectos observados en la vida real. El modelo de inferencia se realiza para entender el sistema real observado.

El proceso, de manera simplificada, se compone de tres fases, como se observa en la figura 2.3:

- Se abstrae el sistema real a un modelo, el cual puede trascender a otros sistemas reales.
- El modelo realiza un proceso de inferencia que busca averiguar las implicaciones derivadas de las premisas.
- Los resultados se utilizan para analizar, interpretar y aplicar a un sistema real. Esta aplicación no requiere ser práctica, es una relación entre las inferencias obtenidas con el modelo y el sistema real.

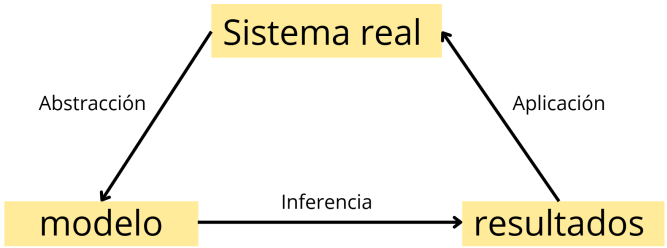


Figura 2.3: Representación gráfica del modelado científico. Este proceso busca relacionar eventos del mundo real utilizando un modelo que realiza una inferencia.

Sin embargo, estos modelos suelen carecer de las herramientas necesarias que les permiten deducir las implicaciones lógicas necesarias. Es aquí donde la computación permite agilizar este proceso realizando simulaciones computacionales.

Las computadoras permiten explorar y analizar modelos formales para resolver modelos matemáticos. Además, dado que un modelo computacional puede ser definido mediante un modelo matemático, el rol de la computación es simplificar este proceso.

Los resultados que se obtienen al ejecutar la simulación computacional se derivan de aplicar reglas algorítmicas, por lo que se requiere expandir el proceso de abstracción y diseñar un modelo formal que será ejecutado por la computadora, como se muestra en la figura 2.4.

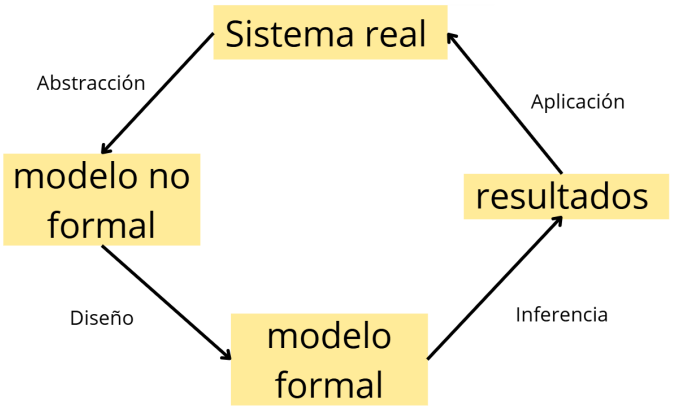


Figura 2.4: Representación gráfica del modelado computacional. Relaciona eventos del mundo real utilizando un modelo computacional que ayuda a agilizar el proceso de inferencia.

Se requiere de un experto en el sistema real que pueda abstraer el problema y un modelador que formalice dicha abstracción para ejecutarla en una computadora que infiera los resultados, mismos que utilizará nuevamente el experto en el sistema real para entablar la relación con el sistema real [28].

Para modelar sistemas complejos existen múltiples herramientas; una de ellas son los autómatas celulares [29], los cuales siguen la caracterización de los sistemas complejos: elementos idénticos con interacciones locales que pueden generar orden espontáneo, comportamientos emergentes e incluso retroalimentados.

Capítulo 3

Marco teórico: Autómatas celulares

Un autómata celular, concebido por John von Neumann [30], es una colección de células que pueden adquirir un estado y que se agrupan en una red. El autómata celular evoluciona en pasos de tiempo, donde cada célula cambia su estado en función de los estados de su vecindad. Además, durante la evolución cambia de una configuración a otra [31, 32, 33].

Tradicionalmente, se considera una red regular de células, un conjunto finito de estados y evoluciones en pasos discretos de tiempo.

Definición 8. *Un autómata celular es una 4-tupla* $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ *donde:*

- $\mathcal{L} \subseteq \mathbb{Z}^D$ *es el espacio celular D-dimensional. Un conjunto de células se coloca en el espacio* $\mathcal{L}$ *para formar una red.*
- $\mathcal{S}$ *es un conjunto de estados. Cada célula solo puede adquirir un valor de un conjunto finito de diferentes valores:*

$$\sigma_{\vec{i}}(t) \in \mathcal{S} \equiv \{0, 1, 2, \ldots, k-1\},\ \forall \vec{i} \in \mathcal{L}$$

  Donde $\vec{i} \in \mathcal{L}$ *es una célula en el espacio celular,* $\sigma_{\vec{i}}(t)$ *es el estado de la i-ésima célula en el tiempo t y* $\mathcal{S}$ *es un anillo conmutativo finito*[1]*, generalmente definido por los enteros módulo k* $(\mathbb{Z}_k)$*.*

[1]Un anillo conmutativo finito es un conjunto con suma y multiplicación, donde la suma se comporta como una suma normal (conmutativa e inversa), la multiplicación también es conmutativa y ambas operaciones cumplen la propiedad distributiva.

- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_m)$ *es el conjunto de vectores de desplazamiento que definen la vecindad de una célula en* $\mathcal{L}$*. Así, los vecinos de las células ubicadas en* $\vec{i} \in \mathcal{L}$ *se encuentran en las posiciones* $\vec{i} + \vec{v}_j, \forall j \in \{1, 2, \ldots, m\}$
- $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ *es la regla local del autómata:*

$$\sigma_{\vec{i}}(t+1) = \mathcal{R}(\{\sigma_{\vec{i}+\vec{v}_j}\}_{j=1}^{m})$$

donde $\mathcal{R}$ *es la función de transición local que asigna el nuevo estado de una célula a partir de los estados de sus* m *vecinos.*

Esta formulación sigue de manera general a [34, 35, 36].

3.1. Vecindad

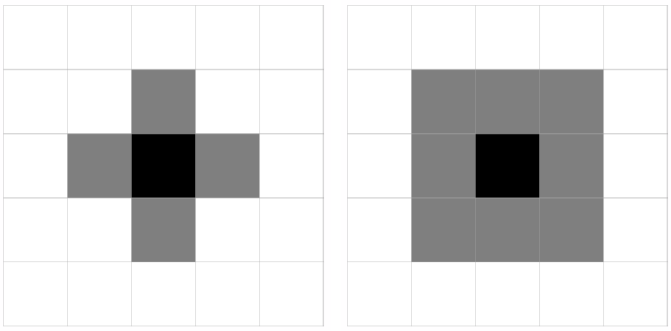
La vecindad de una célula está estrechamente relacionada con la dimensionalidad del autómata celular. John von Neumann propone una vecindad para $\mathbb{Z}^2$ de cinco células formada por ella misma y las cuatro células ortogonales más cercanas [37] como se observa en la figura 3.1a. Esta vecindad es expandida por Moore añadiendo los cuatro vecinos diagonales más cercanos, dando una vecindad de nueve elementos [38]. Esta vecindad, representada por la figura 3.1b, fue utilizada por John Conway en el "Juego de la Vida" [39].

La figura 3.1c representa una vecindad de radio uno en un espacio unidimensional, es decir, todas aquellas células a una distancia menor o igual a uno, incluyéndose a su misma. Fue utilizada por Stephen Wolfram para definir los Autómatas Celulares Elementales [40].

Las vecindades no solo son cuadradas; también pueden tener otras formas, como hexágonos o triángulos, que se muestran en las figuras 3.2a y 3.2b. Además, existen vecindades con comportamiento dinámico, como la vecindad de Margolus mostrada en la figura 3.2c, cuya configuración alterna entre dos disposiciones en cada paso evolutivo [41].

3.2. Estados

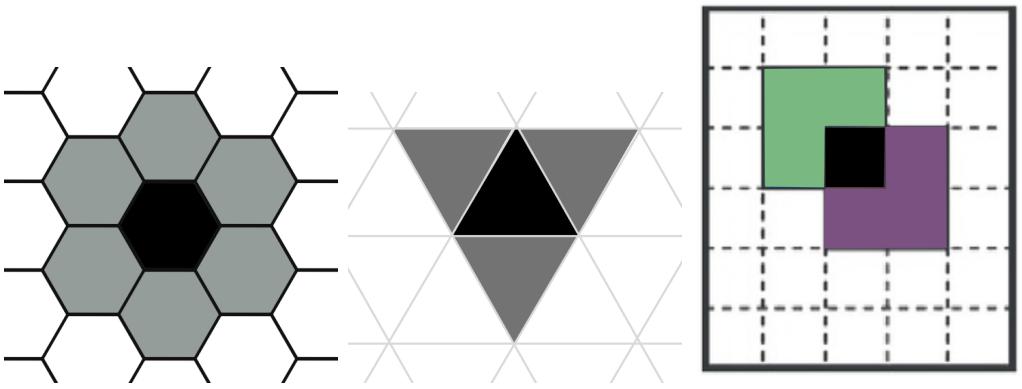
Cada célula adquiere un único estado s en cada paso de tiempo, donde s pertenece a un conjunto de estados $s \in \mathcal{S}$. Por ejemplo, el autómata celular de John von



(a) Vecindad de Neumann (b) Vecindad de Moore

(c) Vecindad de radio uno

Figura 3.1: Las vecindades 3.1a y 3.1b son para CA bidimensionales; la 3.1c es para CA unidimensionales. La célula negra representa la célula actual y, junto con las grises, conforma su vecindad.



(a) Vecindad hexagonal (b) Vecindad triangular (c) Vecindad de Margolus

Figura 3.2: Vecindades para CA bidimensionales. En 3.2a y 3.2b, la célula negra es la actual y las grises conforman su vecindad. En 3.2c, la célula negra es la actual, mientras que las células verdes y moradas representan las vecindades activas en tiempos pares e impares, respectivamente.

t	$\vec{\sigma}(t)$										
0				$\ldots$	q	p	q	$\ldots$			
1			$\ldots$	q	p	p	p	q	$\ldots$		
2		$\ldots$	q	p	p	p	p	p	q	$\ldots$	
3	$\ldots$	q	p	p	p	p	p	p	p	q	$\ldots$

Tabla 3.1: Evolución temporal de un autómata celular unidimensional con una configuración inicial finita. Donde p representa estados activos y q estados inactivos. Los conjuntos $\vec{\sigma}(t)$ y $\vec{\sigma}(t+1)$ son finitos.

Neumann contempla 29 estados, aunque algunos autores han propuesto variantes reducidas a cuatro estados, conservando su capacidad de autoreproducirse [34].

Por otro lado, el juego de la vida se compone de dos estados que denominó "vivas" y "muertas" [39]. De igual forma, Stephen Wolfram utiliza dos estados en los autómatas celulares elementales [40].

Idealmente, el espacio celular $\mathcal{L}$ es infinito, lo que permitiría infinitas combinaciones de estados. Sin embargo, esto es inviable computacionalmente, por lo que, al igual que el autómata celular de John von Neumann, se requiere establecer una restricción de finitud, como lo marca [34].

Definición 9. *Sea $\mathcal{R}$ la regla local de un autómata celular. Un estado $\sigma_q \in \mathcal{S}$ es inactivo si $\mathcal{R}(\sigma_q, \sigma_q, \ldots, \sigma_q) = \sigma_q$.*

Es decir, si una célula y todos sus vecinos están en estado inactivo, la célula permanece inactiva tras aplicar $\mathcal{R}$. Cabe notar que todos los vecinos deben estar en dicho estado, por lo que, si alguno cambia, podría reactivarse.

Para trabajar computacionalmente, [34] define las configuraciones finitas de la siguiente manera:

Definición 10. *Sea $\sigma_q \in \mathcal{S}$ un estado inactivo, $\sigma_p \neq \sigma_q \in \mathcal{S}$ un estado no inactivo y sea $\mathcal{L}$ un espacio celular infinito. Se considera que un autómata celular tiene configuración finita si solo un número finito de celdas están en estados activos σ_p.*

Si la configuración inicial es finita, entonces la evolución en el siguiente paso de tiempo también será finita (ver tabla 3.1).

Sea $\mathcal{C} = \mathcal{S}^{\mathcal{L}}$ el conjunto de todas las configuraciones. Definimos $\mathcal{C}_F \subset \mathcal{C}$ como el subconjunto de todas las configuraciones finitas. Esta restricción es necesaria para hacer computacionalmente viable la simulación de la evolución.

Finalmente, resulta necesario definir un estado estable, el cual servirá más adelante para establecer una clasificación [34].

Definición 11. *Dado un autómata celular y un estado $s \in \mathcal{S}$, se dice que s es un estado estable si $\mathcal{R}(s, s, \ldots, s) = s$.*

Esto es, un estado estable s que evoluciona siempre en sí mismo, por lo que $\sigma(t) = \sigma(t+1)$.

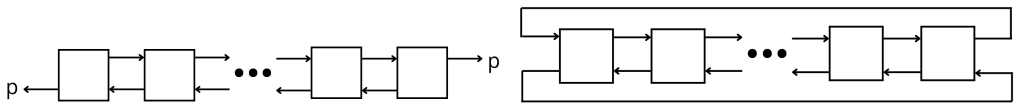
3.3. Condiciones de frontera

Idealmente, los autómatas celulares pueden evolucionar en espacios infinitos; sin embargo, para su simulación computacional es necesario establecer límites. Estos límites, llamados condiciones de frontera, restringen el espacio celular $\mathcal{L}$ [34, 42].

Definición 12. *Un autómata celular es finito si el espacio celular $\mathcal{L} \subseteq \mathbb{Z}^D$ es finito; en caso contrario, se denomina infinito.*

En un autómata celular finito se consideran principalmente dos tipos de condiciones de frontera: abierta y periódica [42].

- **Condición de frontera abierta**: Se les asigna un estado fijo a todos aquellos vecinos faltantes en las células de la frontera.
- **Condición de frontera periódica**: Los vecinos faltantes de las células en la frontera son otras células en la frontera. La figura 3.3b muestra cómo se conectan las células de la frontera en un espacio unidimensional.



(a) Condición de frontera abierta (b) Condición de frontera periódica

Figura 3.3: Condiciones de frontera con dos vecinos. La figura 3.3a asigna un estado no inactivo al vecino faltante. La figura 3.3b entabla relación de vecindad entre las células en la frontera, formando un anillo.

Las condiciones de frontera periódica conectan a los autómatas celulares finitos con los infinitos que tienen configuraciones que se repiten [34].

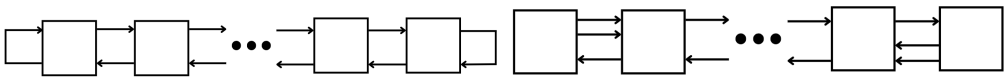
Definición 13. *Una configuración $\vec{\sigma} \in \mathcal{C}$ de un autómata celular D-dimensional se dice periódica si es invariante ante un conjunto de traslaciones linealmente independientes en D direcciones. Es decir:*

$$\vec{\sigma} = T_i^{n_i}(\vec{\sigma}),\ \forall i \in \{1, 2, \ldots, D\}$$

donde $T_i^{n_i}$ denota un desplazamiento de n_i posiciones en la dimensión i.

Definimos entonces a $\mathcal{C}_P \subset \mathcal{C}$ como el conjunto de todas las configuraciones periódicas. Estas configuraciones preservan la condición de frontera periódica en cada traslación [34].

La frontera reflectiva asigna a los vecinos faltantes el mismo estado de la célula en la frontera (figura 3.4a). En este caso, las células de la frontera son vecinas de sí mismas. La frontera reflexiva, en cambio, imita el estado de los vecinos más próximos de la célula en el borde (figura 3.4b), generando una doble relación de vecindad en los extremos [34, 42].



(a) Condición de frontera reflectiva (b) Condición de frontera reflexiva

Figura 3.4: Condiciones de frontera con dos vecinos. La figura 3.4a muestra que las células en la frontera son vecinas entre sí, mientras que la figura 3.4b indica que las células que están al lado de las células de la frontera pueden tener varios vecinos.

3.4. Regla local

Sea $\mathcal{S}$ el conjunto de estados y $m = |\mathcal{N}|$ el número de vecinos. La función de transición o regla local se define como

$$\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$$

donde $\mathcal{R}$ asigna un nuevo estado a cada célula en función de los estados de sus m vecinos (incluyéndose a sí misma si aplica).

Para autómatas celulares unidimensionales, dado un rango arbitrario r, la regla puede ser escrita como:

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \ldots, \sigma_i(t), \ldots, \sigma_{i+r}(t)),\ \mathcal{R} : S^{2r+1} \to S$$

Donde el rango r especifica el tamaño de la vecindad para cada célula, la cual puede adquirir un valor k. Por esto, $\mathcal{R}$ puede ser definido especificando el valor asignado para cada una de las k^{2r+1} posibles $(2r+1)$-tuplas dada una vecindad.

Un ejemplo de regla local es el juego de la vida [39]. En este caso, el conjunto de estados es binario $\mathcal{S} = \{0, 1\}$, donde 0 representa células muertas y 1 las vivas. La regla de transición se define como:

$$\mathcal{R} : \{0, 1\}^9 \to \{0, 1\}$$

Ya que la vecindad de Moore en 2D involucra 8 vecinos más la propia célula, es decir, $m = 9$. El juego de la vida sigue las siguientes tres reglas:

- **Nacimiento**: Una célula muerta en el tiempo t nace en el tiempo $t+1$ si tiene exactamente tres vecinos vivos en el tiempo t.
- **Muerte**: Una célula viva en el tiempo t muere en $t+1$ por:
  - sobrepoblación; si hay más de tres vecinos vivos en t.
  - soledad: si hay menos de tres vecinos vivos en t.
- **Supervivencia**: Una célula viva en el tiempo t se mantiene viva en $t+1$ si tiene dos o tres vecinos vivos en t.

Una regla es semitotalística si la transición depende de la suma de los valores en la vecindad, pero distingue entre diferentes posiciones, como el centro y los bordes. El juego de la vida es un ejemplo de regla semitotalística porque cuenta el número de vecinos vivos, pero trata de forma distinta el estado de la célula central [43].

En cambio, una regla es totalística si la transición depende únicamente de la suma de los estados en todos los vecinos, sin distinguir posiciones [44]. Por ejemplo, para un autómata celular unidimensional con $|\mathcal{S}| = 3$ estados y radio $r = 1$, solo hay $(|\mathcal{S}| - 1)|\mathcal{N}| + 1 = (2-1) \cdot 3 + 1 = 4$ posibles valores de suma.

111	110	101	100	011	010	001	000
↓	↓	↓	↓	↓	↓	↓	↓
$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$

Tabla 3.2: Cada posible combinación de tres células puede adquirir uno de dos valores. Cuando la célula evaluada encaja en este patrón, su valor cambia al establecido por la regla. Este conjunto de 256 forma parte de los autómatas celulares elementales.

Por otro lado, una regla local arbitraria sí distingue las posiciones de cada elemento en la vecindad. La configuración dada para el autómata celular elemental considera $k = 2$ (es decir, $\sigma_i \in \{0,1\}$), un radio $r = 1$ ($|\mathcal{N}| = 3$) y la regla $\mathcal{R} : \{0,1\}^3 \to \{0,1\}$ que se define como:

$$\phi(\sigma_{i-1}(t), \sigma_i(t), \sigma_{i+1}(t))$$

Esta dinámica está dada por las posibles configuraciones que las tres células adyacentes pueden adquirir en el tiempo t. Podemos encontrar $k^{|\mathcal{N}|} = 2^3 = 8$ patrones posibles en la vecindad, número correspondiente a la combinatoria de dos estados en tres posiciones. Cada vez que encontremos un patrón, podemos asignarle un nuevo estado a $\sigma_i(t+1)$ ($\{0,1\}$), por lo que el número total de reglas asciende a $k^{k^{|\mathcal{N}|}} = 2^8 = 256$, como se muestra en la tabla 3.2.

Estas reglas se pueden representar de forma tabular y también como un número decimal. La tabla 3.3 muestra la asociación entre la forma tabular y la regla decimal.

Nótese que la regla decimal solo aplica cuando las condiciones son las mismas. Es decir, cuando el número de vecinos $|\mathcal{N}|$ y estados $|\mathcal{S}|$ son invariantes; a cada regla se le puede asociar un número entero único, pero dos reglas que difieren en vecindad o estados pueden compartir el número decimal. Por ejemplo, considerando la regla 10 con $|\mathcal{N}| = 2$ y $|\mathcal{S}| = 2$ es 1010, la cual es completamente diferente cuando $|\mathcal{N}| = 3$ y $|S| = 2$, lo que da como resultado 000000101.

Binario	111	110	101	100	011	010	001	000	
Decimal	7	6	5	4	3	2	1	0	**regla**
Valor	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	1	1
					⋮				⋮
	0	0	0	1	1	1	1	0	30
					⋮				⋮
	0	1	1	0	1	1	1	0	110
					⋮				⋮
	1	1	1	1	1	1	1	1	255

Tabla 3.3: Los valores binarios representan las posibles configuraciones de estados que pueden aparecer en el espacio, cada uno asociado a un valor decimal, y el valor es la potencia asociada a esa combinación. Para convertir la regla tabular en la decimal, solo es necesario convertir el número binario.

3.5. Ejemplos

3.5.1. John Von Neumann

El matemático y físico John von Neumann (1903-1957) fue uno de los pioneros en el estudio de los autómatas celulares, especialmente en lo que atañe a la reproducción automática y la universalidad computacional. En su obra póstuma, *Theory of Self-Reproducing Automata*, presentó un modelo de autómata celular bidimensional diseñado para construir máquinas autorreplicantes.

A continuación se presentan los principales elementos de su autómata celular, adaptados al formato de definición formal de este tipo de autómata:

- **Espacio celular:** Von Neumann definió un espacio bidimensional como una cuadrícula regular de celdas, idealmente infinita.
- **Estados:** En su versión original, cada célula podía adoptar uno de 29 estados posibles que permiten distinguir células en reposo, células en transmisión de "señal", células constructoras, etc.
- **Vecindad:** Utiliza la vecindad que más tarde se denominaría "Vecindad de von Neumann": la célula central más sus cuatro vecinos ortogonales [45].

- **Función de transición:** Todas las células actualizan su estado de forma síncrona, de acuerdo con una regla común que depende del estado actual de la propia célula y de los estados de sus vecinos. En el modelo de von Neumann, esta regla está diseñada de modo que permita que ciertas configuraciones de células funcionen como un constructor universal capaz de autorreplicarse.

El objetivo era mostrar de manera formal que se puede crear un autómata celular que se reproduzca sin fin y que, además, pueda hacer construcción universal (ser un constructor universal) dentro de ese sistema celular [46, 47].

3.5.2. El juego de la vida

El juego de la vida es un autómata celular propuesto por el matemático John Horton Conway en 1970. Consiste en una cuadrícula de celdas que pueden estar en uno de dos estados posibles: vivas o muertas. A lo largo del tiempo, las celdas evolucionan de acuerdo con un conjunto de reglas locales que determinan su supervivencia, muerte o nacimiento, en función del número de vecinos vivos que posean [48].

Siguiendo la definición formal de un autómata celular, el *Juego de la Vida* se caracteriza por los siguientes elementos:

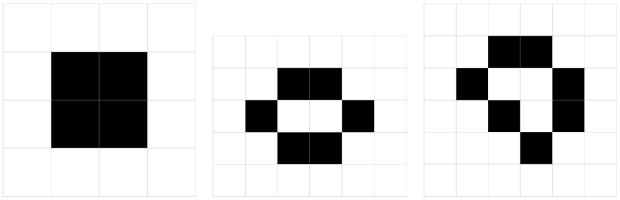
- **Espacio celular**: Se define sobre una cuadrícula bidimensional regular, teóricamente infinita.
- **Estados**: Cada célula puede encontrarse en uno de los estados $\{\text{muerto}, \text{vivo}\}$, representados comúnmente como $\{0, 1\}$.
- **Vecindad**: Se utiliza una vecindad de Moore semitotalística, la cual incluye las ocho celdas adyacentes a la célula central.
- **Función de transición**: El estado futuro de cada célula depende de su estado actual y del número de vecinos vivos, siguiendo las reglas:
  - Una célula viva sobrevive si tiene dos o tres vecinos vivos.
  - Una célula viva con más de tres vecinos vivos muere por sobrepoblación.
  - Una célula viva con menos de dos vecinos vivos muere por soledad.
  - Una célula muerta con exactamente tres vecinos vivos renace.

Estas reglas pueden modificarse para crear variantes del autómata, cambiando las condiciones de nacimiento o supervivencia. En [39] se analizan distintas funciones de transición y configuraciones espaciales.

Una notación común para describir este tipo de reglas es la notación *Birth/Survival*, que especifica los números de vecinos vivos requeridos para cada caso. El juego de la vida tradicional corresponde a la configuración $B3/S23$: una célula muerta nace con tres vecinos vivos, y una viva sobrevive si tiene dos o tres vecinos vivos [39].

Dentro de este sistema emergen patrones característicos, que se agrupan en tres categorías principales:

- **Naturaleza muerta**: Estructuras estáticas que permanecen invariantes en el tiempo.



(a) *Block* (b) *Beehive* (c) *Loaf*

Figura 3.5: Ejemplos de configuraciones de naturaleza muerta.

- **Osciladores**: Patrones que repiten su configuración después de un número finito de iteraciones, manteniendo su posición en la cuadrícula.

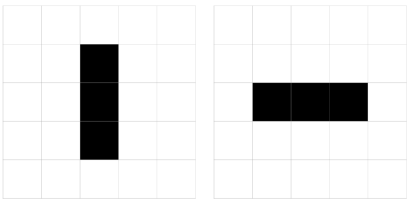


Figura 3.6: *Blinker*.

- **Naves espaciales**: Son osciladores que, además de repetir su patrón, se desplazan por el espacio celular.

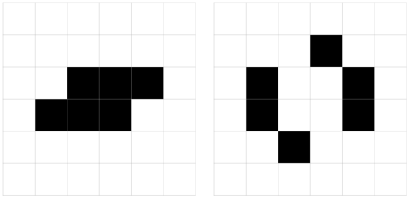


Figura 3.7: *Toad*.

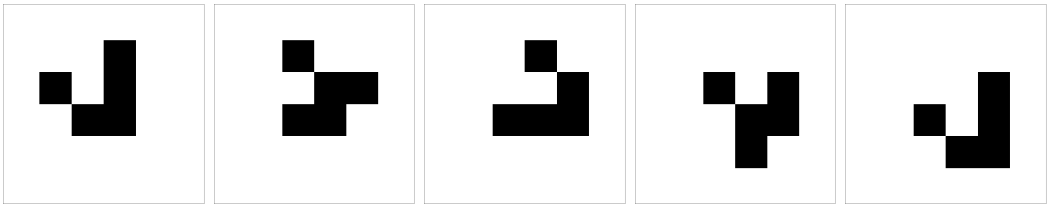


Figura 3.8: *Glider*.

Paul Rendell [49] demostró que, a partir de una configuración inicial apropiada, el *Juego de la Vida* es Turing completo, es decir, tiene la capacidad de realizar cualquier cálculo computable mediante la simulación de una máquina de Turing.

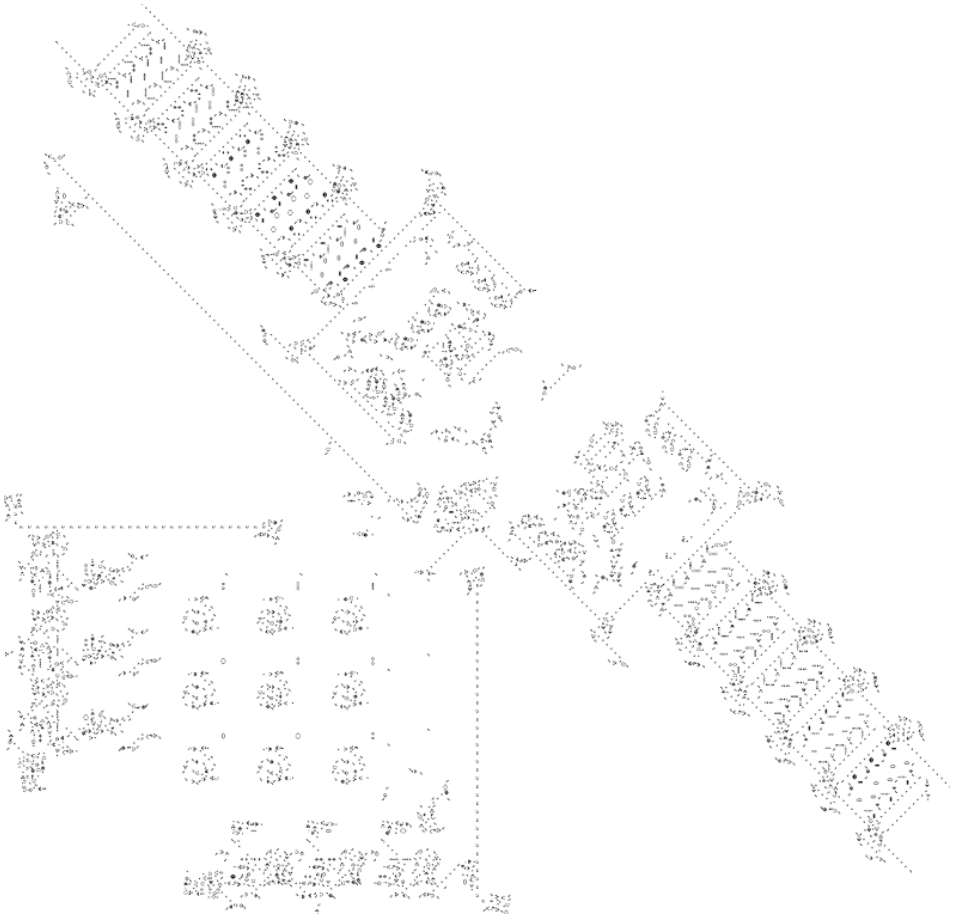


Figura 3.9: Máquina de Turing implementada en el *Juego de la Vida* [49].

Debido a su naturaleza semitotalística, la vecindad puede dividirse en dos conjuntos, y con ello determinar el número de posibles estados que puede adoptar cada uno:

Vecindad	**Estados**	**Total**
Frontera	$\{0, 1, 2, 3, 4, 5, 6, 7, 8\}$	9
Centro	$\{0, 1\}$	2

Así, puede definirse un vector de 18 elementos que representa la función de transición, como se muestra en la tabla 3.4.

Frontera	8	7	6	5	4	3	2	1	0	8	7	6	5	4	3	2	1	0
Centro	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0
$B3/S23$	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	0	0	0

Tabla 3.4: Representación binaria de la regla $B3/S23$.

De manera general, el tamaño del vector que define una regla semitotalística puede calcularse mediante la ecuación 3.1. En este caso, dicho vector tiene tamaño 18, por lo que existen $|\mathcal{S}|^{18} = 2^{18} = 262,144$ posibles reglas.

$$\prod_{i=0}^{n}(|\mathcal{S}| - 1)(|n|) + 1, \quad \forall n \in \mathcal{N} \tag{3.1}$$

3.5.3. Autómata celular elemental

Stephen Wolfram propuso una versión más simple de los autómatas celulares, llamada autómata celular elemental. En esta versión, se disminuye la dimensionalidad del espacio celular y, por lo tanto, se reduce el tamaño de la vecindad. A continuación se detallan sus características según la definición formal de autómata celular:

- **Espacio celular**: Está formado por una cadena unidimensional de celdas dispuestas linealmente.
- **Estados**: Cada célula puede encontrarse en uno de dos estados posibles, $\{0, 1\}$.
- **Vecindad**: Se define mediante un radio r que determina cuántas celdas adyacentes influyen en la evolución de cada célula. En el caso del autómata celular elemental, el radio es $r = 1$, por lo que la vecindad está compuesta por la célula central y un vecino a cada lado.
- **Función de transición**: Wolfram no propuso una única regla de evolución, sino el conjunto completo de funciones de transición posibles bajo estas condiciones.

Dado que cada célula considera tres posiciones (la propia y sus dos vecinas) y cada una puede tener dos posibles estados, existen $2^3 = 8$ combinaciones de vecindad. Por tanto, el número total de reglas posibles es $2^8 = 256$.

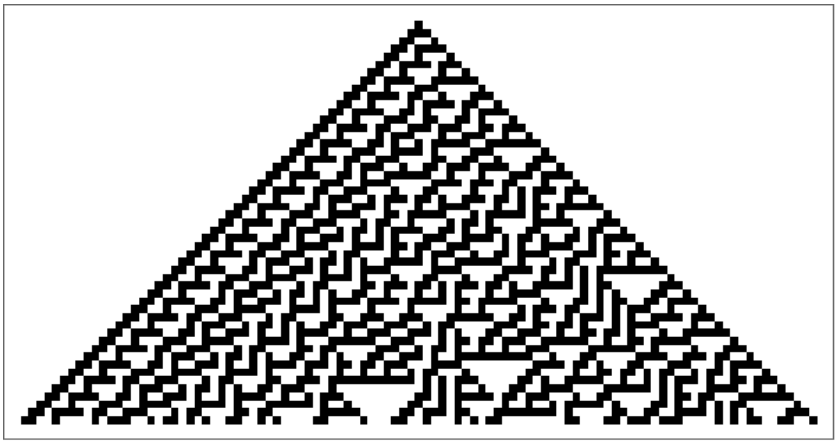
Wolfram asignó a cada una de estas funciones un número entero entre 0 y 255, conocido como número de regla, que identifica de forma unívoca el comportamiento del autómata. Además, propuso una clasificación de las reglas en cuatro categorías, según los patrones que emergen durante su evolución temporal (estas clases se detallan en la siguiente sección).

A continuación, se presentan tres reglas representativas por su relevancia teórica y sus propiedades computacionales.

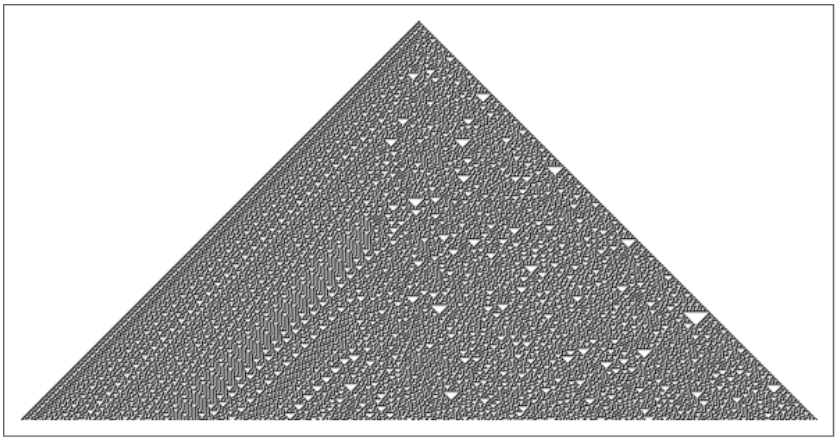
- **Regla 30**: La Regla 30 pertenece a la clase de comportamiento caótico. A partir de una configuración inicial simple, genera patrones altamente complejos y aparentemente aleatorios [50].

  Esta regla plantea tres interrogantes fundamentales sobre la naturaleza del caos y la computabilidad:

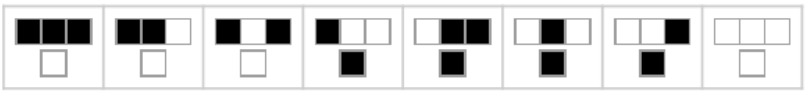
  1. ¿La columna central permanece no periódica a lo largo del tiempo?
  2. ¿Existe equilibrio estadístico entre los estados vivos y muertos?
  3. ¿Es posible calcular el estado de la célula central en el paso n en tiempo $O(n)$?



(a) 50 evoluciones.



(b) 350 evoluciones.



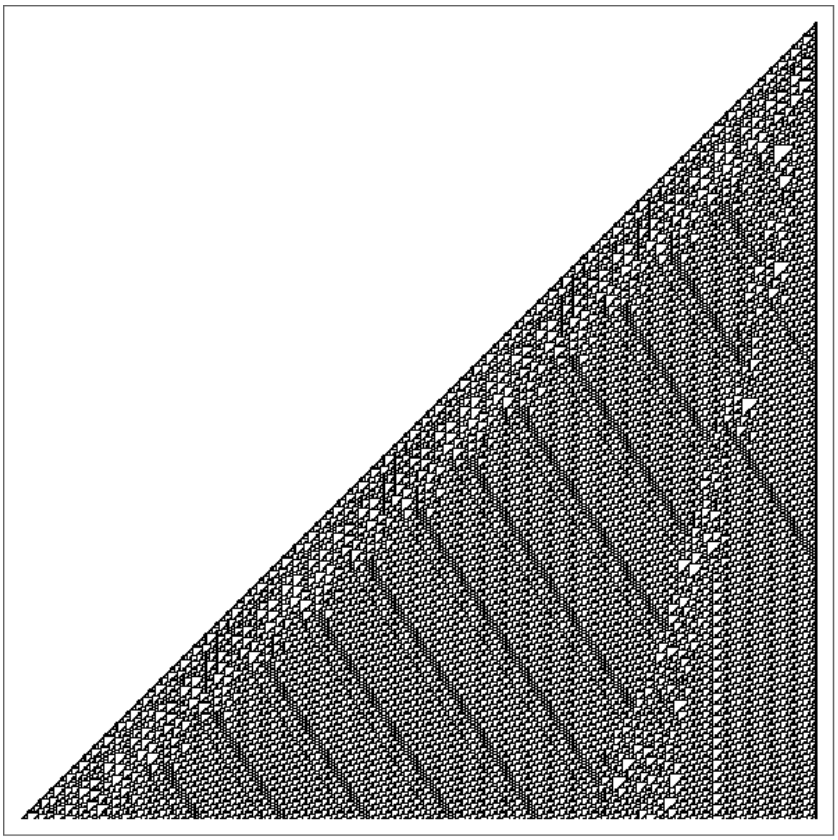
(c) Función de transición

Figura 3.10: Evolución de la regla 30

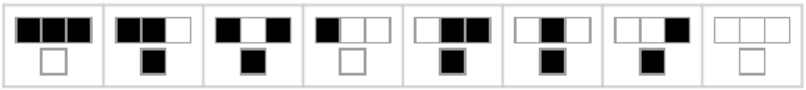
- **Regla 110**: La Regla 110 exhibe un comportamiento mixto: combina regiones periódicas y caóticas que interactúan entre sí de manera compleja. A partir de una única célula activa, genera estructuras móviles y patrones que pueden coexistir en el tiempo [51].

  Lo más notable de esta regla es su universalidad computacional. Wolfram y Matthew Cook demostraron que la Regla 110 es Turing completa, es decir, capaz de simular cualquier máquina de Turing y, por tanto, realizar cualquier

cómputo posible en un sistema discreto determinista [51].



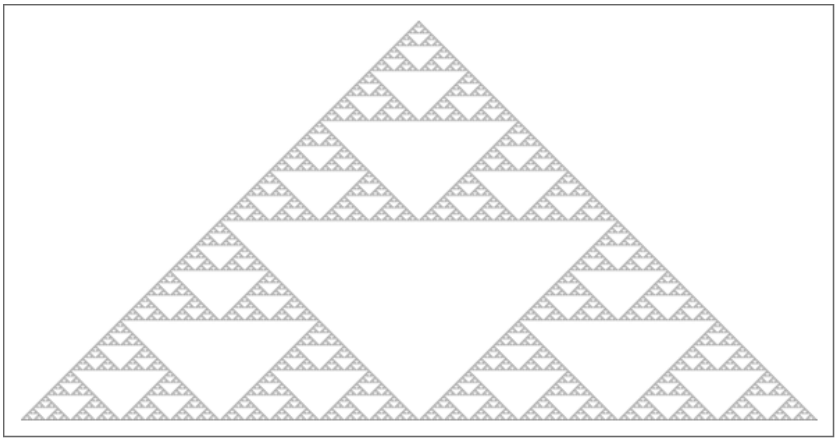
(a) 500 evoluciones.



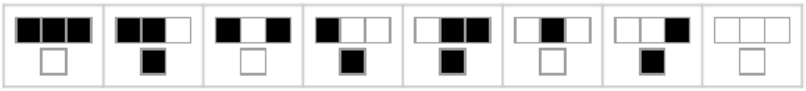
(b) Función de transición

Figura 3.11: Evolución de la regla 110

- **Regla 90**: La Regla 90 se distingue por generar estructuras fractales. Iniciando desde una única célula activa, produce el conocido triángulo de Sierpinski, un patrón autosimilar que aparece en múltiples sistemas naturales y matemáticos [52].



(a) 1025 evoluciones.



(b) Función de transición

Figura 3.12: Evolución de la regla 90

3.6. Clasificación

Los automatas celulares elementales fueron utilizados por diferentes autores por representar la maxima simplicidad y minima configuración que un automata puede adquirir y que tenga un buen rango de comportamientos. Multiples autores han intentando definir una clasificación, a continuación se muestran algunas clasificaciones propuestas por diferentes autores.

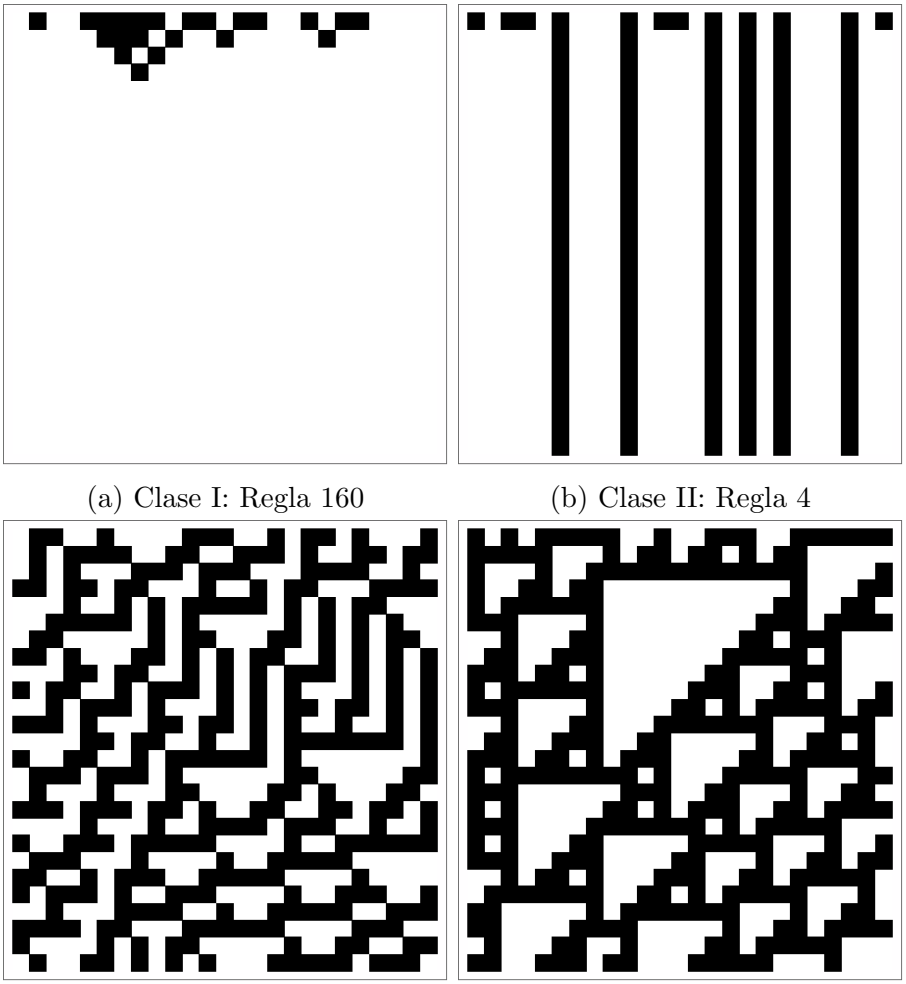
Stephen Wolfram, realiza una clasificación basada en el comportamiento que exiben las ciertas reglas y que es inherente a la configuración inicial. Es por esto que define las siguientes clases de comportamiento, ilustradas en la figura 3.13:

WI Evolucionan a un estado uniforme.

WII Evolucionan a estados cíclicos aislados.

WIII Evolucionan en comportamientos caóticos.

WIV Evolucionan en estados complejos aislados.



(a) Clase I: Regla 160

(b) Clase II: Regla 4

(c) Clase III: Regla 30

(d) Clase IV: Regla 110

Figura 3.13: Clasificación de Wolfram. Las figuras evolucionan 25 pasos una configuración inicial aleatoria con condición de frontera cíclica.

Tipo	Total	Reglas
Clase I	8	0, 8, 32, 40, 128, 136, 160, 168
Clase II	65	1, 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 19, 23, 24, 25, 26, 27, 28, 29, 33, 34, 35, 36, 37, 38, 42, 43, 44, 46, 50, 51, 56, 57, 58, 62, 72, 73, 74, 76, 77, 78, 94, 104, 108, 130, 132, 134, 138, 140, 142, 152, 154, 156, 162, 164, 170, 172, 178, 184, 200, 204, 232.
Clase III	11	18, 22, 30, 45, 60, 90, 105, 122, 126, 146, 150.
Clase IV	4	41, 54, 106, 110.

Tabla 3.5: Relación de reglas del autómata celular elemental con la clasificación de Wolfram.

En la tabla 3.5 se muestra la relación de reglas del autómata celular elemental.

Esta es la clasificación de Genaro [53].

Sin embargo existen reglas bajo diferentes configuraciones que pueden pertenecer a múltiples clases o, en el peor de los casos, no pertenecen a ninguna de ellas [54].

Li y Packard ofrecen otra clasificación que considera seis clases para categorizar el comportamiento dinámico de los autómatas celulares neuronales:

LPI La regla evoluciona a un estado homogéneo donde todas las células adquieren el mismo valor.

LPII Evoluciona hasta una configuración estática no necesariamente homogénea.

LPIII Reglas con un periodo igual a dos.

LPIV Reglas con un periodo de L.

LPV Evolucionan a estructuras caóticas

LPVI Conduce a estructuras largas y estructuras complejas.

Por otro lado, Culick y Yu [55, 56, 57] proponen una clasificación de que sigue las siguientes limitaciones:

- Esta clasificación es para autómatas celulares no totalisticos.

- Cada clase es un subconjunto de la siguiente clase.

CY I Un autómata celular es de clase I si existe un estado estable $s \in \mathcal{S}$ tal que todas las configuraciones finitas evolucionan a una configuración homogénea de s.

Es decir, dada cualquier configuración inicial en un autómata celular finito, todas las células adquieren el estado s tras cierto número de pasos evolutivos. Esto se aplica a todas las finitas configuraciones iniciales.

CY II Un autómata celular es de clase II si existe un estado estable $s \in \mathcal{S}$ tal que se repite a n pasos evolutivos.

Esto implica que la evolución se repite cada n número de pasos ($\sigma(t) = \sigma(t+n)$). Resulta evidente que la clase I está contenida en la clase II, haciendo que $n = 1$, lo que implica un periodo de uno.

Una clasificación es s-finita si solo difiere del estado estable en una región finita.

CY III Un autómata celular es de clase III si existe un estado estable $s \in \mathcal{S}$ tal que para cualquier par de configuraciones s-finitas C_1 y C_2, es decidible si C_1 evoluciona a C_2.

Es decir, si podemos determinar en tiempo finito si el CA evoluciona de C_1 a C_2, entonces pertenece a clase III. En caso contrario, pertenece a la clase IV, la cual contiene a todos los autómatas celulares [57].

CY IV Todos los autómatas celulares pertenecen a la clase IV.

3.7. Extensiones y cambios de la definición

Otra clasificación considera extensiones o modificaciones en la definición que alteran el comportamiento del autómata celular. A continuación, se describen algunas de estas extensiones para el modelado de sistemas complejos.

- **Autómatas celulares en grafos**: Amplían la idea de la estructura de red clásica al permitir que las vecindades no sean iguales: cada célula puede tener un número diferente de vecinos. Esta flexibilidad los hace adecuados para modelar fenómenos sobre grafos, como redes sociales o flujos en infraestructura. Sin embargo, diseñar manualmente las reglas de transición en

estos entornos es una tarea compleja, lo que ha motivado propuestas como los autómatas celulares neuronales en grafos, que permiten aprender dichas reglas automáticamente [58].

- **Autómatas celulares probabilísticos**: Estos extienden el modelo clásico al incorporar reglas de transición estocásticas. En vez de una evolución que sigue un camino fijo, los estados futuros dependen de probabilidades que se ven afectadas por el entorno. Esto ayuda a entender la incertidumbre, el ruido y los comportamientos que surgen de manera inesperada. Son especialmente útiles para simular dinámicas como propagación de epidemias o sistemas sociales, donde los mecanismos de transición no siempre están completamente definidos [59].

- **Autómatas celulares no homogéneos**: Permiten que diferentes células tengan diferentes funciones de transición. Para esto, se utilizan dos planos llamados "plano de datos", el cual contiene la evolución del sistema; y "plano de control", el cual define las reglas de transición para cada célula. Esto permite estudiar el comportamiento y la interacción entre diferentes reglas que comparten espacio [60].

- **Autómatas celulares continuos**: Son autómatas celulares que pueden ser continuos en los estados, en el tiempo y en el espacio [40]:

  - Los estados continuos adquieren valores reales y no solo enteros.
  - El tiempo continuo se define mediante una función que determina el intervalo de espera para actualizar la célula en función de su estado actual.
  - El espacio continuo se obtiene al asignar diferentes pesos a las células dentro de la vecindad [61].

Aunque estas variantes han ampliado el uso de los autómatas celulares, un problema que se repite es definir correctamente las reglas de transición para lograr un fenómeno específico. Este desafío ha llevado a la creación de autómatas celulares neuronales. En estos, la función de transición no se define de manera clara, sino que se aprende a través de una red neuronal artificial. Pero antes de definirlos, es necesario comprender qué son las redes neuronales.

Capítulo 4

Marco teórico: Inteligencia artificial y redes neuronales

Las redes neuronales artificiales son un modelo de inteligencia artificial inspirado en el pensamiento humano, más específicamente, en cómo un conjunto de neuronas interconectadas comparten información.

El desarrollo de las redes neuronales tiene sus orígenes en los modelos propuestos por McCulloch y Pitts en 1943, quienes establecieron las bases del perceptrón. Posteriormente, en 1957 Rosenblatt formalizó su implementación computacional. Sin embargo, el campo experimentó un periodo de estancamiento tras las críticas de Minsky y Papert en 1969, recuperando impulso en los años ochenta gracias al surgimiento de la retropropagación y de las redes recurrentes [62].

Las redes neuronales artificiales son un conjunto de neuronas artificiales interconectadas que realizan cálculos y transmiten información para realizar una operación, imitando el comportamiento del cerebro.

Pero para entender este modelo, es necesario tener los fundamentos de inteligencia artificial y la clasificación de aprendizaje supervisado.

4.1. Inteligencia artificial y aprendizaje supervisado

La inteligencia artificial fue definida por primera vez por John McCarthy en 1955 como *la ciencia e ingeniería de hacer máquinas inteligentes* [63]. Siguiendo a [63, 64, 65], se propone la siguiente definición:

Definición 14. *La inteligencia artificial es la ciencia que utiliza modelos computacionales para crear sistemas capaces de realizar tareas que, si fueran hechas por humanos, requerirían inteligencia.*

Una de las ramas de la inteligencia artificial es el aprendizaje automático, el cual se centra en desarrollar sistemas con la capacidad de aprender en función de los datos [66]. Esto permite reconocer patrones y realizar predicciones a partir de grandes cantidades de datos [67].

Por lo que, con base en [68, 69] podemos definir al aprendizaje automático de la siguiente manera:

Definición 15. *El aprendizaje automático es la disciplina que permite a las computadoras mejorar su desempeño en una tarea mediante la experiencia con datos, sin necesidad de instrucciones explícitas.*

Esta tecnología funciona trabajando a través del entrenamiento de algoritmos en conjuntos de datos para conseguir un resultado esperado. Es el proceso de optimizar el modelo para que pueda predecir la respuesta correcta. A mayor calidad en los datos de entrenamiento, mayor precisión se tendrá en el modelo [67].

El entrenamiento se refiere al proceso de enseñarle a un modelo de inteligencia artificial a realizar una o varias tareas mediante la exposición a grandes cantidades de datos [70]. Este proceso facilita a los humanos el proceso de análisis de grandes volúmenes, especialmente en la era actual, donde se generan más datos que nunca en la historia [71].

Dentro del aprendizaje automático se definen tres métodos en cómo una computadora puede aprender. Estos métodos son aprendizaje supervisado, no supervisado y por refuerzo.

- El aprendizaje supervisado se caracteriza por el uso de conjuntos de datos etiquetados para entrenar algoritmos que clasifiquen datos o predigan resultados con precisión. Durante el proceso de entrenamiento, el modelo recibe

entradas junto con las salidas correctas, lo que le permite al modelo ajustar sus parámetros hasta que sea capaz de realizar predicciones precisas sobre nuevos datos [69].

- El aprendizaje no supervisado, a diferencia del supervisado, no necesita de un conjunto de datos etiquetados generados por el ser humano. Este tipo de aprendizaje puede descubrir patrones y estadísticas sin ninguna orientación o instrucción explícita [72].
- El aprendizaje por refuerzo busca imitar la forma de aprendizaje humana de prueba y error. Este tipo de aprendizaje permite a las máquinas aprender a tomar decisiones y mejorar sus resultados con base en los errores cometidos [73].

IBM [69] menciona algunos de los algoritmos más comunes de aprendizaje automático:

1. **Redes neuronales:** Simulan el funcionamiento del cerebro humano, con un gran número de nodos de procesamiento conectados entre sí, llamados neuronas.
2. **Regresión lineal:** Predice valores numéricos, con base en una relación lineal entre diferentes valores.
3. **Regresión logística:** Realiza predicciones para variables de respuestas categóricas.
4. **Agrupación en clústeres:** Identifican patrones en los datos y los agrupan en centroides.
5. **Árboles de decisión:** Utilizan una secuencia ramificada de decisiones vinculadas que se representan con un diagrama de árbol.
6. **Bosques aleatorios:** Combina los resultados de varios árboles de decisión y predice un valor o clasificación.

Las redes neuronales presentan ventajas que pueden llegar a agilizar y optimizar recursos de las organizaciones que lo implementan [74]: mejora la productividad, reduce el error humano, mejora la disponibilidad de servicios, reduce tareas repetitivas, descubre patrones ocultos a simple vista, entre otros.

Para comprender cómo las redes neuronales logran estas capacidades, es necesario explorar primero los fundamentos que inspiraron el desarrollo. En particular, su diseño es una abstracción de las neuronas biológicas, cuya estructura y funcionamiento ofrecen una analogía al procesamiento de información.

4.2. Neurona biológica

Las neuronas son células especializadas del sistema nervioso cuyo objetivo es recibir, procesar y transmitir información por medio de señales químicas y eléctricas [75, 76].

Cada neurona se compone (principalmente) de tres elementos (como se muestra en la figura 4.1):

- Las dendritas, que reciben señales provenientes de otras neuronas.
- El núcleo o cuerpo, que integra las señales provenientes de las dendritas y produce una nueva señal eléctrica.
- El axón, el cual conduce la señal producida por el cuerpo hacia otras neuronas haciendo sinapsis, es decir, transmite la señal de una célula a otra.

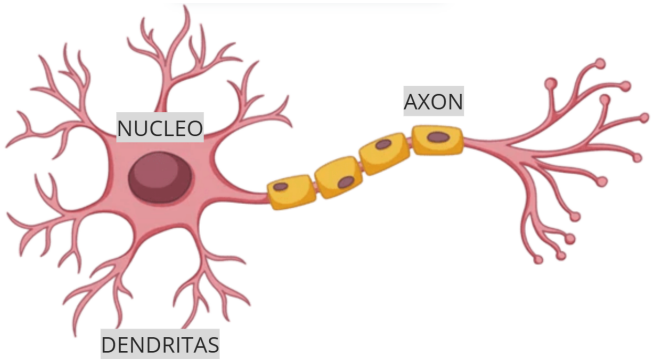


Figura 4.1: Representación de una neurona [77]. Se encuentran señaladas las dendritas, el núcleo y el axón, encargado de recibir, procesar y transmitir la información, respectivamente.

Las redes neuronales naturales son un sistema de neuronas que permite el procesamiento cerebral de la información; son altamente complejas, no lineales y con procesamiento en paralelo.

4.3. Neurona artificial

La neurona artificial se considera como una unidad de procesamiento de información [75]. Sigue la misma estructura del modelo biológico con algunas consideraciones adicionales, como lo marcan [75, 78, 79]:

- Las dendritas equivalen al conjunto de datos de entrada. Sin embargo, no toda la información tiene la misma relevancia; para esto se le asigna a cada entrada un peso sináptico.
- El cuerpo se sustituye por dos funciones: (1) Una función de agregación que combina todos los datos de entrada (generalmente es una combinación lineal entre las entradas y los pesos sinápticos) y (2) una función de activación que determina si la neurona está activa y en qué grado.

La cual se suele representar como en la figura 4.2.

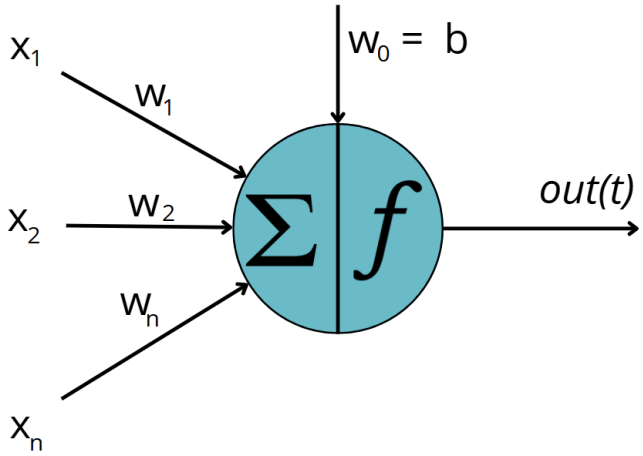


Figura 4.2: Representación gráfica de una neurona artificial. Contiene las entradas $(x_1, x_2, \ldots, x_n)$, los pesos sinápticos $(w_1, w_2, \ldots, w_n)$, el sesgo (w_0 o b), la función de agregación (Σ), la función de activación (f) y la salida.

Definición 16. *Una neurona artificial es una función* $f : \mathbb{R}^n \to \mathbb{R}$ *dada por:*

$$f(x) = \phi(\omega(X, W, b))$$

donde:

- $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}$ *es el conjunto de datos que recibe cada neurona.*
- $W = \{w_1, w_2, \ldots, w_n\} \in \mathbb{R}$ *es el conjunto de pesos sinápticos que indican la importancia de cada entrada.*
- b *o* $w_0 \in \mathbb{R}$ *es un sesgo que desplaza la función de activación y controla el umbral de aceptación.*
- $\omega(X, W, b)$ *es una función de agregación* $\omega : \mathbb{R}^n \to \mathbb{R}$ *que combina las entradas. Usualmente se define como una combinación lineal:*

$$\omega(X, W, b) = \sum_{i=1}^{n} w_i x_i + b$$

- $\phi(\omega(X, W, b))$ *es la función de activación* $\phi : \mathbb{R} \to \mathbb{R}$ *que transforma* $\omega(X, W, b)$ *en una salida activada.*

Además de la combinación lineal, la función de agregación puede adoptar otras formas, como el máximo, el mínimo o incluso combinaciones lógicas (AND, OR) en redes con entradas binarias. No obstante, la suma ponderada continúa siendo la más utilizada en aplicaciones prácticas [62].

La función de activación es la que determina el valor de salida de cada neurona. Este valor, normalmente, se asigna entre un umbral para que limiten el comportamiento de la red neuronal [80].

4.4. Redes neuronales

Una neurona únicamente puede realizar una operación. Para aumentar la capacidad de entender patrones más complejos, es necesario añadir múltiples neuronas interconectadas, lo que se conoce como red neuronal artificial [81]. Esta puede definirse como un grafo dirigido [82].

Definición 17. *Una red neuronal artificial es un grafo dirigido* $G = (V, E)$ *donde cada nodo* $v \in V$ *representa una neurona y cada arista* $(u, v) \in E$ *transporta la salida* y_u *desde* u *hasta* v *ponderada por el peso* w_{uv}*.*

Existe un conjunto de nodos de entrada $X \subseteq V$ *que reciben los datos iniciales y un conjunto de nodos de salida* $T \subseteq V$ *cuya activación final representa la salida de la red.*

La red en su conjunto implementa una función parametrizada.

$$F_\theta : \mathbb{R}^n \to \mathbb{R}^m$$

donde n es el número de datos de entrada, m el número de datos de salida y θ el conjunto de parámetros entrenables.

Por lo que podemos concluir con la definición de [83], la cual dice que

Definición 18. *Una red neuronal es una función* $F_\theta : \mathbb{R}^n \to \mathbb{R}^m$ *donde* $\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ *es el conjunto de parámetros entrenables.*

4.4.1. Funciones de activación

En esta sección se presentan las funciones de activación más comunes utilizadas en redes neuronales. Para cada función se muestra su definición matemática, una breve descripción de su comportamiento práctico y una figura ilustrativa.

4.4.1.1. Binary step (escalón binario)

La función escalón binario se define por partes:

$$f(x) = \begin{cases} 1 & \text{si } x \geq 0, \\ 0 & \text{si } x < 0. \end{cases}$$

Produce salidas discretas (0/1) y fue utilizada en perceptrones tempranos como función de decisión. No es diferenciable en el punto de umbral y su derivada es nula en casi todas partes, por lo que no es adecuada para entrenamiento mediante gradiente (backpropagation). Aun así, se usa conceptualmente para tomar decisiones discretas o como capa de postprocesado/umbral en modelos ya entrenados.

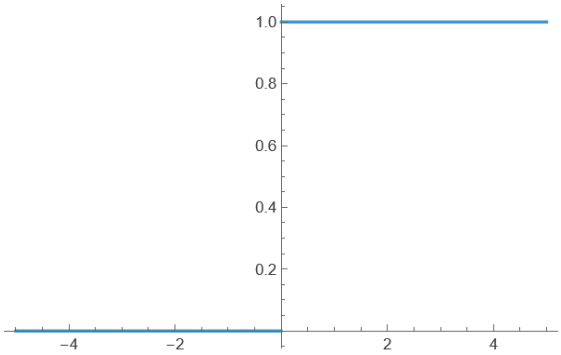


Figura 4.3: Función de activación escalón binario.

4.4.1.2. Lineal

La función lineal (o identidad) se define como:

$$f(x) = ax + b \quad \vee \quad f(x) = x.$$

Produce una salida proporcional a la entrada y mantiene la linealidad en la red. Se utiliza principalmente en la capa de salida para tareas de regresión donde se requiere predecir valores continuos. No es adecuada para capas ocultas cuando se desea que la red aprenda transformaciones no lineales.

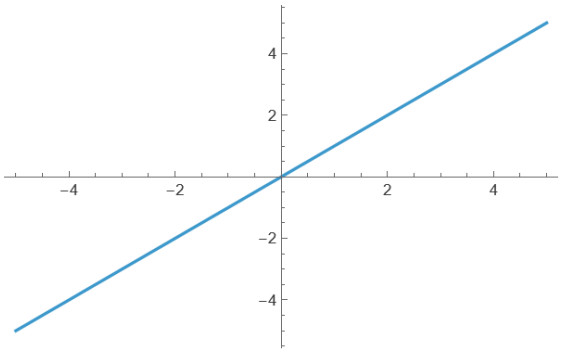


Figura 4.4: Función de activación lineal.

4.4.1.3. Sigmoide

La función sigmoide logística se define como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Tiene la propiedad de:

$$\sigma'(x) = \sigma(x)\,(1 - \sigma(x)).$$

Comprime la entrada en el intervalo $(0, 1)$, por lo que es apropiada para modelar probabilidades y se ha usado históricamente en la salida de clasificadores binarios. Sufre saturación cuando los valores son muy grandes, lo que provoca gradientes muy pequeños (esto se llama el problema del gradiente desvanecido) y una salida que no está centrada (es positiva). Por eso, en las capas ocultas modernas, a menudo se reemplaza por ReLU o tanh.

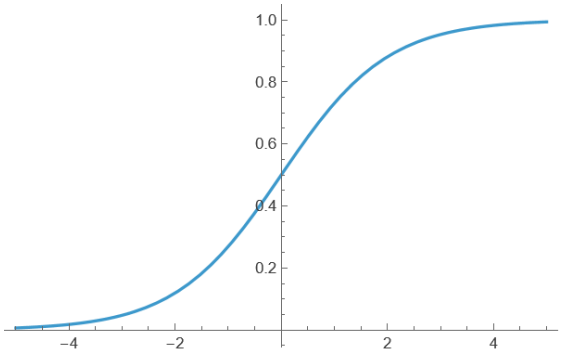


Figura 4.5: Función de activación sigmoide.

4.4.1.4. tanh (hiperbólica)

La tangente hiperbólica se define como:

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}.$$

Su rango es $(-1, 1)$ y su derivada puede expresarse en función de sí misma.

Es una versión centrada en cero de la sigmoide, lo que facilita el aprendizaje (gradientes con medias cercanas a cero). Se usa en capas ocultas cuando se necesita una salida limitada y centrada. Era una opción común en problemas secuenciales

y en redes neuronales tradicionales antes de que se popularizaran arquitecturas y funciones de activación más modernas.

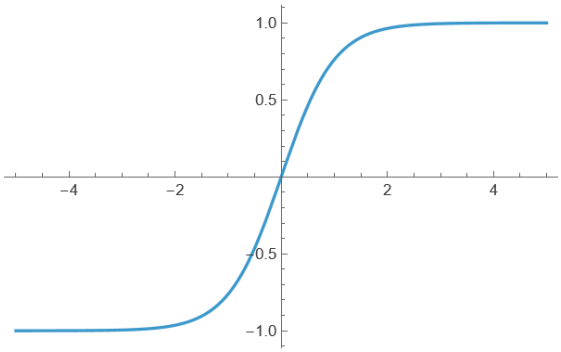


Figura 4.6: Función de activación `tanh`.

4.4.1.5. ReLU (Rectified Linear Unit)

La función ReLU se define por partes:

$$f(x) = \text{máx}(0, x).$$

Activa linealmente las entradas positivas y pone a cero las negativas. Es simple y computacionalmente eficiente, favorece la esparcidad de activaciones y reduce en la práctica el problema del gradiente desvanecido en redes profundas. Muy utilizada en capas ocultas de arquitecturas modernas. Tiene la desventaja de poder provocar neuronas "muertas" (gradiente cero para entradas negativas).

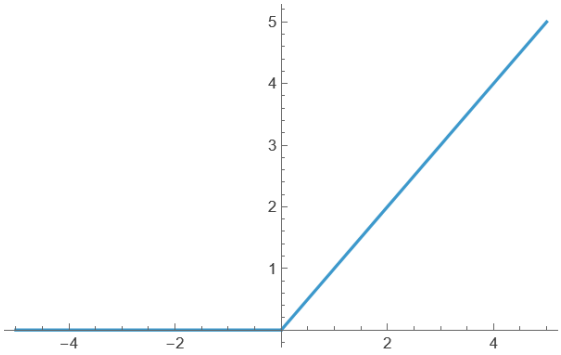


Figura 4.7: Función de activación ReLU.

4.4.2. Aprendizaje en redes neuronales

Dada una red neuronal artificial con los parámetros entrenables totalmente configurados, es capaz de realizar su clasificación o predicción de manera correcta. Sin embargo, resulta imposible configurar estos pesos manualmente, por lo que requiere de un "entrenamiento" que ayude a definir los pesos sinápticos. A continuación se define la función de pérdida, optimización y entrenamiento, elementos necesarios para obtener esa configuración específica.

En los primeros modelos la actualización de los pesos se realizaba a partir del error cuadrático del valor de activación. La regla de actualización se expresaba como:

$$\Delta w_j = -\alpha(c - a)x_j$$

Donde a representa la activación y α la tasa de aprendizaje. Este método permitió superar la no diferenciabilidad de la función escalón en los perceptrones originales [62].

4.4.2.1. Función de pérdida

La función de pérdida se usa para medir el rendimiento del modelo. Calcula la desviación que existe entre el valor predicho y el valor esperado. Esta función puede ser un escalar o un vector que permite aproximar la tasa de error para llegar a una solución óptima [84].

Definición 19. *Sea F_θ una red neuronal artificial y $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{n}$ un conjunto de datos de entrenamiento donde $x^{(i)}$ es la entrada y $y^{(i)}$ es la salida esperada; una función de pérdida es un mapeo.*

$$\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}_{\geq 0}$$

Tal que mide la discrepancia entre la predicción del modelo y la salida real:

$$\mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

Es importante que la función de pérdida sea diferenciable. El gradiente de la

función de pérdida permite conocer la dirección hacia la cual debe moverse para disminuir la pérdida.

Las funciones de pérdida se pueden utilizar para regresión y para clasificación. La primera mide el error en valores continuos, usado para estimar valores contables como el precio, tamaño, tiempo. La segunda mide los errores en valores discretos, como la clasificación o la categoría.

Diferentes funciones de pérdida priorizan diferentes tipos de error; la elección de la función de pérdida dependerá de la tarea que se desea realizar. Además, la función de pérdida únicamente se limita a una muestra; para una población de muestras, se utiliza la función de costo, la cual se define a continuación [84, 85]:

Definición 20. *Sea F_θ una red neuronal artificial y $\mathcal{L}$ un conjunto de datos de entrenamiento; la función de costo es el promedio de las pérdidas:*

$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}(F_\theta(x^{(i)}), y^{(i)})$$

4.4.2.2. Optimización

El entrenamiento de una red consiste en ajustar los parámetros entrenables θ para minimizar la función de pérdida. La optimización resulta en resolver el problema de minimización dado por:

$$\theta^* = \arg\min_\theta J(\theta)$$

Esto se puede conseguir mediante diferentes técnicas, siendo el descenso del gradiente la más común. Este método actualiza los parámetros en la dirección opuesta al gradiente de la función de pérdida:

$$\theta^{t+1} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$$

donde $\eta > 0$ es la tasa de aprendizaje. Una tasa de aprendizaje alta puede divergir en la solución, mientras que una tasa baja puede tener una convergencia lenta [83].

En la práctica, es común iniciar el entrenamiento con una tasa de aprendizaje relativamente alta y luego reducirla gradualmente a medida que el modelo

converge. También se suele incorporar un término de *momentum* para acelerar la convergencia y reducir oscilaciones [62]

4.4.2.3. Entrenamiento

Antes del entrenamiento, las variables de entrada deben normalizarse para ajustarse al rango de activación de la red (por ejemplo, $[0, 1]$ o $[-1, 1]$). Este paso evita que una variable con valores grandes domine la función de agregación y mejora la estabilidad del entrenamiento. Es importante que la normalización preserve la distribución original de los datos y la proporción entre atributos [62].

Finalmente, el entrenamiento consiste en ajustar todos los parámetros $\theta = \{W^{(l)}, b^{(l)}\}$ para minimizar la función de costos $J(\theta)$ en un conjunto de datos de entrenamiento. Este proceso se compone de las siguientes etapas:

1. Se inicializan los parámetros, generalmente de forma aleatoria: $\theta_0 \sim \mathcal{D}_{init}$
2. Se hace la propagación hacia adelante o *forward pass*, es decir, para cada entrada $x^{(i)}$ se evalúa $\hat{y}^{(i)} = F_\theta(x^{(i)})$
3. Se calcula la pérdida: $\mathcal{L}^{(i)} = \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$
4. Se calculan los gradientes propagando la información hacia atrás o *backpropagation* utilizando la regla de la cadena.

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

   Se actualizan los parámetros mediante la función de optimización: $\theta^{t+1} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$
5. Se repite este proceso sobre un *dataset* hasta la convergencia de la solución o hasta alcanzar un criterio de parada [83].

   Una vez entrenada la red, puede evaluarse la importancia relativa de las entradas mediante un análisis de sensibilidad. Este procedimiento consiste en:

   a) Calcular el valor medio de cada entrada.

   b) Evaluar la salida de la red con todos los valores medios.

c) Modificar una entrada a la vez (aumentándola o disminuyéndola) y medir el cambio en la salida.

d) Repetir el proceso para cada variable de entrada.

De esta manera, se identifica qué variables influyen más en el resultado, lo cual aporta interpretabilidad al modelo [62].

La elección de la topología de la red, en particular el número de neuronas ocultas, suele determinarse empíricamente. Una regla práctica consiste en comenzar con un número de neuronas ocultas similar al número de entradas y ajustar a partir del rendimiento obtenido. Otras heurísticas recomiendan iniciar con el doble de nodos de entrada y reducir gradualmente si se detecta sobreajuste [62].

En resumen, el entrenamiento de la red neuronal es un proceso iterativo de optimización que consiste en:

1. Propagar los datos de entrada por la red.
2. Comparar la salida con el valor real usando una función de pérdida.
3. Propagar el error hacia atrás para obtener gradientes.
4. Actualizar los parámetros con un algoritmo de optimización.
5. Repetir hasta que el modelo aproxime adecuadamente la función deseada.

4.5. Tipos de redes neuronales

Las redes neuronales pueden clasificarse atendiendo a diferentes criterios, ya sea por el número de capas, el tipo de conexiones, el grado de conectividad o la arquitectura general. En este trabajo se han seleccionado dos tipos principales, de manera enunciativa, mas no limitativa, que son especialmente relevantes para su eventual implementación en un autómata celular neuronal: las redes neuronales convolucionales y las recurrentes. La razón de esta elección es la siguiente:

Por un lado, las evoluciones de un autómata celular ocurren de forma secuencial en el tiempo, es decir, las entradas y salidas de la red corresponden a estados del espacio celular y, en algunos casos, las reglas pueden tener una relación temporal o reversible. De ahí que las redes recurrentes resulten particularmente apropiadas.

Por otro lado, como se explicará más adelante, se ha demostrado que los autómatas celulares y las redes neuronales convolucionales tienen algunas similitudes, especialmente porque el *kernel* de la red convolucional es similar a la vecindad del autómata celular.

A continuación se desarrollan ambas arquitecturas de manera formal.

4.5.1. Redes neuronales convolucionales

Las redes neuronales convolucionales constituyen una clase de redes profundas cuya estructura está diseñada para explotar la topología de los datos que tienen forma de rejilla. Por ejemplo, imágenes en dos dimensiones.

Una red neuronal convolucional consta de una capa de entrada, varias capas ocultas (que pueden incluir capas de convolución, normalización, *pooling*, activación, etc.) y una capa de salida.

Se asume un volumen de entrada con dimensiones ($H \times W \times C$), es decir, altura H, anchura W y C canales (por ejemplo, RGB en imágenes). La red maneja este volumen usando filtros deslizantes. Aunque normalmente no se describen en términos de estados discretos de autómatas, se puede entender que cada neurona de una capa representa un estado continuo real (por ejemplo, una activación $a \in \mathbb{R}$).

El elemento clave es su operación de convolución: un filtro (o *kernel*) de tamaño $k \times k$ se desliza sobre la entrada, compartiendo pesos y produciendo mapas de activación. Se puede ver como una vecindad local de la neurona que influye en su valor de activación que se aplica en cada capa de convolución:

$$Y = f(X * K + b)$$

Donde X es el volumen de entrada, K es el filtro (o filtros) de convolución, b es el sesgo y f es una función de activación. A continuación pueden seguir operaciones de *pooling* y capas totalmente conectadas para producir la salida final [86].

Se aplica principalmente en clasificación de imágenes, segmentación, reconocimiento de video y procesamiento de audio. En el contexto de los autómatas celulares, se puede ver que el filtro de la red neuronal convolucional es similar a la vecindad del autómata. Esto permite entenderlo como una operación de actualización local que se repite en un espacio organizado, igual que en un autómata celular.

4.5.2. Redes neuronales recurrentes

Las redes neuronales recurrentes son un tipo de redes que tienen conexiones que vuelven sobre sí mismas. Esto permite que la información de momentos anteriores afecte el procesamiento que se está realizando en el presente.

Una red neuronal recurrente trabaja con una serie de entradas $(x_1, x_2, \ldots, x_T)$ y genera una serie de salidas $(y_1, y_2, \ldots, y_T)$. En cada momento, la red tiene un estado oculto h_t que se actualiza de la siguiente manera:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h),$$
$$y_t = g(W_{hy}h_t + b_y),$$

Donde W_{xh}, W_{hh} y W_{hy} son matrices de pesos, b_h y b_y son sesgos, y f, g son funciones de activación.

La red funciona en una secuencia de tiempo, donde cada momento puede verse como una çapa virtual.º una repetición del autómata. En cada instante t, el estado oculto h_t muestra la "memoria.º el contexto acumulado hasta ese momento.

La vecindad temporal está dada por la conexión de la unidad de tiempo t con la anterior $t-1$ (y en variantes bidireccionales, también con $t+1$). Es decir, la salida de un paso se inserta como parte de la entrada del siguiente paso.

Como se mostró más arriba, la actualización recursiva del estado oculto actúa como la función de transición temporal de la red. La red integra información histórica para producir la salida en cada paso de tiempo.

Estas redes se utilizan frecuentemente en el procesamiento de lenguaje natural, reconocimiento de voz, series temporales, traducción automática y otras tareas en las que el orden temporal de los datos es esencial [87].

Dado que un autómata celular evoluciona en pasos discretos de tiempo y cada uno puede depender del estado previo del sistema, las redes neuronales recurrentes aportan un modelo natural para capturar dependencias temporales entre estados sucesivos del autómata. Esto resulta especialmente útil si la regla del autómata celular tiene memoria o se desea que aprenda dinámicas espacio-temporales.

Una vez comprendidas las redes neuronales, podemos retomar la definición de un autómata celular neuronal: una arquitectura que surge de la integración entre los autómatas celulares y las redes neuronales artificiales.

Capítulo 5

Marco teórico: Autómatas celulares neuronales

En los últimos años, los autómatas celulares neuronales surgieron como una extensión de los clásicos, con las capacidades de aprendizaje y generalización propias de las redes neuronales artificiales. Esta integración permite que la regla de transición, tradicionalmente fija, sea parametrizable y entrenable mediante técnicas de aprendizaje supervisado.

Las primeras aproximaciones copiaban el funcionamiento de un autómata celular usando una red neuronal convolucional [88]. Este método fue dado a conocer por Mordvintsev [10], quienes entrenaron un modelo que aprende de dinámicas locales a partir de ejemplos. Como resultado, las interacciones locales básicas cumplen un objetivo en específico, mostrando propiedades de crecimiento, regeneración, autoorganización y morfogénesis.

Los autómatas celulares neuronales son entonces un tipo de autómatas celulares, pero cuya función de transición está definida como una red neuronal artificial. Retomando las definiciones previas, podemos decir entonces que:

Definición 21. *Un autómata celular es neuronal si la regla local $\mathcal{R}$ está dada como:*

$$\mathcal{R} = F_\theta : \mathbb{R}^{|\mathcal{N}|} \to \mathbb{R}^1$$

donde:

- F_θ *es una red neuronal parametrizada por* θ*.*
- $\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ *es el conjunto de parámetros entrenables.*
- $\mathbb{R}^{|\mathcal{N}|}$ *corresponde al espacio de entrada definido por la vecindad* $\mathcal{N}$*.*
- $\mathbb{R}^1$ *define el valor de salida que representa el nuevo estado de la célula central.*

La figura 5.1 muestra la relación existente entre estas dos tecnologías. En el paso N se tiene una configuración bidimensional con vecindad de Neumann. El estado de cada elemento dentro de la vecindad funciona como valor de entrada para la red neuronal entrenada y devuelve un único resultado. Este resultado corresponde a la salida de la célula central, es decir, a la cual se le está aplicando.

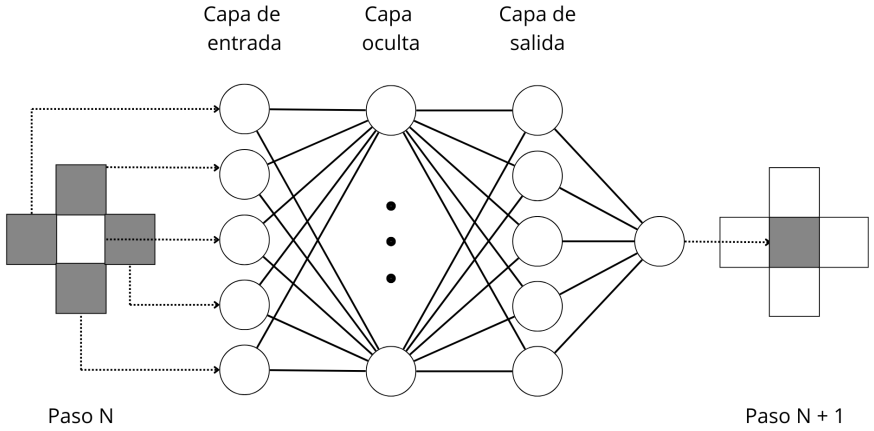


Figura 5.1: Representación gráfica de la red neuronal.

Cada célula aplica una misma red neuronal local F_θ sobre su vecindad inmediata. La entrada de la red consiste en los estados de las celdas dentro de la vecindad y la salida es el nuevo estado de la célula central.

F_θ suele representarse como una red neuronal artificial convolucional que permite procesar localmente la información espacial preservando la estructura D-dimensional.

A diferencia de los autómatas clásicos, donde las reglas son explícitas y definidas manualmente, los autómatas celulares neuronales aprenden sus reglas mediante un proceso de entrenamiento. Se establece un grupo de condiciones iniciales X_0 y un estado final X^*. Luego, la red F_θ se ajusta para reducir una función de pérdida

$\mathcal{L}(X_T, X^*)$, que mide la diferencia entre la configuración obtenida después de T pasos y el estado deseado. Este proceso puede expresarse como:

$$\theta^* = \arg\min_{\theta} \mathcal{L}(F_\theta^{(T)}(X_0), X^*).$$

Durante el entrenamiento, se emplean técnicas de retropropagación a través del tiempo y optimización por gradiente descendente. Esto permite que la red aprenda comportamientos dinámicos complejos, tales como el crecimiento morfológico, la regeneración parcial o la adaptación frente a perturbaciones.

Como las redes neuronales generan valores reales, se puede medir la tasa de error entre la predicción que hacen y el comportamiento real del sistema al usar la regla que han aprendido. Esta métrica de error se propone como base para el desarrollo del presente trabajo. Sin embargo, también se puede comparar la pérdida con otros elementos cercanos, ya que dos células pueden tener muchos vecinos en común, lo que podría explicar el error que se observa en la predicción.

Como se muestra a continuación, esta herramienta ha demostrado ser capaz de generar patrones autoorganizados y adaptativos sin necesidad de una programación explícita de reglas. Algunos ejemplos incluyen:

- **Morfogénesis:** Crecimiento de formas y estructuras similares a organismos biológicos.
- **Regeneración:** Capacidad de reconstruir regiones dañadas o eliminadas de la configuración.
- **Modelado de sistemas complejos:** Simulación de fenómenos físicos, ecológicos o sociales mediante interacciones locales entrenables.

Esta mezcla de comportamiento que aparece por sí mismo y la habilidad de aprender hace que los autómatas celulares neuronales sean una herramienta muy útil para estudiar sistemas que se organizan solos y que se adaptan.

Los autómatas celulares neuronales surgen de la necesidad de definir funciones de transición más precisas que permitan mejorar la simulación de sistemas complejos, como por ejemplo la propagación del fuego.

Consideremos un autómata celular bidimensional con tres posibles estados: $\{materia\ viva, fuego, materia\ quemada\}$, una vecindad de Moore y una función de transición que especifique, por ejemplo, que si una célula en estado de materia

viva se encuentra adyacente a una célula en estado de fuego, entonces se quema; y que si una célula en estado de fuego está rodeada principalmente de materia quemada, entonces se apaga.

Sin embargo, esta formulación que se basa en reglas claras es una simplificación excesiva que reduce la capacidad del modelo para imitar comportamientos que se ven en sistemas reales. Para superar esta limitación, los *autómatas celulares neuronales* incorporan mecanismos de aprendizaje automático que permiten definir la función de transición de manera adaptativa.

En este enfoque, el autómata aprende a partir de ejemplos. Por ejemplo, secuencias de video que muestran la propagación del fuego, donde cada estado puede representarse mediante un color y cada célula corresponde a un píxel o conjunto de píxeles. De esta manera, la red neuronal subyacente aprende una función de transición capaz de imitar la dinámica del sistema complejo, ajustando sus parámetros a partir de los datos observados.

Adicionalmente, los autómatas celulares tradicionales suelen operar con configuraciones reducidas, es decir, con pocos estados, vecindades mínimas y reglas básicas, ya que el espacio de configuraciones crece exponencialmente al incrementar cualquiera de estos factores. Esta limitación dificulta modelar sistemas complejos presentes en la naturaleza, los cuales suelen involucrar múltiples variables e interacciones de alta dimensionalidad.

5.1. Estado del arte, aplicaciones y lineas actuales de investigación

Un primer avance en esta línea fue presentado por Gilpin [88], demostrando que una red neuronal convolucional puede aprender el comportamiento de un autómata celular clásico. Su estudio destaca que tanto los autómatas celulares como las redes neuronales convolucionales comparten dos propiedades fundamentales: (1) la actualización basada en una vecindad o *kernel*, y (2) la aplicación simultánea de dicha función en todo el espacio.

En particular, Gilpin entrenó una red neuronal convolucional con un *kernel* de 3×3 (mismo tamaño que la vecindad de Moore) para imitar el comportamiento del *glider*[1] en el Juego de la Vida de John Conway. Este trabajo estableció una conexión entre los autómatas celulares y las redes neuronales convolucionales, mos-

[1] Una de las configuraciones más importantes por su capacidad de moverse en el espacio

trando que estas redes artificiales pueden actuar como mecanismos de transición local aprendidos.

Posteriormente, esta línea de investigación fue extendida por Mordvintsev et al. [10], quienes exploraron representaciones continuas para morfogénesis artificial; y por Niklasson et al. [89], que aplicaron el enfoque a la generación de texturas dinámicas sobre superficies tridimensionales. Otros autores también han sugerido usar autómatas celulares neuronales en robótica, clasificación de patrones y simulación de sistemas fotónicos. Esto ha fortalecido su papel como herramienta para modelar sistemas que se autoorganizan.

5.1.1. Morfogénesis

Mordvintsev diseñó un autómata celular neuronal que permite a una célula crecer para formar un patrón, persistir en el tiempo, regenerarse e incluso rotar. Fue diseñado un modelo que utiliza un autómata celular continuo con un vector de estados de 16 dimensiones. Este modelo incorpora gradientes espaciales utilizando filtros de Sobel.



Figura 5.2: Evolución del *emoji* de una lagartija a partir de una única célula activa después de 219 pasos evolutivos [10].

El autómata celular neuronal diseñado tiene las siguientes características (las cuales fueron retomadas por múltiples autores que tomaron a [10] como referencia):

- El vector de cada célula (cada valor en un rango de $[0, 1]$) incluye los canales RGB y un canal α que funciona como umbral para definir si una célula está viva o muerta (establece el resto de valores del vector a 0 si $\alpha \leq 0.1$).

  Considerando las dimensiones $(\text{largo}, \text{ancho}, 16)$ y un kernel de 3×3 y se calcula el gradiente en las dos direcciones (x y y) usando un filtro Sobel. Esto genera un nuevo vector que se utilizará como entrada para la red neuronal.

$$16(\text{estado propio}) + 16(\text{gradiente en } x) + 16(\text{gradiente en } y) = 48$$

- Se usaron convoluciones de 1×1 con una función $ReLU$ y alrededor de 8000 parámetros entrenables y devuelve un δ que se suma al estado actual de célula antes del siguiente paso.
- Se omite el cálculo para las células muertas y la actualización de cada célula se realiza con una probabilidad del 50 %. La actualización debe ser estocástica para lograr un mejor rendimiento [89].

Eyvind Niklasson et al. [89] compararon el rendimiento del modelo en [10] con actualización síncrona y asíncrona. Donde, tras haber llevado a cabo pruebas con diferentes tamaños de paso, concluyeron que la función de pérdida crece exponencialmente cuando la actualización es síncrona. Sin embargo, esto ocurre únicamente cuando el tamaño de paso es mayor a uno.

Una hipótesis de este comportamiento es que, al tener células que se actualizan en diferentes periodos de tiempo, cada célula puede conocer parte del contexto de su vecindad. Esto no se logra únicamente conociendo los valores actuales de cada vecino.

Se realizaron los siguientes experimentos en [10]:

1. Crecer una única célula activa para formar una figura dada mediante *backpropagation*. Cuando el autómata celular llega a la figura deseada, si se siguen aplicando las reglas de evolución, puede crecer sin límites, quedarse en la misma figura, eliminar todas las células o crear patrones que son caóticos o que se repiten.
2. Convertir las figuras objetivo en atractores usando *sample pool*. Se entrenó mediante múltiples condiciones iniciales, obtenidas a partir de entrenamientos fallidos, para llegar al objetivo desde diferentes configuraciones.
3. Regenerar partes de la figura. Se eliminaron partes de la figura objetivo y se añadieron al conjunto de entrenamiento.
4. Rotaron la figura objetivo. Dado que los filtros de Sobel funcionan como direcciones ortogonales, se puede aplicar una rotación mediante:

$$\begin{bmatrix} K_x & K_y \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \cdot \begin{bmatrix} Sobel_x & Sobel_y \end{bmatrix}$$

   Para modificar el ángulo sin necesidad de reentrenar el modelo.

5.1.1.1. Autómatas celulares neuronales isotrópicos

En [90], [91] y [92], Mordvintsev et al. presentan una nueva configuración que no tenga una dirección preferida, la cual nombraron autómatas celulares neuronales isotrópicos.

Para que esta regla sea isotrópica, es necesario que sea independiente de la dirección (sin distinguir "arriba" de "izquierda"). Esto permite crear rotaciones o simetrías sin que el sistema se descontrole.

Se sustituyen las funciones de los filtros de Sobel en [10] por un único filtro laplaciano:

$$K_{lap} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

El cual se concatena al vector de cada célula y alimenta a una red neuronal artificial de 9408 parámetros.

A continuación, se describen los resultados obtenidos con dos diferentes enfoques:

1. Estrategia de semilla estructurada: Se necesitan al menos tres células con valores diferentes y no colineales para definir la orientación y la posición. Modificar la posición de estos puntos implica una evolución acorde a la posición de los mismos.

2. Estrategia de una única semilla: Se modifica la función de pérdida para que sea invariante ante rotaciones y reflejos. Para evitar comparar la figura resultante con todas las rotaciones posibles, se mapean ambas figuras a coordenadas polares y, para cada posible ángulo de rotación, se calcula la diferencia. Para disminuir la complejidad computacional, se aplica la transformada rápida de Fourier y se hace uso del teorema de la convolución:

   $$s * t = \mathcal{F}^{-1}(\mathcal{F}(S) \cdot \mathcal{F}(T))$$

   Se obtienen todas las diferencias únicamente como una multiplicación de transformadas.

   Para evitar que se formen patrones simétricos mezclados (como figuras con cabeza-cabeza o cola-cola), se agrega un nuevo canal llamado *binary auxiliar*

channel que da valores de -0.5 y 0.5 a la parte superior e inferior de la imagen respectivamente.

Con estas nuevas configuraciones se puede obtener el mismo resultado que en [10], pero sin la dependencia de una dirección explícita y con una red neuronal artificial con menos parámetros.

5.1.1.2. Autoclasificación

En [93] Mordvintsev utiliza los autómatas celulares neuronales para la autoclasificación de MNIST[2] las cuales fueron binarizadas con un umbral de 0.1.

El objetivo es que, a partir de una configuración inicial de células vivas, el sistema pueda reconocer la forma solo usando reglas locales. En este caso, se trata de identificar los números de MNIST.

Estos son los cambios del autómata celular MNIST respecto al modelo original [10]:

- Vector de la célula: Originalmente se usaban los canales RGB; ahora se usan 10 canales que funcionan como distribución de probabilidad de cada posible número. Durante la evaluación se utiliza el mayor de ellos.
- Célula viva o muerta: En [10] se define mediante el canal α, pero las células muertas siguen siendo visibles para el resto. En [93] únicamente interactúan las células vivas.
- Topología: Todas las células son un único componente conexo.

Con el nuevo modelo, Mordvintsev pudo realizar clasificaciones con una precisión del 95.3 % con una función de pérdida L_2 y ruido añadido. Este experimento demostró que los autómatas celulares neuronales no solo pueden generar patrones y autorregenerarse, sino también realizar tareas de clasificación distribuidas sin una capa de decisión global.

Aunque los modelos en [10] y [93] son robustos frente a diferentes tipos de perturbaciones, su comportamiento se puede ver afectado por estados individuales de las células, parámetros del modelo y el campo perceptivo (la manera en cómo las células perciben su vecindad).

[2] Conjunto de datos de imágenes 28×28 de números en el rango $[0, 9]$

Es por esto que Randazzo et al. [94] buscan enfrentar estrategias adversarias en el mismo espacio celular para estudiar la reprogramación de los autómatas celulares mediante dos experimentos:

1. **Inyección de adversarios**: Se entrena un nuevo modelo de autómata celular, que se coloca en el entorno que ejecuta uno de los modelos originales, para secuestrar el comportamiento del colectivo.
2. **Perturbación de estados**: A un modelo ya entrenado, se le aplican perturbaciones a todas las células del sistema de forma simultánea.

Para la inyección de adversarios se utilizan dos modelos: el modelo original del autómata celular para MNIST con probabilidad de 90 % y un nuevo modelo que clasifica a todos los números como "ocho" (10 %). Se introducen varias células invasoras en figuras que ya han sido clasificadas, y estas células muestran un comportamiento dominante al clasificar todos los elementos como "ocho". Además, una vez retiradas las células invasoras, el sistema es capaz de recuperarse la mayoría de las veces.

Sin embargo, al aplicar el mismo procedimiento para [10], los resultados no tuvieron el mismo rendimiento. Probaron modificar la morfología de la imagen de una lagartija sin cola y con otro color, pero ninguna estrategia fue capaz de invadir por completo el modelo original.

Una hipótesis es que para MNIST no necesita cambiar completamente su valor para producir algo diferente de lo que generaba antes, ya que el estado de la mutación es un estado que el modelo original ya tenía en cuenta. Por otro lado, al cambiar el color y la forma en [10], se están explorando configuraciones nunca antes vistas por el modelo original.

Para el segundo experimento (perturbación de estados), se sigue la línea de modificar el color y la forma de la imagen de una lagartija. Se aplica una perturbación global de estados cada paso de tiempo con el objetivo de identificar la perturbación específica que dé como resultado un patrón estable.

En cada paso evolutivo se multiplica el vector de cada célula por una matriz simétrica A de 16×16 [10]. Para definir los valores de la matriz, se inicia con una matriz simétrica I y, a diferencia del modelo original, la configuración inicial se establece como la figura objetivo del mismo.

Esta configuración permite crecer un sistema con perturbaciones, alterar su forma y color una vez crecido y modificar el impacto y el número de perturbaciones posibles. Al manejar matrices simétricas, se puede representar como

$$A = Q\Lambda Q^T$$

donde:

- Λ es la matriz diagonal de los valores propios.
- Q es la matriz unitaria de los vectores propios.

Si $\Lambda = I \rightarrow A = I$, por lo que no existe perturbación (multiplicación por matriz identidad). Bajo esta premisa, se propone la siguiente fórmula:

$$A_k = Q(kD + I)Q^T$$

Donde:

- A es la matriz original.
- k es el factor de escala.
- D es la dirección de la perturbación $\Lambda - I$ en el espacio de valores propios.

Si $k = 0$, no existe perturbación en el sistema:

$$A_k = Q(0D + I)Q^T$$
$$A_k = QIQ^T$$

Si $k = 1$, se conserva la perturbación original:

$$A_k = Q(1D + I)Q^T$$
$$A_k = Q(\Lambda - I + I)Q^T$$
$$A_k = Q\Lambda Q^T$$

Mostrando que se puede escalar el tamaño de la perturbación, incluso con perturbaciones inversas con $k = -1$.

Si consideramos dos perturbaciones cuyos vectores propios son similares: $A_k = Q(k_A D_A + I)Q^T$ y $B_k = Q(k_B D_B + I)Q^T$, podemos combinarlos como

$$comb(A_k, B_k) = Q(k_A D_A + k_B D_B + I)Q^T = k_A A + k_B B + (1 - k_A - k_B)I$$

Esta nueva combinación permite configurar múltiples perturbaciones, las cuales se vuelven más sensibles conforme aumentan los k.

Estos experimentos muestran cómo los autómatas celulares neuronales pueden ser vulnerables a diferentes estrategias y cómo las mutaciones "exactas" pueden modificar la estructura de todo el sistema.

Todas estas representaciones han tenido un lugar en un espacio bidimensional. A continuación se presentan algunas propuestas en espacios tridimensionales.

5.1.1.3. Representaciones tridimensionales

Zhang et al. [95] proponen un modelo generativo tridimensional denominado autómata celular generativo que extiende el paradigma de los autómatas celulares hacia la generación progresiva de formas tridimensionales. A diferencia de enfoques que intentan reconstruir directamente la forma completa, el autómata celular generativo formula el proceso como una cadena de Markov, donde cada paso corresponde a una transición local sobre una rejilla tridimensional.

El modelo se entrena para que estas reglas locales, al repetirse, converjan hacia formas específicas. Para ello, se representa la forma como puntos de superficie en una rejilla 3D y se emplea una red convolucional dispersa que opera únicamente sobre las regiones activas.

Durante el entrenamiento, se utilizan secuencias ligeramente distintas a las cadenas de muestreo reales, pero que convergen hacia las formas del conjunto de datos. Esto permite que el modelo aprenda reglas de transición robustas, capaces de generar formas diversas desde configuraciones parciales. Con una sola inferencia, el sistema puede tomar diferentes ejemplos de la distribución que ha aprendido, obteniendo buenos resultados en tareas de completar y generar formas en 3D.

Este enfoque es una aportación importante a la morfogénesis computacional, ya que muestra que las reglas locales que se autoorganizan pueden aplicarse a estructuras 3D complejas utilizando redes neuronales adaptadas.

En [95], se muestra que la autoorganización local se puede escalar a tres dimensiones. Por otro lado, Shyam Sudhakaran et al. [96] amplían el trabajo de [10] a tres dimensiones, usando el juego de Minecraft como espacio celular[3]. El autómata celular neuronal entrenado puede crear estructuras de hasta 3584 bloques utilizando 50 bloques diferentes, manteniendo las propiedades de regeneración de su versión en 2D.

Una entidad en Minecraft se representa como una red 3D donde cada elemento es un vector que almacena (1) el tipo de bloque (incrustado en un vector *one-hot*), (2) su estatus (replicando el comportamiento del canal α de [10]) y (3) estados ocultos.

Se utilizan convoluciones para permitir la generación y manipulación de estructuras 3D. Además, se sustituye el uso de filtros de Sobel por una red de percepción aprendible con un kernel de $3 \times 3 \times 3$, *stride*[4] de 1 y canales de salida equivalentes a tres veces el número de canales del estado celular. Esta red permite que el sistema aprenda activamente a percibir su entorno local, adaptando sus filtros durante el entrenamiento y capturando patrones espaciales relevantes de manera dinámica.

Se utiliza una actualización estocástica y un umbral de activación similar a [10]. Finalmente, todos los modelos se inicializan con una distribución normal estándar, con media cero y desviación estándar en el rango de 0.0001 a 0.1, lo que proporciona un punto de partida estable para el entrenamiento y la evolución del sistema. Este sistema hereda las propiedades regenerativas de su versión 2D.

Finalmente, Horibe et al. [97] lleva las propiedades regenerativas al mundo físico. Se implementa el enfoque regenerativo en robots blandos mediante autómatas celulares neuronales. Cada robot tiene un conjunto de volúmenes de reposo para cada vóxel[5]. Estos volúmenes se activan con una señal global que tiene forma de onda sinusoidal, lo que causa que se expandan y se contraigan. Esta dinámica permite simular locomoción a través de ciclos de actuación, evaluando el desplazamiento del centro de masa como métrica de fitness. Los robots blandos se componen de distintos tipos de vóxeles: músculos en fase y contrafase, huesos rígidos y flexibles, cada uno con un rol en la locomoción.

El modelo entrenado demuestra que los robots blandos pueden recuperar parcialmente su morfología original tras sufrir daño y restaurar su capacidad de mo-

[3] En Minecraft, se pueden crear estructuras estáticas (como árboles) y estructuras dinámicas (como máquinas voladoras)

[4] Cuántas posiciones se desplaza el *kernel* sobre la entrada en cada operación de convolución

[5] Unidad básica de un espacio tridimensional, similar a un píxel en imágenes 2D, pero con volumen

vimiento sin intervención externa. Este comportamiento surge de la dinámica autoorganizada del autómata celular neuronal, que codifica de manera implícita la forma objetivo y su capacidad de reparación.

Para evaluar la capacidad de locomoción y regeneración de los robots blandos generados mediante autómatas celulares neuronales, se emplea el simulador físico Voxelyze. Los experimentos se realizan en configuraciones 2D y 3D, utilizando rejillas de 7×7 y $9 \times 9 \times 9$ vóxeles, respectivamente. Las condiciones de frontera tipo Dirichlet limitan el crecimiento morfológico a regiones internas (5×5 en 2D y $7 \times 7 \times 7$ en 3D).

Este entorno permite comprobar cómo los autómatas celulares neuronales se organizan por sí mismos para crear formas que son funcionales y resistentes. Además, proporciona una base cuantitativa para comparar la eficiencia locomotora antes y después de procesos de regeneración. La estructura en voxeles ayuda a combinar reglas locales y programación distribuida, y se relaciona con modelos inspirados en la biología sobre cómo se forma la estructura.

En conjunto, estos trabajos evidencian cómo los autómatas celulares neuronales pueden extender sus principios regenerativos y de autoorganización a dominios tridimensionales cada vez más complejos. Mientras Autómatas celulares generativos establece un marco generativo para formas 3D a partir de reglas locales aprendidas, los experimentos en Minecraft muestran que dichas propiedades pueden preservarse incluso en entornos con alta variabilidad estructural. Finalmente, la aplicación en robots blandos confirma el potencial bioinspirado de los autómatas celulares neuronales: no solo permiten generar morfologías funcionales, sino que además dotan a los sistemas físicos de resiliencia ante daños, abriendo camino hacia formas de robótica autorreparable.

Otra forma de implementar los autómatas celulares neuronales en 3D es mediante planos, mallas y texturas autoorganizadas que cubren la superficie de un objeto tridimensional.

5.1.2. Autómata celular neuronal en malla

Niklasson et al. [98] y Mordvintsev et al. [99] [100] introducen el concepto de texturas autoorganizadas, cuyo objetivo es reproducir la apariencia global de una textura de referencia sin generar copias exactas. Para esto, entrenan un autómata celular neuronal utilizando la misma estructura neuronal que se propuso en [10], para que pueda reconocer y crear patrones distintivos como tableros de ajedrez,

líneas o mosaicos. Aunque este enfoque tiene un carácter demostrativo, constituye una evidencia significativa de que los autómatas celulares neuronales pueden aprender y generalizar comportamientos visuales locales en lugar de memorizar imágenes específicas.

Niklasson et al. proponen los autómatas celulares neuronales en mallas [101], que es un método creado para crear texturas en movimiento en superficies en 3D. A diferencia de los experimentos en cuadrículas de dos dimensiones, los autómatas celulares neuronales en mallas aplican el mecanismo de autoorganización a mallas de cualquier forma, incluso si están en movimiento. El modelo es consistente en diferentes situaciones, es resistente a cambios externos y permite ajustar características como la densidad, la dirección y la rotación de los patrones que aparecen. Además, varios autómatas celulares neuronales en redes pueden combinarse, creando cambios suaves entre texturas sin interrupciones.

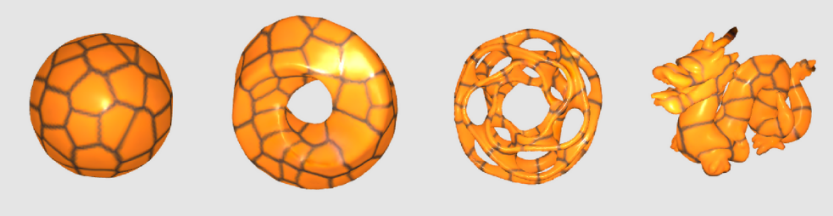


Figura 5.3: El entrenamiento se realiza sobre un poliedro simétrico y el autómata celular neuronal puede ser extrapolado a cualquier objeto tridimensional [10].

Sin embargo, este enfoque presenta un reto de escalabilidad. El número de células involucradas suele encontrarse en el rango de $10^4 \sim 10^5$, lo que limita la resolución alcanzable a entre 64×64 y 256×256 píxeles. Como el tiempo de entrenamiento escala de forma cuadrática con respecto al número de celdas, el aumento de resolución conlleva un cuello de botella en la propagación de información a lo largo de la malla.

Para solucionar esta limitación, Niklasson et al. sugieren en [102] un modelo híbrido que une autómatas celulares neuronales con representaciones neuronales implícitas. La idea central es desacoplar el tamaño de la cuadrícula de los autómatas celulares neuronales de la resolución final, delegando el renderizado detallado a un decodificador ligero denominado red local de producción de patrones. De este modo, el autómata celular neuronal mantiene una cuadrícula explícita y relativamente pequeña que evoluciona de manera local, mientras que la red local de producción de patrones se encarga de producir el color en cualquier punto del espacio continuo a partir de información interpolada de las celdas vecinas.

En términos formales, para cada punto de muestreo p, la red local de pro-

ducción de patrones recibe dos entradas: Una es un estado celular promedio $s(p)$, obtenido mediante interpolación de las celdas cercanas en un primitivo geométrico; la otra es un vector de coordenadas locales $u(p)$, que codifica la posición relativa de p dentro de dicho primitivo. Dependiendo de cómo se divida el espacio, estas formas básicas pueden ser rectángulos, cubos, triángulos o polígonos en general. Estas formas están definidas por coordenadas $\lambda_i(p)$ que deben cumplir con ciertas propiedades, como ser una partición unitaria y no ser negativas. Para asegurar una continuidad en el espacio, se cambian las coordenadas usando funciones sinusoidales en cuadrículas cartesianas o en redes triangulares, creando así campos posicionales suaves.

El resultado es un sistema que conserva las propiedades autoorganizativas de los autómatas celulares neuronales al tiempo que permite alcanzar mayor resolución sin un aumento proporcional del costo computacional. Desde una perspectiva biológica, el autómata celular neuronal puede interpretarse como un plano morfogenético que define la organización global, mientras que la red local de producción de patrones añade los detalles finos sin necesidad de simular interacciones locales a gran escala. Este paralelismo, aunque principalmente ilustrativo, refuerza la idea de que los autómatas celulares neuronales pueden actuar como generadores de estructuras robustas y adaptables, mientras que módulos complementarios aportan precisión y escalabilidad.

A pesar de estas mejoras, una limitación persistente es que cada autómata celular neuronal debe entrenarse de manera específica para una textura o patrón dado. Incluso en escenarios donde múltiples autómatas celulares neuronales coexisten, cada modelo requiere su propio entrenamiento independiente. Esto ha motivado nuevas propuestas orientadas a la construcción de espacios compartidos de autómatas celulares neuronales, capaces de generalizar a múltiples texturas dentro de un mismo marco de entrenamiento.

5.1.3. Modelos generativos

Cada imagen creada con un autómata celular neuronal tiene una red neuronal artificial que ha sido entrenada específicamente para copiar ese comportamiento. Alejandro Hernández et al. [103] proponen unas redes neuronales artificiales de mayor capacidad que permitan crecer múltiples imágenes, a lo que llamaron colector de autómatas celulares neuronales.

El colector de autómatas celulares neuronales usa un modelo de convoluciones dinámicas para modelar un espacio de autómatas celulares neuronales. Utiliza un

autoencoder que se compone de un codificador y un decodificador: El codificador transforma la entrada en una representación continua y luego categórica, que el decodificador reconstruye en el espacio original.

Un elemento destacable de la arquitectura propuesta es un parámetro entrenable llamado *Leak Factor* (LF) que mejora la estabilidad durante el entrenamiento. Este parámetro regula la cantidad de información que se filtra en cada paso del modelo; por ejemplo, un valor bajo retiene el conocimiento acumulado. Este valor se incrementa durante el entrenamiento a medida que mejora el rendimiento.

La arquitectura también emplea convoluciones dinámicas. Los *kernels* se generan a partir del *embedding* de cada instancia, permitiendo adaptabilidad sin modificar la estructura del autómata celular neuronal. El colector de autómatas celulares neuronales inicia desde una imagen semilla y actualiza cada celda mediante percepciones locales (filtros Sobel + normalización) y capas densas por píxel, moduladas por el LF. Se exploran actualizaciones estocásticas y sincrónicas, siendo esta última más precisa y estable, tal como en [89].

Los experimentos muestran que la codificación y la actualización al mismo tiempo mejoran la calidad de la reconstrucción, incluso sin mecanismos de enmascaramiento. También se observa que aumentar la dimensionalidad del *embedding* mejora significativamente los resultados.

Por otro lado, Kevin Frans en [104] propone un modelo llamado autómatas celulares neuronales condicionales, cuya semilla recibe un *embedding*, el cual define la imagen en la cual va a crecer.

Dado un diseño deseado, una red de codificador lo comprime en un *embedding*, el cual se utiliza como condición inicial en la autómata celular neuronal, que luego itera para crecer en la figura objetivo; esto permite crear nuevas configuraciones.

Utilizando un *dataset* con aproximadamente 3000 *emojis* de 32×32 píxeles, fue capaz de entrenar a un autómata celular neuronal condicional utilizando el mismo método de [10] para poder dibujar todos los *emojis* con un único autómata celular neuronal.

Para generar números de MNIST, se adopta un enfoque tipo red adversarial generativa, donde el objetivo es crecer dígitos desde ruido aleatorio. Dado que entrenar directamente un autómata celular neuronal como generador de red adversarial generativa resultó inestable, se utilizó una estrategia en dos etapas: (1) Se entrena primero un generador tradicional (*feedforward*) para mapear vectores aleatorios a dígitos MNIST y (2) el autómata celular neuronal se entrena para imitar la salida de ese generador, logrando una generación más estable.

Este enfoque permite que el autómata celular neuronal genere dígitos sin necesidad de un *dataset* explícito, aprendiendo directamente a partir del objetivo red adversarial generativa. Además, se observa un patrón de crecimiento distintivo: mientras los *emojis* crecen en ondas desde el centro, los dígitos MNIST se desarrollan siguiendo la forma del trazo.

Mingxiang Chen et al. [105] combinan los principios de codificador automático variacional con autómata celular neuronal. Utilizan un *encoder* que transforma una imagen en una distribución de probabilidad, las cuales forman un *embedding*, el cual funciona como condición inicial del autómata celular neuronal. Después, Rasmus Berg Palm et al. [106] llaman a este modelo "autómatas celulares neuronales variacionales". También afirman que ninguno de los otros modelos mencionados ([10], [103], [95], [97] y [104]) es realmente generativo.

El autómata celular neuronal variacional aprende una distribución de probabilidad que codifica los datos observados. A partir de esta representación, el *encoder* genera imágenes mediante un autómata celular neuronal que simula el crecimiento espacial desde una semilla. A diferencia del codificador automático variacional, que decodifica la imagen directamente, el autómata celular neuronal variacional utiliza el *embedding* como configuración inicial de una cuadrícula de celdas que evolucionan siguiendo las reglas definidas por una red neuronal convolucional.

Por otra parte, este modelo incorpora mecanismos de duplicación celular inspirados en la mitosis. Cada M pasos, las celdas se duplican hacia la derecha y hacia abajo, permitiendo a la imagen crecer de manera autoorganizada.

Los experimentos realizados en [106] demuestran que el autómata celular neuronal variacional es capaz de aprender procesos generativos autoorganizados tanto en dominios simples como MNIST, como en conjuntos más complejos como CelebA[6]. En MNIST, el modelo logra reconstrucciones precisas y muestras plausibles desde el espacio latente, con una log-verosimilitud de $\log p(x) \geq -84.23$ nats[7]; su rendimiento es comparable a modelos autorregresivos como PixelCNN. Además, el análisis del espacio latente muestra una estructura más clara y organizada en comparación con los decodificadores convolucionales tradicionales. Esto indica que la codificación es más semántica y eficiente.

En CelebA, aunque las muestras generadas directamente no alcanzan calidad visual comparable a modelos de última generación, el autómata celular neuronal variacional muestra capacidad de reconstrucción robusta y mejora visual significativa

[6] *Dataset* de imágenes de celebridades

[7] Unidad de medida utilizada en aprendizaje automático y estadística para expresar la log-verosimilitud o la entropía de un modelo

al ajustar el parámetro β, que regula el equilibrio entre fidelidad de reconstrucción y calidad de generación.

Finalmente, se explora la resiliencia de los autómatas celulares neuronales variacionales ante daño estructural durante el crecimiento. Al eliminar las capas de duplicación y entrenar con un esquema de recuperación de daño, el modelo aprende dinámicas estables capaces de restaurar imágenes deterioradas casi por completo, incluso tras múltiples pasos de evolución. Este comportamiento sugiere que el autómata celular neuronal variacional no solo codifica la imagen, sino que también es un proceso de reconstrucción en el estado celular.

5.1.4. Representaciones y aplicaciones adicionales

Hasta ahora, muchos de los estudios sobre autómatas celulares neuronales se han enfocado en la morfogénesis, que es el proceso de formación de formas, y en su habilidad para crear estructuras visuales que se organizan solas. Sin embargo, también han surgido enfoques que amplían el marco conceptual hacia nuevas formas de representación y aplicaciones en dominios específicos. A continuación, se presentan algunas de estas variantes.

Mattie Tesfaldet et al. [107] extienden los autómatas celulares neuronales mediante mecanismos de atención inspirados en arquitecturas de *transformers*, dando lugar a autómatas celulares con *transformers* de visión. Esta arquitectura híbrida permite que las interacciones locales tengan acceso contextual a regiones más amplias a través de *self-attention*, lo que habilita organización global sin incurrir en la complejidad cuadrática típica de la atención completa. Los autómatas celulares con *transformers* de visión adapta el proceso de los *transformers* de visión tradicionales, que dividen una imagen en parches y los convierten en secuencias de *tokens*, al formato de una cuadrícula de celdas, donde cada celda almacena información visual, posicional y oculta. En lugar de aplicar atención global, restringe el cálculo a vecindarios locales, manteniendo así la naturaleza distribuida del sistema y reduciendo el costo computacional. El entrenamiento se realiza de forma iterativa, alternando entre inyectar entradas corruptas y actualizar parcialmente los estados celulares hasta reconstruir la imagen. De este modo, los autómatas celulares con *transformers* de visión combina eficiencia, capacidad de autoorganización y organización espacial coherente.

En un ámbito distinto, Jian Tang et al. [108] emplean una red neuronal convolucional para simular el desarrollo de microestructuras en la solidificación de metales. Cada celda del dominio contiene un estado de fase $P \in [0, 1]$ (donde $p = 0$ repre-

senta líquido y $p = 1$ sólido), tres ángulos de Euler $\theta = (\alpha_1, \alpha_2, \alpha_3)$ que definen su orientación cristalográfica, y la magnitud de subenfriamiento $\Delta T = T_{melting} - T$. La evolución se modela con un autómata celular neuronal que aplica recurrentemente una red neuronal convolucional para predecir los incrementos en fase, orientación y estados ocultos. Estos canales ocultos facilitan el aprendizaje de patrones temporales complejos sin necesidad de reglas explícitas. Con ello, el modelo reproduce fenómenos como crecimiento competitivo o direcciones preferentes, con una eficiencia computacional sustancialmente mayor que la de muchos autómatas celulares tradicionales y con capacidad de generalización fuera del rango de entrenamiento.

Por otro lado, Gordon H. Y. Li et al. [109] proponen el modelo de autómatas celulares neuronales fotónicos, que aprovecha la velocidad y la interconexión masiva de la fotónica para implementar autómata celular neuronal en hardware óptico. A diferencia de arquitecturas convencionales basadas en perceptrón multicapa o red neuronal convolucional, difíciles de trasladar a entornos fotónicos por sus conexiones densas, el autómata celular neuronal fotónico utiliza reglas locales aprendidas mediante técnicas de aprendizaje profundo. Esto permite realizar clasificación de imágenes en sistemas ópticos, donde la actualización celular se implementa mediante interferencia lineal de luz y activaciones ópticas no lineales. Los autores demuestran clasificación binaria con alta precisión utilizando un número muy reducido de parámetros fotónicos programables, mostrando además robustez frente a datos fuera de distribución. El autómata celular neuronal fotónico representa una vía prometedora para llevar el aprendizaje autoorganizado a plataformas fotónicas escalables.

En el ámbito de la biología computacional, Caitlin Grasso y Josh Bongard [110] introducen los autómatas celulares neuronales potenciados. Esta es una extensión que incorpora el concepto de *empowerment*, la cantidad de control que una célula puede ejercer sobre su entorno, formalizado mediante medidas de información mutua entre acciones pasadas y estados sensoriales futuros. Incluir *empowerment* como un objetivo adicional en las tareas de morfogénesis mejora la capacidad de evolución y genera patrones de crecimiento más fuertes y adaptables. Esto permite que cada célula actúe como un agente independiente dentro de un sistema distribuido y facilita el estudio de la coordinación y comunicación celular desde una perspectiva computacional.

Finalmente, mientras que modelos como autómatas celulares con *transformers* d e visión, autómata celular neuronal fotónico o autómatas celulares neuronales potenciados amplían las capacidades arquitectónicas y las plataformas de aplicación de los autómatas celulares neuronales, Richardson et al. [111] utilizan los autómatas celulares neuronales como herramienta para modelar fenómenos dinámi-

cos complejos. Entrenando autómata celular neuronal mediante optimización basada en gradientes para reproducir trayectorias espacio-temporales generadas por ecuaciones diferenciales parciales (por ejemplo, Gray–Scott), este enfoque captura tanto estructuras transitorias como patrones estacionarios, incluyendo la formación de patrones tipo Turing, y aborda el problema inverso de inferir reglas locales a partir de comportamiento emergente. Con una complejidad de red mínima, los autores muestran capacidad de generalización, respeto por simetrías y robustez frente a perturbaciones, posicionando a los autómatas celulares neuronales como un marco explicativo y eficiente para sistemas biológicos y físicos.

Se observa que los autómatas celulares neuronales han evolucionado desde sus primeras aplicaciones en morfogénesis, donde se exploró su capacidad de generar estructuras complejas partiendo de una única célula activa, hacia modelos que buscan una mayor versatilidad generativa, capaces de producir dinámicas visuales, texturas, comportamientos adaptativos e incluso manipular estrategias, muchos de ellos inspirados en la definición de Mordvintsev. Además, se han desarrollado arquitecturas híbridas que no generan un autómata celular neuronal individual, sino que modelan un espacio de autómatas semejantes, así como configuraciones generativas y bioinspiradas.

En resumen, estos estudios muestran que los autómatas celulares neuronales no solo son un concepto interesante desde el punto de vista teórico, sino que también son útiles en inteligencia artificial, ciencias naturales e ingeniería. Esta diversidad de enfoques abre la puerta a explorar nuevas adaptaciones y propuestas, especialmente en áreas de aprendizaje profundo, autoorganización y simulación de procesos dinámicos.

Capítulo 6

Marco metodológico

El marco metodológico se constituye como el resultado de la aplicación sistemática y lógica de los conceptos y fundamentos abordados en el marco teórico. Azuero et al. [112] lo definen como un conjunto de acciones destinadas a describir y analizar el fondo del problema planteado, mediante procedimientos específicos que incluyen técnicas de observación y recolección de datos, los cuales determinarán el cómo se llevará a cabo la investigación.

Este capítulo se dividirá en secciones que definan la estructura del marco metodológico, con el fin de sustentar los métodos y recursos elegidos para el desarrollo de la investigación. Se asegurará que cada fase esté alineada con los principios teóricos fundamentales y que los resultados obtenidos sean fiables, replicables y efectivos en el tratamiento de la problemática planteada.

6.1. Diseño de la investigación

6.1.1. Diseño experimental

El diseño experimental es un método sistemático para planificar, ejecutar y analizar experimentos, con el objetivo de identificar relaciones causales entre variables. En otras palabras, se manipula una variable independiente para observar su efecto sobre la variable dependiente [113].

De acuerdo con su nivel de control, los diseños experimentales se clasifican en tres tipos: preexperimentos, cuasiexperimentos y experimentos puros. En el

presente trabajo se adopta un diseño cuasiexperimental por las características propias del entorno de simulación y la naturaleza del objeto de estudio.

El diseño cuasiexperimental resulta apropiado en este contexto. Se manipula una variable independiente (la definición de la función de transición mediante una red neuronal) para observar sus efectos sobre el comportamiento del sistema simulado. Esto se hace sin que exista una asignación aleatoria ni un grupo control en un sentido estricto.

6.1.1.1. Diseño cuasiexperimental

El diseño cuasiexperimental busca establecer relaciones causales sin la necesidad de asignar aleatoriamente a los participantes o unidades experimentales a los grupos de estudio. Se caracteriza por la existencia de una intervención deliberada sobre una variable independiente, mientras que los grupos de comparación se definen con base en condiciones preexistentes [114].

En el caso de esta investigación, el estudio se desarrolla en un entorno computacional controlado, donde las condiciones experimentales se reproducen mediante simulaciones. El objetivo principal es estudiar cómo diferentes factores, como la forma de la red neuronal o la configuración del autómata, afectan los patrones que aparecen en los autómatas celulares neuronales. Este enfoque permite observar efectos y comportamientos resultantes, manteniendo un control suficiente sobre las variables involucradas sin requerir un esquema de aleatorización.

6.2. Enfoque de la investigación

Este proyecto utiliza un enfoque cuantitativo, que se basa en recoger y analizar datos numéricos para describir fenómenos a través de métodos estadísticos y matemáticos [115]. Este enfoque prioriza la objetividad y medición precisa para relacionar causales entre las variables observadas.

A diferencia de los enfoques cualitativos o mixtos, este no requiere de estudiar percepciones, significados ni experiencias humanas, por lo que es adecuado para investigaciones donde las variables pueden ser definidas, controladas y medidas [116].

El enfoque cuantitativo se justifica porque el análisis se realiza a partir de datos generados mediante simulación computacional. En este proceso no hay intervención

directa con humanos y los resultados obtenidos no están sujetos a interpretaciones. Este enfoque permite comparar escenarios, desempeño de modelos e influencia de parámetros sobre la dinámica emergente del sistema.

6.3. Población

La población de estudio se define como un conjunto cerrado, finito y accesible de casos (personas, animales, expedientes, objetos, etc.), los cuales cumplen con un criterio de inclusión y que sirven como base para la investigación [117]. En otras palabras, es un conjunto conformado por todos los elementos a los que se pretende generalizar los resultados del estudio.

En este trabajo, la población que se estudia está formada por todos los autómatas celulares finitos que se encuentran en un espacio bidimensional con una distribución de cuadrícula regular. Se consideran condiciones de frontera un conjunto de estados finito y cualquier función que pueda mapear de $\mathbb{R}^{|\mathcal{N}|} \to \mathbb{R}$.

6.4. Muestra y muestreo

La muestra corresponde a un subconjunto de la población de estudio y consiste en el número de casos que realmente serán estudiados. Esta debe ser representativa para garantizar la validez de los resultados. Por otro lado, el muestreo se refiere al proceso aplicado por el investigador para seleccionar a los participantes dentro de la población [118].

Existe el muestreo probabilístico (donde cada miembro de la población tiene una probabilidad de ser escogido) y no probabilístico (donde la selección se basa en la conveniencia, el juicio del investigador u otros criterios) [118].

Para esta investigación se realiza un muestreo no probabilístico intencional, donde los elementos seleccionados para la muestra son elegidos por el criterio del investigador. Esto se justifica, ya que, al existir una infinitud de autómatas celulares dadas las restricciones en la población, es necesario delimitar y tratar de replicar los comportamientos. Además, dado que cada autómata celular resuelve un problema, es necesario identificar de forma precisa el problema que se está resolviendo [7].

6.5. Tipo de investigación

Con base en los objetivos, esta investigación se considera aplicada. La investigación aplicada, también conocida como práctica o empírica, se caracteriza porque busca poner en práctica o utilizar los conocimientos adquiridos. En este tipo, las consecuencias prácticas son un interés primordial del investigador [119].

Además, se utiliza un diseño cuasiexperimental, porque se cambian variables independientes para ver cómo afectan los resultados de la simulación, sin tener un grupo control o asignar condiciones experimentales de forma aleatoria.

Este tipo de investigación es útil porque ayuda a encontrar relaciones de causa y efecto entre los elementos del modelo y los comportamientos que surgen en el sistema, mientras se mantiene un entorno computacional que es controlado y se puede repetir.

6.6. Técnicas e instrumentos de recolección de datos

La recolección de datos se realiza a partir de las simulaciones computacionales realizadas. Dado que se trata de enfoque cuantitativo, no es necesario usar técnicas de observación como encuestas, sino procedimientos de registro automatizado para su posterior análisis numérico.

6.6.1. Técnicas de recolección de datos

- Simulación computacional: Ejecución del modelo del autómata celular bajo diferentes configuraciones de vecindad, estados y arquitectura de la red neuronal artificial.
- Registro automático de datos: Datos generados por la simulación. Como el historial evolutivo, patrones y métricas de estabilidad, error y desempeño.
- Análisis estadístico: Se aplican métricas cuantitativas que permiten comparar configuraciones para evaluar la efectividad del modelo. Métricas como precisión, convergencia, tasa de error y comportamiento temporal de los patrones.

6.6.2. Instrumentos de recolección de datos

Se consideran los siguientes instrumentos para obtener y analizar los datos:

- Simulador desarrollado: Herramienta computacional implementada en Python que permite ejecutar y visualizar el comportamiento de los autómatas celulares neuronales.
- Códigos de registro y procesamiento: Conjunto de programas auxiliares que capturan las métricas numéricas de cada simulación y las almacenan en archivos.
- Herramientas de análisis estadístico y visualización: Librerías y herramientas que permiten obtener representaciones gráficas con los datos obtenidos.

6.7. Procedimiento

El desarrollo metodológico de la investigación se llevará a cabo en diversas fases secuenciales. Estas abarcan desde el diseño e implementación del simulador hasta la evaluación del modelo de autómatas celulares neuronales. Cada fase está orientada a cumplir los objetivos planteados y a generar datos cuantitativos que permitan analizar el comportamiento de los sistemas simulados.

- **Diseño, desarrollo e implementación del simulador**: Se construye una herramienta computacional capaz de simular autómatas celulares bidimensionales. Aunque el fin último es estudiar los autómatas celulares neuronales, este simulador debe poder aceptar configuraciones clásicas. Esto debido a que, para las siguientes fases, es necesario conocer el comportamiento de configuraciones conocidas.
- **Diseño, desarrollo e integración del módulo red neuronal artificial**: Se implementa un módulo adicional que permite entrenar una red neuronal artificial para aprender el comportamiento de autómatas celulares existentes. La salida de este módulo es la función de transición aprendida. Además, este módulo se integra dentro del simulador principal para permitir la experimentación directa.
- **Imitación de patrones conocidos**: Se diseñan y ejecutan experimentos para evaluar la capacidad del modelo neuronal de imitar el comportamiento

de autómatas celulares clásicos, como el juego de la vida. Se busca, además, medir la precisión del aprendizaje y la estabilidad de las configuraciones generadas.

- **Simulación de otros sistemas complejos:** Una vez validado el funcionamiento del modelo neuronal, se aplican las mismas técnicas a simulaciones que representen otros sistemas complejos. Se busca probar diferentes configuraciones iniciales y parámetros para analizar la capacidad de generalizar la aparición de nuevos patrones.

Cada una de estas etapas genera resultados que ayudan a mejorar el proceso de desarrollo. Esto permite ajustar tanto la estructura de la red neuronal como las condiciones del simulador.

6.8. Técnicas y herramientas de procesamiento de datos

Una vez recolectados los datos provenientes de las simulaciones computacionales, es fundamental aplicar técnicas adecuadas de procesamiento y análisis para garantizar la validez de los resultados y su correcta interpretación. Dado que el enfoque de la investigación es cuantitativo, el procesamiento se centra en el análisis numérico de los datos generados por el simulador, permitiendo identificar patrones, tendencias y relaciones entre las variables del sistema.

6.8.1. Procesamiento de datos cuantitativos

Los datos cuantitativos se obtienen de las ejecuciones del simulador, donde se registran variables como número de iteraciones, estados promedio de las celdas, métricas de error y estabilidad de los patrones emergentes durante un periodo determinado. Posteriormente, estos datos se organizan, procesan y analizan mediante herramientas estadísticas y de visualización.

- **Análisis estadístico descriptivo:** Se calculan medidas de tendencia central y de dispersión con el fin de describir cuantitativamente el comportamiento de las variables analizadas y detectar posibles irregularidades o tendencias [120].

- **Visualización de datos:** Se elaboran gráficos de barras, histogramas y diagramas de dispersión para representar visualmente los resultados obtenidos, facilitando la comprensión de la dinámica del sistema y la comparación entre diferentes configuraciones experimentales.

El procesamiento y análisis de los datos se lleva a cabo utilizando herramientas computacionales. Los resultados obtenidos servirán como base para la interpretación y discusión en las etapas posteriores de la investigación.

6.9. Contextualización de la tecnología

El desarrollo de un simulador de sistemas complejos mediante autómatas celulares neuronales implica la integración de múltiples tecnologías, seleccionadas estratégicamente para satisfacer los requerimientos funcionales y técnicos del proyecto. Esto incluye crear una interfaz gráfica de usuario fácil de usar, simular autómatas celulares en dos dimensiones con diferentes estados, vecindades y reglas de transición, personalizar las condiciones iniciales y entrenar sistemas para que imiten comportamientos que el usuario proporciona.

La elección de estas tecnologías responde a criterios de compatibilidad, escalabilidad, facilidad de implementación y disposición de herramientas específicas para el procesamiento de datos e inteligencia artificial.

Con el fin de proporcionar una visión clara y comprensiva de las bases técnicas del proyecto, a continuación se presentan tanto el área de estudio, elementos clave para orientar el desarrollo del simulador y validarlo, como los fundamentos y el contexto de las tecnologías utilizadas, así como los entornos de trabajo que hacen posible este trabajo.

6.9.1. Fundamentos y contexto de las tecnologías

Un lenguaje de programación es una herramienta que permite el desarrollo de software y programas para computadora. Son empleados para diseñar programas encargados de definir el comportamiento físico y lógico de una computadora [121].

Debido a que la elección del entorno y lenguaje de programación resulta crítica para el éxito de cualquier proyecto, es importante considerar las metas y alcances

del mismo, puesto que no todos los lenguajes son óptimos para resolver ciertos tipos de problemas [122].

Entre los factores clave para la elección de un lenguaje de programación se encuentran la escalabilidad, el rendimiento, el costo de mantenimiento, la compatibilidad y el sistema de herramientas, entre otros [122].

Es por esto que, como lenguajes de programación, se decidió utilizar C++/CUDA para el núcleo de simulación y cálculos pesados en paralelo, y Python para la interfaz, visualización, control de simulación y diseño de redes neuronales.

A continuación se presentan las definiciones pertinentes de los lenguajes de programación, frameworks y librerías utilizadas para el desarrollo de la aplicación móvil.

6.9.1.1. C++

C++ es un lenguaje de programación compilado y multiparadigma desarrollado como una extensión de C [123]. Se considera un lenguaje de bajo nivel, ya que permite manipular directamente la memoria, lo que lo convierte en un lenguaje muy poderoso. Es orientado a objetos, permite la creación de clases y objetos y es ampliamente utilizado para aplicaciones que requieren un alto rendimiento.

Es utilizado en motores de videojuegos, software de sistemas, aplicaciones en tiempo real, simulaciones científicas y cálculo numérico [124]. Para esta investigación, se utilizarán las siguientes bibliotecas de C++:

- **pybind11**: facilita la integración de código C++ con Python, permitiendo exponer funciones y clases para análisis y visualización [125].
- **iostream**: Declara objetos que controlan la lectura y escritura de datos de un programa en C++ [126].
- **vector**: Contenedor para una secuencia cuando el rendimiento de acceso aleatorio es importante [127].
- **cmath**: Incluye *math.h* de la biblioteca estándar de C para realizar operaciones matemáticas [128].
- **map**: Almacena elementos de la forma "llave → valor". Los elementos se acceden mediante las llaves, estas son únicas y se ordenan de forma ascendente [129].

- **utility**: Define los tipos, funciones y operadores de la biblioteca estándar de C++ que ayudan a construir y administrar pares de objetos [130].
- **algorithm**: Contiene funciones que realizan algoritmos comunes [131].

6.9.1.2. CUDA

CUDA es una plataforma de computación paralela y un modelo de programación desarrollado por NVIDIA que permite utilizar la GPU para cálculos de propósito general. Se emplea para acelerar operaciones matemáticas y simulaciones que requieren ingente cantidad de procesamiento paralelo, como el entrenamiento de redes neuronales [132].

6.9.1.3. Python

De acuerdo con la página oficial de la documentación de Python [133], es un lenguaje de programación potente y fácil de aprender. Tiene estructuras de datos de alto nivel, sintaxis elemental y elegante, un tipado dinámico, naturaleza interpretada, etc.

Además de ser gratuito, se puede integrar bien en todos los equipos y sistemas. En adición a esto, [134] señala que entre los beneficios que Python incluye se encuentran los siguientes:

- Los programas son fáciles de leer y comprender.
- Mayor productividad al escribir un programa con menos líneas de código en comparación con muchos otros lenguajes de programación.
- Se puede usar fácilmente con otros lenguajes de programación.
- Soporte rápido de la comunidad.
- Se puede trasladar a través de diferentes sistemas operativos de computadora.

En adición a lo anterior, cuenta con una amplia colección de librerías y *frameworks* diseñados para tareas que resultaron muy convenientes en el desarrollo de la aplicación móvil. Se detallan tanto librerías como *frameworks* a utilizar a continuación.

- **Os:**

  Es una librería que provee funcionalidades dependientes del sistema operativo como abrir un archivo, manipular rutas, leer archivos, crear archivos temporales, etc. [135].

- **NumPy** es una librería de Python especializada en el cálculo numérico y el análisis de datos. Es ideal para el manejo de grandes volúmenes de datos [136].

  Puede hacer operaciones hasta 50 veces más rápido que las listas normales usando un objeto llamado arreglo que permite representar colecciones de datos en varias dimensiones [136].

- **CuPy** es una biblioteca de matrices de código abierto para la computación acelerada por GPU con Python. Utiliza bibliotecas del CUDA Toolkit para hacer un uso completo de la arquitectura GPU. Es compatible con NumPy y permite definir núcleos CUDA personalizados [137].

- **PyTorch** es una biblioteca de tensores optimizada para el aprendizaje profundo utilizando GPU y CPU [138]. Se emplea para crear redes neuronales, combinando la biblioteca de ML de Torch con una API de alto nivel basada en Python.

  Es uno de los marcos de ML destinados a comunidades académicas y de investigación por su flexibilidad y sencillez de uso, además de admitir una amplia variedad de estructuras de redes neuronales [139].

- **PyQt5 / PySide6** son herramientas que permiten usar Python como un lenguaje alternativo de desarrollo de aplicaciones a C++ en todas las plataformas soportadas [140]. PyQt5 también puede integrarse en aplicaciones basadas en C++ para permitir a los usuarios de dichas software configurar o mejorar la funcionalidad de las mismas, mientras que PySide6 ofrece compatibilidad oficial con las bibliotecas Qt desarrolladas por The Qt Company [141].

- **h5py** es una librería que proporciona una interfaz entre Python y el formato de almacenamiento de datos jerárquico HDF5 (Hierarchical Data Format). Este formato permite manejar grandes volúmenes de datos de manera eficiente, organizándolos en estructuras tipo árbol con grupos y conjuntos. Con h5py es posible almacenar, leer y modificar datos numéricos de considerable tamaño directamente desde Python, lo cual resulta especialmente útil en simulaciones científicas que generan extensas matrices multidimensionales [142].

- **pybind11** es una biblioteca de C++ diseñada para exponer tipos y funciones de C++ en Python y viceversa, facilitando la creación de extensiones de Python basadas en código C++ [125]. Puede generar enlaces entre ambos lenguajes de manera compacta, eficiente y con un código más simple, posibilitando así integrar cálculos de alto rendimiento escritos en C++ dentro de entornos Python.

- **Pandas** es una librería especializada en el manejo y análisis de estructuras de datos. Entre sus principales características, [143] menciona:

  - Define nuevas estructuras de datos basadas en los arrays de la librería NumPy.
  - Permite leer y escribir fácilmente ficheros en formato CSV, Excel y bases de datos SQL.
  - Permite acceder a los datos mediante índices o nombres para filas y columnas.
  - Ofrece métodos para reordenar, dividir y combinar conjuntos de datos.

- **Matplotlib** De acuerdo con [144], Matplotlib es una librería de Python especializada en la creación de gráficos en dos dimensiones.

  Además, mencionan que permite crear y personalizar los gráficos más comunes, entre los que se encuentran:

  - Diagramas de barras.
  - Histograma.
  - Diagramas de sectores.
  - Diagramas de caja y bigotes.

- **Scikit-learn** es una librería que cuenta con algoritmos de clasificación, regresión, clustering y reducción de dimensionalidad, etc. Es compatible con otras librerías de Python [145].

  Entre sus principales características, [146] destacan las siguientes:

  - Clustering.
  - Ensemble methods.
  - Validación cruzada.
  - Varios conjuntos de datos o datasets de prueba.
  - Reducción de la dimensionalidad.
  - Optimización o ajuste de hiperparámetros.
  - Selección de características.

6.9.2. Entornos de trabajo

El desarrollo del presente proyecto requiere de un entorno colaborativo, flexible y bien estructurado que facilite tanto la construcción como la validación de los distintos componentes del sistema. Para ello, se han seleccionado diversas plataformas que permiten organizar tareas, escribir y probar código, almacenar datos, así como documentar y presentar avances de manera clara y profesional.

6.9.2.1. Plataformas de desarrollo

Con el fin de cubrir las distintas etapas del proyecto, desde la implementación hasta la comunicación de resultados, se utilizaron múltiples herramientas que favorecen la eficiencia y el trabajo en equipo. Las siguientes plataformas han sido clave para mantener un flujo de trabajo continuo y bien coordinado.

- **Visual Studio Code** Comúnmente conocido como VSCode, es un editor de código gratuito, de código abierto y compatible con múltiples sistemas operativos. Fue creado por Microsoft, empresa con amplia trayectoria en el desarrollo de entornos integrados (IDE), lo que se refleja en la eficiencia y funcionalidad que ofrece esta herramienta, muy valorada por la comunidad de desarrolladores [147].

  Aunque su nombre puede sugerir una relación directa con Visual Studio, en realidad VSCode es una aplicación independiente, construida desde cero con tecnologías diferentes. Mientras Visual Studio se orienta principalmente a tecnologías propias de Microsoft, VSCode destaca por su flexibilidad y capacidad de adaptarse a prácticamente cualquier lenguaje de programación [148].

- **Github** Es una plataforma basada en la nube donde puedes almacenar, compartir, trabajar y versionar junto con otros usuarios para escribir código [149]. En adición a lo anterior, también permite realizar lo siguiente:

  - Presentar o compartir el trabajo.
  - Seguir y administrar los cambios en el código a lo largo del tiempo.
  - Dejar que otros usuarios revisen el código y realicen sugerencias para mejorarlo.
  - Colaborar en un proyecto compartido, sin preocuparse de que los cambios afectarán al trabajo de los colaboradores antes de que esté listo para integrarlos.

El versionamiento, una de las características fundamentales de GitHub, es posible gracias al software de código abierto Git, en el que se basa GitHub [149].

- **Canva** Lanzada en 2013, Canva es una herramienta de diseño y publicación online que tiene como misión poner el poder del diseño al alcance de todo el mundo, para que cualquier persona pueda crear lo que quiera y publicar donde quiera [150].
- **TexStudio** TeXstudio es un entorno de escritura integrado para la creación de documentos LaTeX. Cuenta con numerosas características como resaltado de sintaxis, visor integrado, comprobación de referencias y varios asistentes. TeXstudio es de código abierto y está disponible para todos los principales sistemas operativos [151].

Habiendo definido el enfoque metodológico, justificado las decisiones técnicas del estudio y contextualizado la tecnología utilizada, se cuenta con una base sólida para la implementación y ejecución de los experimentos. Esto permite garantizar que las simulaciones se realicen bajo condiciones controladas, que los datos recolectados sean confiables y que los análisis posteriores aporten resultados válidos y pertinentes para responder a las preguntas de investigación planteadas.

Capítulo 7

Análisis de software

Hasta el momento conocemos la metodología utilizada para la investigación completa; sin embargo, es necesario especificar el desarrollo del software, ya que es necesario garantizar el correcto diseño e implementación del simulador para poder realizar las pruebas adecuadas con el menor número de errores. Es por esto que en este capítulo se detalla el proceso de desarrollo del simulador, lo que incluye la especificación de la metodología utilizada, requerimientos del sistema y modelado, diseño e implementación de cada una de las entregas.

7.1. Metodología de desarrollo

Para el desarrollo de software se utiliza un modelo de entrega por etapas. Este modelo, como se observa en la figura 7.1, realiza las primeras etapas de forma secuencial y permite diseñar e implementar los componentes del proyecto por separado. Seguir este modelo permite entregar partes del proyecto y tener visibilidad del progreso.

Estas son algunas de las razones por las cuales se opta por este modelo:

- El tamaño del equipo de desarrollo es reducido: Al tener un número de integrantes reducido, resulta inviable desarrollar múltiples componentes de forma paralela.
- No existe un cliente: El desarrollo de este producto de software es únicamente un medio para el proceso experimental.

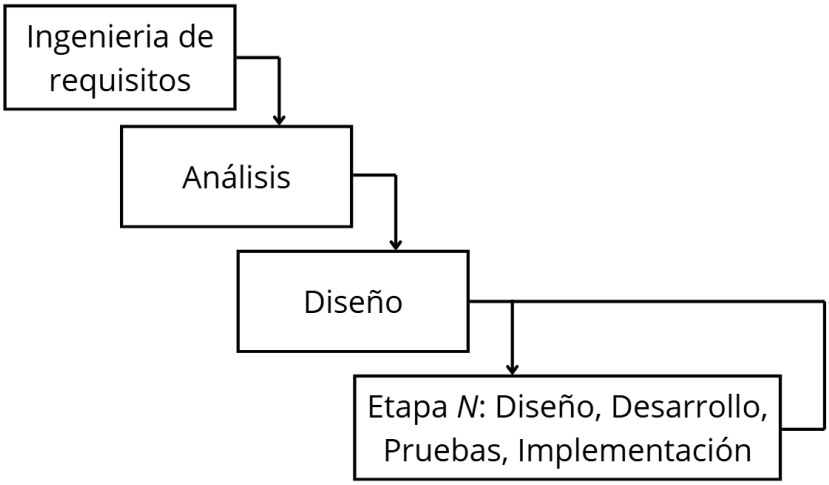


Figura 7.1: Representación gráfica del modelo de entrega por etapas para el desarrollo del software

- No es un producto comercial: No se planea monetizar dado que no existe un cliente. Por lo que en la ingeniería de requisitos se consideran los necesarios propuestos por el equipo de investigación para satisfacer las necesidades tecnológicas.
- No hay procesos iterativos: Al no tener cliente, no es necesario tener una validación externa, ni cambios significativos en los requerimientos del software.

- Simulador funcional del cualquier autómata celular en dos dimensiones en un espacio celular finito.
- Personalización de la vecindad, conjunto de estados, condición de frontera y función de transición.
- Personalización del diseño de estados e interacción en tiempo real con el espacio celular.
- Controles de simulación para avanzar, retroceder, reiniciar y manipular la velocidad de la animación.

Para mayor detalle de esta primer etapa, mas adelante se especifican los casos de uso.

7.2. Reglas de negocio

RN1 El estado de una celda en el tiempo $t + 1$ depende únicamente de su estado actual y de los estados de sus vecinos.

RN2 Para iniciar la simulación es necesario que el conjunto de estados, la topología de la vecindad, la condición de frontera y la función de transición hayan sido definidas.

RN3 Cuando la simulación está en curso, no se puede cambiar el conjunto de estados, la topología de la vecindad ni la función de transición.

RN4 El usuario puede manipular el estado de una célula y la condición de frontera en tiempo de ejecución.

RN5 Los datos importados deben cumplir con el formato definido por el sistema para garantizar su correcta interpretación.

RN6 Los datos exportados deben conservar el mismo formato que los datos importados.

RN7 El usuario puede pausar, detener, reanudar o retroceder la simulación sin pérdida de información.

RN8 Solo se permite una simulación activa por sesión.

7.3. Requerimientos

7.3.1. Requerimientos funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe ofrecer al usuario para cumplir con los objetivos establecidos.

RF1 El sistema debe mostrar una cuadrícula regular y finita que represente el espacio celular.

RF2 El sistema debe permitir al usuario interactuar con el espacio celular, asignando o modificando los estados de las celdas.

RF3 El sistema debe permitir al usuario definir el conjunto de estados, la topología de la vecindad local y las condiciones de frontera del espacio celular.

RF4 El sistema debe permitir la evolución del espacio celular mediante un único paso o de forma continua, ajustando la velocidad de actualización según las capacidades computacionales del sistema.

RF5 El sistema debe permitir la importación y exportación de configuraciones y evoluciones del espacio celular.

RF6 El sistema debe permitir al usuario establecer la función de transición de manera manual.

RF7 El sistema debe permitir configurar la red neuronal para imitar el comportamiento de una evolución señalada.

RF8 El sistema debe entrenar una red neuronal dados los parámetros del usuario y la evolución que desea replicar.

7.3.2. Requerimientos no funcionales

Los requerimientos no funcionales especifican las condiciones de calidad, desempeño y compatibilidad bajo las cuales el sistema debe operar.

RNF1. El sistema debe poder ejecutarse en Fedora 42 sin requerir modificaciones en el código fuente.

RNF2. El sistema debe ser capaz de procesar al menos 10,000 celdas o registros por segundo utilizando una GPU NVIDIA RTX 5080.

RNF3. El sistema debe garantizar disponibilidad continua durante la ejecución, sin fallos o interrupciones críticas.

RNF4. La interfaz del sistema debe tener mensajes claros y funcionalidades bien definidas.

7.3.3. Requerimientos de sistema

7.3.3.1. Recomendado

Los siguientes requerimientos describen el entorno principal y recomendado de ejecución, en el cual el sistema garantiza un rendimiento óptimo y estabilidad durante los procesos de simulación:

- **Procesador:** Intel Core i9-14900K × 32 núcleos.
- **GPU:** NVIDIA GeForce RTX 5080.
- **Sistema operativo:** Fedora Linux 42 (Workstation Edition).
- **Entorno de escritorio:** GNOME 48.
- **Sistema de ventaneo:** Wayland.

7.3.3.2. Mínimo

También se establecen los requisitos mínimos necesarios para usar el simulador, teniendo en cuenta que el rendimiento y la velocidad de procesamiento dependerán de las capacidades del hardware disponible y que el entrenamiento podría no ser posible.

- **Procesador:** Intel Core i7-1185G7 × 4 núcleos.
- **GPU:** Intel Corporation Raptor Lake-S GT1 [UHD Graphics 770].
- **Sistema operativo:** Fedora Linux 42 (Workstation Edition).
- **Entorno de escritorio:** GNOME 48.
- **Sistema de ventaneo:** Wayland.

7.4. Análisis de riesgos

El análisis de riesgos se realiza tomando en cuenta el tiempo estimado de desarrollo de aproximadamente 6 meses, los recursos disponibles para el desarrollo tecnológico y las habilidades técnicas para el diseño de simuladores de autómatas celulares, cómputo paralelo, redes neuronales y programación orientada a eventos.

7.4.1. Identificación de riesgos

7.4.1.1. Riesgos técnicos

RT1 Incompatibilidad entre diferentes lenguajes de programación: los módulos desarrollados en Python, C++ y CUDA generan errores de integración al

intercambiar información y sincronizar procesos.

RT2 Indisponibilidad de hardware: La tarjeta gráfica no tiene el rendimiento suficiente.

RT3 Incompatibilidad entre tecnologías: El sistema operativo, los lenguajes de programación y los elementos de hardware no son compatibles para el desarrollo y ejecución.

RT4 Dependencias de librerías: La instalación y configuración de diferentes librerías varía según el sistema operativo.

RT5 Mala visualización: El renderizado de los componentes gráficos es incorrecto o está mal optimizado.

7.4.1.2. Riesgos operativos

RO1 Documentación deficiente: La documentación no permite mantener ni ampliar el proyecto a futuro.

RO2 Problemas personales: Falta de motivación o problemas de salud física o mental se presentan durante el desarrollo.

RO3 Curva de aprendizaje elevada: Se encuentra una curva de aprendizaje elevada y se carece de experiencias con la tecnología a utilizar.

7.4.1.3. Riesgos de requerimientos

RR1 Requerimientos ambiguos: La definición incompleta o ambigua de los requerimientos puede provocar cambios en etapas avanzadas del desarrollo.

7.4.1.4. Riesgos de estimación

RE1 Subestimación del tiempo de desarrollo: La complejidad de enlazar diferentes tecnologías puede extender los plazos planeados.

7.4.1.5. Riesgos de alcance

RA1 Limitación del alcance funcional: Las restricciones de tiempo pueden dar pie a limitar el número de funcionalidades requeridas.

RA2 Carga de trabajo imprevista: El poder de cómputo resulta insuficiente, por lo que se reduce la complejidad de las simulaciones.

7.4.2. Probabilidad e impacto

Siguiendo la tabla 7.1, a continuación se enlistan los niveles de riesgo para cada riesgo identificado.

		Probabilidad				
		Nada probable 10 %	Levemente probable 30 %	Poco probable 50 %	Probable 70 %	Muy probable 90 %
Severidad	Nula	Bajo	Bajo	Bajo	Medio	Medio
	Menor	Bajo	Bajo	Medio	Medio	Medio
	Moderado	Medio	Medio	Medio	Alto	Alto
	Crítico	Medio	Medio	Alto	Alto	Muy alto
	Fatal	Medio	Alto	Alto	Muy alto	Muy alto

Tabla 7.1: Matriz de riesgos [152]

Riesgo técnico	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Incompatibilidad entre diferentes lenguajes de programación	Poco probable	Crítico	Alto
Indisponibilidad de hardware	Levemente probable	Moderado	Medio
Incompatibilidad entre tecnologías	Probable	Fatal	Muy alto
Dependencias de librerías	Levemente probable	Menor	Bajo
Mala visualización	Poco probable	Menor	Medio

Tabla 7.2: Nivel de riesgos técnicos

Riesgo operativo	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Documentación deficiente	Probable	Moderado	Alto
Problemas personales	Poco probable	Crítico	Alto
Curva de aprendizaje elevada	Muy probable	Moderado	Alto

Tabla 7.3: Nivel de riesgos operativos

Riesgo de requerimientos	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Requerimientos ambiguos	Levemente probable	Menor	Bajo

Tabla 7.4: Nivel de riesgos de requerimientos

Riesgo de estimación	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Subestimación del tiempo de desarrollo	Probable	Fatal	Muy alto

Tabla 7.5: Nivel de riesgos de estimación

Riesgo de alcance	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Limitación del alcance funcional	Poco probable	Moderado	Medio
Carga de trabajo imprevista	Probable	Crítico	Alto

Tabla 7.6: Nivel de riesgos de alcance

7.4.3. Plan de mitigación

Riesgos técnicos	
Incompatibilidad entre diferentes lenguajes	Verificar la documentación de compatibilidad entre los lenguajes de programación y realizar pruebas de transferencia de datos y sincronización previas al desarrollo.
Indisponibilidad de hardware	Asegurar la disponibilidad de equipos de respaldo o entornos virtuales alternativos para continuar el desarrollo en caso de fallos de hardware.
Incompatibilidad entre tecnologías	Realizar pruebas de compatibilidad del hardware y software en diferentes sistemas operativos y versiones antes de la integración.
Dependencias de librerías	Documentar y verificar las versiones de librerías utilizadas, además de considerar alternativas equivalentes que garanticen continuidad en caso de incompatibilidad.
Mala visualización	Realizar pruebas de usabilidad y ajustar la interfaz de acuerdo con las resoluciones de pantalla y dispositivos objetivo.

Tabla 7.7: Plan de mitigación de riesgos técnicos

Riesgos operativos	
Documentación deficiente	Establecer un estándar de documentación y realizar revisiones periódicas del avance para garantizar la calidad y completitud de los registros técnicos.
Problemas personales	Definir roles de respaldo o distribuir responsabilidades para minimizar el impacto de ausencias o contratiempos personales.
Curva de aprendizaje elevada	Programar capacitaciones iniciales y establecer un periodo de adaptación con apoyo técnico y materiales de referencia.

Tabla 7.8: Plan de mitigación de riesgos operativos

Riesgos de requerimientos	
Requerimientos ambiguos	Realizar reuniones periódicas con los usuarios o stakeholders para aclarar requerimientos y validar el entendimiento del equipo de desarrollo.

Tabla 7.9: Plan de mitigación de riesgos de requerimientos

Riesgos de estimación	
Subestimación del tiempo de desarrollo	Dividir el proyecto en fases pequeñas y manejables, aplicando seguimiento semanal y reajuste de cronograma según los resultados obtenidos.

Tabla 7.10: Plan de mitigación de riesgos de estimación

Riesgos de alcance	
Limitación del alcance funcional	Definir claramente los objetivos y alcances desde el inicio, con aprobación formal de los responsables del proyecto antes de iniciar nuevas funcionalidades.
Carga de trabajo imprevista	Implementar revisiones periódicas del plan de trabajo y ajustar el alcance en función de los recursos disponibles y los resultados obtenidos.

Tabla 7.11: Plan de mitigación de riesgos de alcance

7.4.4. Plan de contingencia

A continuación se describen los planes de contingencia para los riesgos de nivel fatal:

RT3 Incompatibilidad entre tecnologías.

RE1 Subestimación del tiempo de desarrollo.

Nombre del riesgo	**Incompatibilidad entre tecnologías**
Probabilidad	Probable
Impacto	Fatal
Plan de evasión	Realizar pruebas de integración tempranas entre las distintas tecnologías implicadas. Documentar las versiones, dependencias y configuraciones utilizadas para garantizar compatibilidad. Establecer un entorno de desarrollo unificado que minimice diferencias entre plataformas.
Mitigación	Sustituir o actualizar los componentes que generen incompatibilidad. Implementar adaptadores o módulos intermedios que permitan la interoperabilidad entre sistemas. Reasignar recursos técnicos para resolver los conflictos de integración y evitar retrasos prolongados.

Tabla 7.12: Plan de contingencia para incompatibilidad entre tecnologías

Nombre del riesgo	**Subestimación del tiempo de desarrollo**
Probabilidad	Probable
Impacto	Fatal
Plan de evasión	Realizar estimaciones realistas basadas en experiencias previas y tareas medibles. Incluir márgenes de tiempo de reserva en el cronograma para absorber imprevistos. Efectuar reuniones de seguimiento semanales para detectar desviaciones tempranas.
Mitigación	Reasignar recursos o personal adicional para recuperar el tiempo perdido. Reducir temporalmente el alcance funcional o posponer características no críticas. Reajustar el cronograma de entregas y comunicar los cambios al equipo y partes interesadas.

Tabla 7.13: Plan de contingencia para subestimación del tiempo de desarrollo

7.5. Estudio de factibilidad

El proyecto cuenta con los recursos de hardware necesarios para llevar a cabo simulaciones de alto nivel computacional. El equipo de cómputo disponible permite implementar técnicas de paralelización, asegurando la capacidad de realizar múltiples inferencias en tiempos razonables. Adicionalmente, se dispone de herramientas tecnológicas de alto rendimiento que brindan la ventaja computacional requerida para ejecutar la simulación de manera eficiente.

En cuanto a la factibilidad económica, la mayor parte del software utilizado es de libre acceso, incluyendo Python, C++, Visual Studio Code y TeXstudio. Se utiliza también Wolfram Mathematica, que requiere una licencia anual de 195 dólares, la cual se encuentra disponible de manera gratuita para estudiantes del IPN, garantizando así la viabilidad financiera del proyecto.

Desde el punto de vista operativo, el diseño de la interfaz facilita la definición de reglas y la ejecución de simulaciones, optimizando el flujo de trabajo del grupo de investigadores. Esto permite acelerar significativamente el proceso de experimentación y análisis de resultados, maximizando la eficiencia del equipo durante el desarrollo del proyecto.

Finalmente, se cumple la factibilidad temporal, ya que el cronograma de desarrollo estimado permite realizar todas las simulaciones necesarias para el análisis posterior, asegurando que el proyecto se pueda completar dentro del tiempo previsto.

7.6. Casos de uso

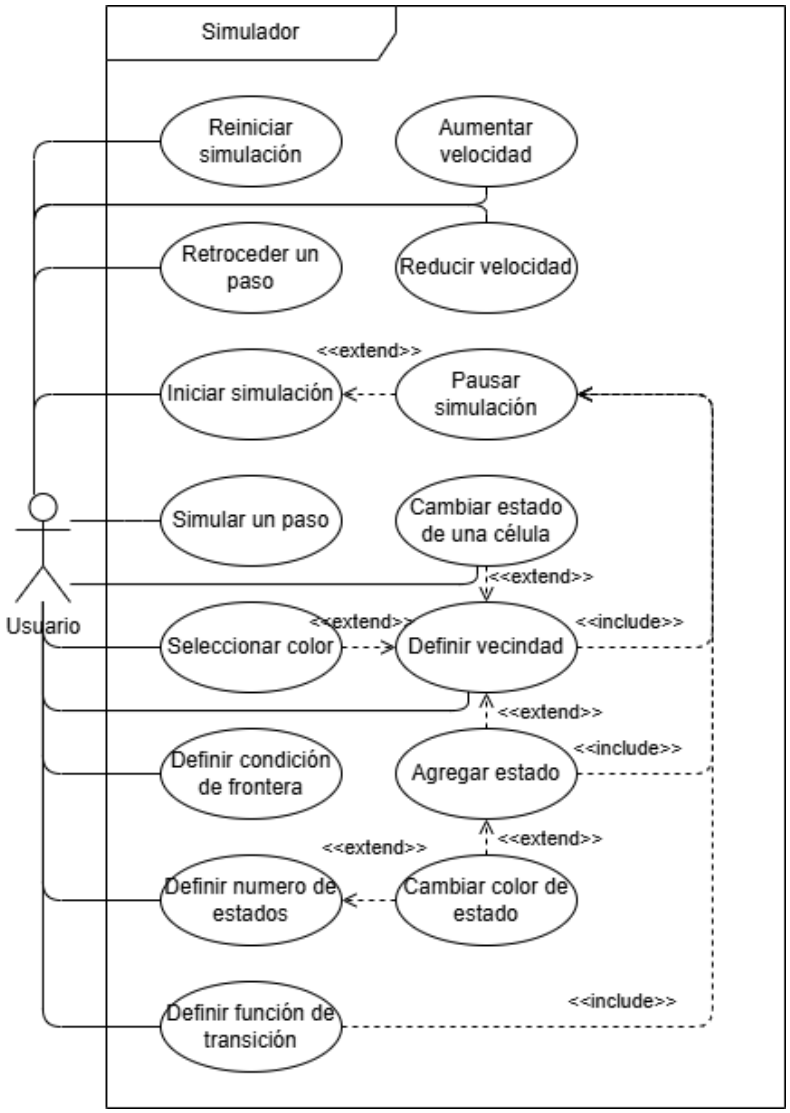


Figura 7.2: Diagrama de casos de uso

7.7. Diccionario

7.7.1. Entradas

- Controles de simulación:

    - BTN1 Iniciar simulación
    - BTN2 Pausar simulación
    - BTN3 Aumentar velocidad
    - BTN4 Disminuir velocidad
    - BTN5 Simular un paso
    - BTN6 Retroceder la simulación un paso
    - BTN7 Reiniciar simulación
- Configuración del autómata celular:
    - BTN8 Definir vecindad
    - BTN9 Añadir vecindad
    - BTN10 Estado
    - BTN11 Cambiar frontera
    - BTN12 Definir estados
    - BTN13 Definir regla
    - IN1 Estados
    - IN2 Regla
- Definición de la interfaz de vecindad:
    - BTN14 Guardar cambio de vecindad
    - BTN15 Cancelar cambio de vecindad

7.7.2. Mensajes

- MSG1 Vecindad generada con incompatibilidad
- MSG2 Incompatibilidad de estados

7.7.3. Interfaces gráficas

- UI1 Control: Elemento que contiene el control sobre la simulación del espacio celular.
- UI2 Configuración: Elemento que permite realizar la configuración de la condición de frontera, los estados y la función de transición.

- UI3 Vecindad: Ventana emergente que contiene un espacio celular donde cada célula actúa. Muestra la célula central y permite al usuario seleccionar aquellas células que quiere que formen parte de la vecindad.
- UI4 Simulación: Elemento que contiene la cuadrícula del espacio celular y muestra la evolución del autómata celular.

7.8. Especificaciones de casos de uso

7.8.1. Controles de simulación

Los siguientes casos de uso permiten al usuario controlar la evolución del autómata celular. Esto implica evolucionar la sistema hacia adelante, hacia atrás y definiendo velocidades.

7.8.1.1. UC01 Iniciar simulación

Actor	Usuario
Descripción	El sistema inicia la evolución en pasos de tiempo constante.
Tipo	Primario
Entradas	▪ Desde mouse BTN1 Iniciar simulación.
Salidas	Ninguna
Precondición	▪ Simulación detenida. ▪ Botón BTN1 Iniciar simulación habilitado.
Postcondición	▪ Simulación en ejecución. ▪ Botón BTN1 Iniciar simulación deshabilitado. ▪ Botón BTN2 Pausar simulación habilitado.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN1 Iniciar simulación desde UI1 Control.	2. Inicia la simulación. Cada paso evolutivo se realiza a una velocidad constante. 3. Deshabilita el botón BTN1 Iniciar simulación. 4. Habilita el botón BTN2 Pausar simulación.

7.8.1.2. UC02 Pausar simulación

Actor	Usuario
Descripción	Se detiene la simulación.
Tipo	Primario
Entradas	▪ Desde mouse BTN2 Pausar simulación.
Salidas	Ninguna
Precondición	▪ Simulación en ejecución. ▪ Botón BTN2 Pausar simulación habilitado.
Postcondición	▪ Simulación detenida. ▪ Botón BTN2 Pausar simulación deshabilitado. ▪ Botón BTN1 Iniciar simulación habilitado.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN2 Pausar simulación desde UI1 Control.	2. Detiene la simulación. 3. Deshabilita el botón BTN2 Pausar simulación. 4. Habilita el botón BTN1 Iniciar simulación.

7.8.1.3. UC03 Aumentar velocidad

Actor	Usuario
Descripción	Se aumenta la velocidad de actualización del autómata celular.
Tipo	Secundario
Entradas	▪ Desde mouse BTN3 Aumentar velocidad.
Salidas	Ninguna
Precondición	▪ La simulación esta inicializada.
Postcondición	▪ Aumenta la velocidad de simulación.
Errores	▪ Velocidad límite: Se intenta aumentar la velocidad cuando ya encuentra en su velocidad máxima.

Flujo básico	
Actor	**Sistema**
1. Presiona BTN3 Aumentar velocidad desde UI1 Control.	2. Aumenta la velocidad de la simulación.

Flujo alterno 1: Velocidad límite	
Actor	**Sistema**
1.1. Presiona BTN3 Aumentar velocidad desde UI1 Control cuando está en la velocidad límite máxima.	1.2. No realiza ningún cambio, mantiene la velocidad actual.

7.8.1.4. UC04 Reducir velocidad

Actor	Usuario
Descripción	Se disminuye la velocidad de actualización del autómata celular.
Tipo	Secundario
Entradas	▪ Desde mouse BTN4 Disminuir velocidad.
Salidas	Ninguna
Precondición	▪ La simulación esta inicializada.
Postcondición	▪ Disminuye la velocidad de simulación.
Errores	▪ Velocidad límite: Se intenta disminuir la velocidad cuando ya encuentra en su velocidad mínima.

Flujo básico	
Actor	**Sistema**
1. Presiona BTN4 Disminuir velocidad desde UI1 Control.	2. Disminuye la velocidad de la simulación.

Flujo alterno 1: Velocidad límite	
Actor	**Sistema**
1.1. Presiona BTN4 Disminuir velocidad desde UI1 Control cuando está en la velocidad límite mínima.	1.2. No realiza ningún cambio, mantiene la velocidad actual.

7.8.1.5. UC05 Simular un paso

Actor	Usuario
Descripción	Se realiza la simulación del CA, pero únicamente en un paso.
Tipo	Primario
Entradas	▪ Desde mouse BTN5 Simular un paso.
Salidas	Ninguna
Precondición	Ninguna.
Postcondición	▪ Evoluciona el sistema.
Errores	▪ Velocidad máxima: Se intenta dar un paso cuando la simulación está activa en la velocidad máxima.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN5 Simular un paso desde UI1 Control.	2. La simulación avanza un paso.

Flujo alterno 1: Velocidad máxima	
Actor	**Sistema**
1.1. Presiona el botón BTN5 Simular un paso desde UI1 Control cuando la simulación esta activa y la velocidad esta en la velocidad máxima.	1.2. No realiza ningún cambio y mantiene la velocidad actual.

7.8.1.6. UC06 Retroceder un paso

Actor	Usuario
Descripción	El sistema restaura el estado anterior de la simulación.
Tipo	Primario
Entradas	▪ Desde mouse BTN6 Retroceder la simulación un paso.
Salidas	Ninguna
Precondición	▪ Existe un estado previo.
Postcondición	▪ La simulación se pausa. ▪ El sistema restaura la evolución pasada.
Errores	▪ Configuración inicial: El usuario intenta retroceder la simulación de la configuración inicial. ▪ Simulación activa: El usuario intenta retroceder la simulación mientras esta se encuentra activa.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN6 Retroceder la simulación un paso desde UI1 Control.	2. La simulación retrocede un paso.

Flujo alterno 1: Configuración inicial	
Actor	**Sistema**
1.1. Presiona el botón BTN6 Retroceder la simulación un paso desde UI1 Control estando en la configuración inicial (no hay más configuraciones antes).	1.2. No realiza ningún cambio.

Flujo alterno 2: Simulación activa	
Actor	**Sistema**
2.1. Presiona el botón BTN6 Retroceder la simulación un paso desde UI1 Control mientras la simulación esta activa.	2.2. La simulación se pausa. 2.3. La simulación retrocede un paso.

7.8.1.7. UC07 Reiniciar simulación

Actor	Usuario
Descripción	Restablece la simulación a su estado inicial (todas las celdas vacías o configuración guardada).
Tipo	Secundario
Entradas	▪ Desde mouse BTN7 Reiniciar simulación.
Salidas	Ninguna
Precondición	Ninguna
Postcondición	▪ El sistema adquiere la configuración inicial o nula.
Errores	▪ Configuración inicial: El usuario intenta reiniciar la simulación de la configuración inicial.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN7 Reiniciar simulación desde UI1 Control.	2. Regresa la configuración a la última versión modificada mientras estaba en pausa.

Flujo alterno 1: Configuración inicial	
Actor	**Sistema**
1.1. Presiona el botón BTN7 Reiniciar simulación desde UI1 Control estando en la configuración inicial (no hay más configuraciones antes).	1.2. Establece todas las células a 0.

7.8.2. Configuración del autómata celular

Estos casos de uso definen las características generales del autómata celular, como el numero de estados, vecindad y la función de transición.

7.8.2.1. UC08 Definir vecindad

Actor	Usuario
Descripción	Se define el conjunto de posiciones relativas de cada célula que forman parte de la vecindad.
Tipo	Primario
Entradas	▪ Desde mouse BTN8 Definir vecindad. ▪ Desde mouse BTN14 Guardar cambio de vecindad. ▪ Desde mouse BTN15 Cancelar cambio de vecindad.
Salidas	▪ Mensaje MSG1 Vecindad generada con incompatibilidad
Precondición	▪ Simulación en pausa.
Postcondición	▪ Vecindad guardada en el sistema.
Errores	▪ Simulando: Se intenta cambiar la vecindad mientras está una simulación en proceso. ▪ Cancelar: Se cancela el cambio de vecindad. ▪ Incompatibilidad: La regla es incompatible con la nueva vecindad seleccionada.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN8 Definir vecindad en UI2 Configuración. 3. Selecciona los elementos que forman parte de la vecindad seleccionando las células. 4. Presiona el botón BTN14 Guardar cambio de vecindad en UI3 Vecindad.	2. Abre la interfaz UI3 Vecindad. 5. Actualiza la vecindad.

Flujo alterno 1: Simulando	
Actor	**Sistema**
1.1. Presiona el botón BTN8 Definir vecindad en UI2 Configuración mientras la simulación está activa. 1.4. Selecciona los elementos que forman parte de la vecindad seleccionando las células. 1.5. Presiona el botón BTN14 Guardar cambio de vecindad en UI3 Vecindad.	1.2. Detiene la simulación. 1.3. Abre la interfaz UI3 Vecindad. 1.6. Actualiza la vecindad.

Flujo alterno 2: Cancelar	
Actor	**Sistema**
2.4. Presiona el botón de BTN15 Cancelar cambio de vecindad en UI3 Vecindad.	2.5. No se actualiza la vecindad.

Flujo alterno 3: Incompatibilidad (Aceptar)	
Actor	**Sistema**
3.3. Selecciona los elementos que forman parte de la vecindad seleccionando las células. 3.4. Presiona el botón BTN14 Guardar cambio de vecindad en UI3 Vecindad. 3.6. Acepta el cambio.	3.5. Muestra el mensaje MSG1 Vecindad generada con incompatibilidad. 3.7. Actualiza la vecindad.

Flujo alterno 4: Incompatibilidad (Rechazar)	
Actor	**Sistema**
4.3. Selecciona los elementos que forman parte de la vecindad seleccionando las células. 4.4. Presiona el botón BTN14 Guardar cambio de vecindad. 4.6. Rechaza el cambio.	4.5. Muestra el mensaje MSG1 Vecindad generada con incompatibilidad. 4.7. No actualiza la vecindad.

7.8.2.2. UC09 Agregar estado

Actor	Usuario
Descripción	Se añade una nueva vecindad semitotalistica.
Tipo	Secundario
Entradas	▪ Desde mouse BTN9 Añadir vecindad
Salidas	Ninguna
Precondición	▪ Estar en la interfaz UI3 Vecindad
Postcondición	▪ Se añade la nueva vecindad semitotalistica. ▪ La interfaz refleja el nuevo elemento agredo.
Errores	Ninguno.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN9 Añadir vecindad en UI3 Vecindad	2. Añade una nueva vecindad semitotalistica. 3. Añade un botón BTN10 Estado.

7.8.2.3. UC10 Definir condición de frontera

Actor	Usuario
Descripción	Se selecciona entre diferentes opciones de condiciones de frontera.
Tipo	Secundario
Entradas	▪ Desde mouse BTN11 Cambiar frontera.
Salidas	Ninguna
Precondición	Ninguna
Postcondición	▪ Se actualiza la condición de frontera.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN11 Cambiar frontera desde UI2 Configuración.	2. Actualiza la condición de frontera a la siguiente configuración.

7.8.2.4. UC11 Definir numero de estados

Actor	Usuario
Descripción	Se define el tamaño del conjunto de estados.
Tipo	Primario
Entradas	▪ Desde teclado IN1 Estados. ▪ Desde mouse BTN12 Definir estados.
Salidas	▪ Mensaje MSG2 Incompatibilidad de estados.
Precondición	▪ El sistema debe estar detenido. ▪ El valor ingresado debe de ser un numero entero valido.
Postcondición	▪ Se actualizan los estados.
Errores	▪ Simulando: El usuario intenta cambiar los estados mientras está en ejecución. ▪ Incompatibilidad: El número de estados es menor y hay estados inexistentes en el espacio celular. ▪ Invalido: El numero de estados es invalido.

Flujo básico	
Actor	**Sistema**
1. Ingresa el número de estados en el campo IN1 Estados en UI2 Configuración mayor al actual. 2. Presiona el botón BTN12 Definir estados en UI2 Configuración.	3. Cambia el conjunto de estados. 4. Añade los botones BTN10 Estado necesarios a UI2 Configuración.

Flujo alterno 1: Simulando	
Actor	**Sistema**
1.2. Presiona el botón BTN12 Definir estados en UI2 Configuración mientras la simulación está en ejecución.	1.3. Detiene la simulación. 1.4. Cambia el conjunto de estados. 1.5. Añade los botones BTN10 Estado necesarios a UI2 Configuración.

Flujo alterno 2: Incompatibilidad (Aceptar)	
Actor	**Sistema**
2.1. Ingresa el número de estados en el campo IN1 Estados desde UI2 Configuración, menor al actual. 2.2. Presiona el botón BTN12 Definir estados en UI2 Configuración. 2.4. Acepta el cambio.	2.3. Muestra el mensaje MSG2 Incompatibilidad de estados. 2.5. Cambia el conjunto de estados. 2.6. Establece a 0 todos los estados inexistentes de las células. 2.7. Elimina los botones BTN10 Estado necesarios a UI2 Configuración.

Flujo alterno 3: Incompatibilidad (Rechazar)	
Actor	**Sistema**
3.1. Ingresa el número de estados en el campo IN1 Estados desde UI2 Configuración, menor al actual. 3.2. Presiona el botón BTN12 Definir estados en UI2 Configuración. 3.4. Rechaza el cambio.	3.3. Muestra el mensaje MSG2 Incompatibilidad de estados. 3.5. No hace ningún cambio.

Flujo alterno 4: Invalido	
Actor	**Sistema**
4.1 Ingresa el número de estados en el campo IN1 Estados en UI2 Configuración menor a 1. 4.2 Presiona el botón BTN12 Definir estados en UI2 Configuración.	4.3. cambia el valor de IN1 Estados por 1. 4.4. Muestra el mensaje MSG2 Incompatibilidad de estados.

7.8.2.5. UC12 Definir función de transición

Actor	Usuario
Descripción	Se configura la regla que define el comportamiento del CA.
Tipo	Primario
Entradas	▪ Desde teclado IN2 Regla. ▪ Desde mouse BTN13 Definir regla.
Salidas	Ninguna
Precondición	▪ El sistema se encuentra detenido. ▪ La regla esta dentro del rango permitido.
Postcondición	▪ Se guarda la nueva regla de transición.
Errores	▪ Simulando: El cambio de regla se intenta aplicar cuando la simulación está en ejecución. ▪ Fuera de rango: La regla establecida está fuera del rango.

Flujo básico	
Actor	**Sistema**
1. Ingresa la regla en el campo IN2 Regla en UI2 Configuración. 2. Presiona el botón BTN13 Definir regla.	3. Actualiza la regla.

Flujo alterno 1: Simulando	
Actor	**Sistema**
1.2. Presiona el botón BTN13 Definir regla mientras la simulación está activa.	1.3. Pausa la simulación. 1.4. Actualiza la regla.

Flujo alterno 2: Fuera de rango	
Actor	**Sistema**
2.1. Ingresa la regla fuera del rango de reglas en el campo IN2 Regla en UI2 Configuración. 2.2. Presiona el botón BTN13 Definir regla mientras la simulación está activa.	2.3. La regla no es compatible, se ajusta a la configuración del CA. 2.4. Actualiza la regla.

7.8.3. Interacción con el espacio celular

El usuario puede interactuar con el espacio celular para cambiar valores, colores o vecindades. Los siguientes casos de uso están diseñado para mejorar la visibilidad del espacio celular, tanto en UI1 Control como en UI3 Vecindad.

7.8.3.1. UC13 Seleccionar color

Actor	Usuario
Descripción	Se selecciona un estado para cambiar el estado de las células en el espacio celular.
Tipo	Secundario
Entradas	▪ Desde mouse BTN10 Estado.
Salidas	Ninguna
Precondición	▪ Debe de haber botones de BTN10 Estado.
Postcondición	▪ Se guarda el identificador del último BTN10 Estado seleccionado.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona un botón de BTN10 Estado en UI2 Configuración.	2. Guarda el estado del ultimo BTN10 Estado seleccionado.

7.8.3.2. UC14 Cambiar color de estado

Actor	Usuario
Descripción	Se define un color asociado a cada estado.
Tipo	Secundario
Entradas	▪ Desde mouse BTN10 Estado.
Salidas	Ninguna
Precondición	Ninguna
Postcondición	▪ Se le asigna un color al estado seleccionado. ▪ Todas las células con ese estado actualizan su color en el espacio celular. ▪ La configuración de color queda almacenada.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona dos veces un BTN10 Estado en UI2 Configuración. 2. Selecciona un color.	3. Abre un panel de colores. 4. Asigna ese color a ese estado. 5. Cambia el color de todas las células con ese estado en el espacio celular.

7.8.3.3. UC15 Cambiar estado de una célula

Actor	Usuario
Descripción	Se cambia manualmente la configuración del espacio celular, seleccionando el estado de cada célula.
Tipo	Secundario
Entradas	Ninguna
Salidas	Ninguna
Precondición	1. Haber seleccionado un color de un BTN10 Estado.
Postcondición	1. La célula cambia de estado.
Errores	1. Sin estado: No se ha seleccionado un estado.

Flujo básico	
Actor	**Sistema**
1. Selecciona una célula desde UI4 Simulación.	2. La célula se le asigna el estado seleccionado.

Flujo alterno 1: Sin estado	
Actor	**Sistema**
1.1 Selecciona una célula desde UI4 Simulación sin haber seleccionado un BTN10 Estado.	1.2. La célula cambia al estado siguiente.

7.9. Diagramas de secuencia

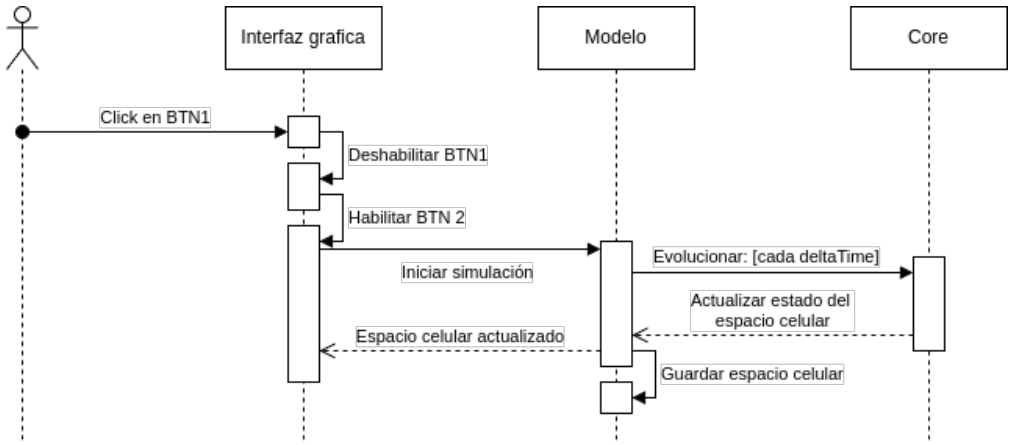


Figura 7.3: Diagrama de secuencia: UC01 Iniciar simulación.

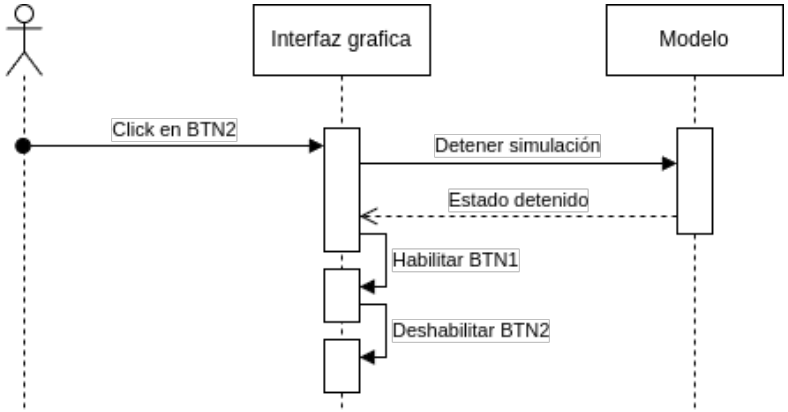


Figura 7.4: Diagrama de secuencia: UC02 Pausar simulación.

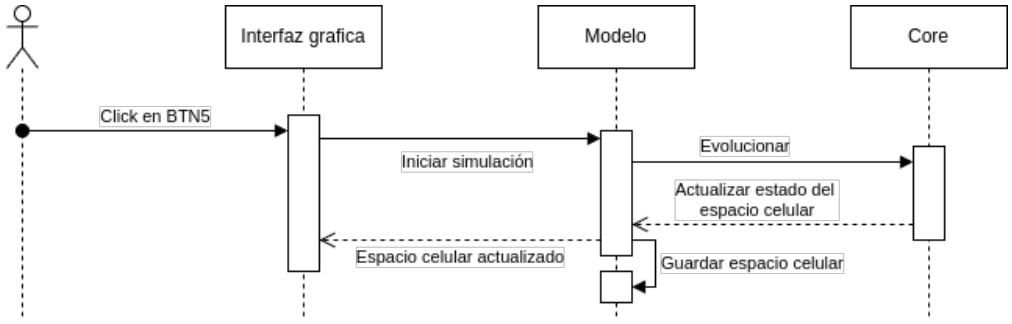


Figura 7.5: Diagrama de secuencia: UC05 Simular un paso.

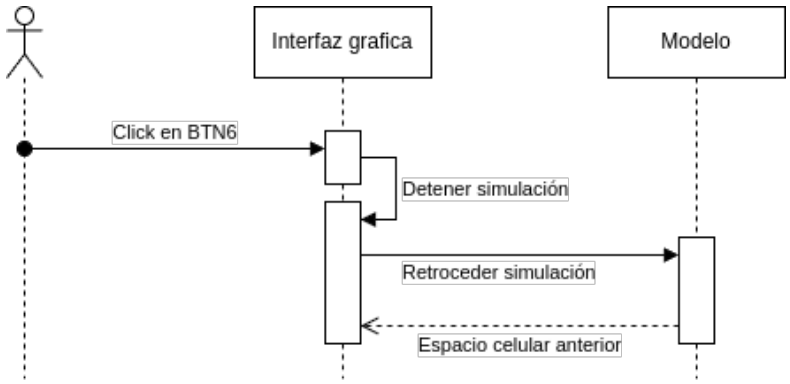


Figura 7.6: Diagrama de secuencia: UC06 Retroceder un paso.

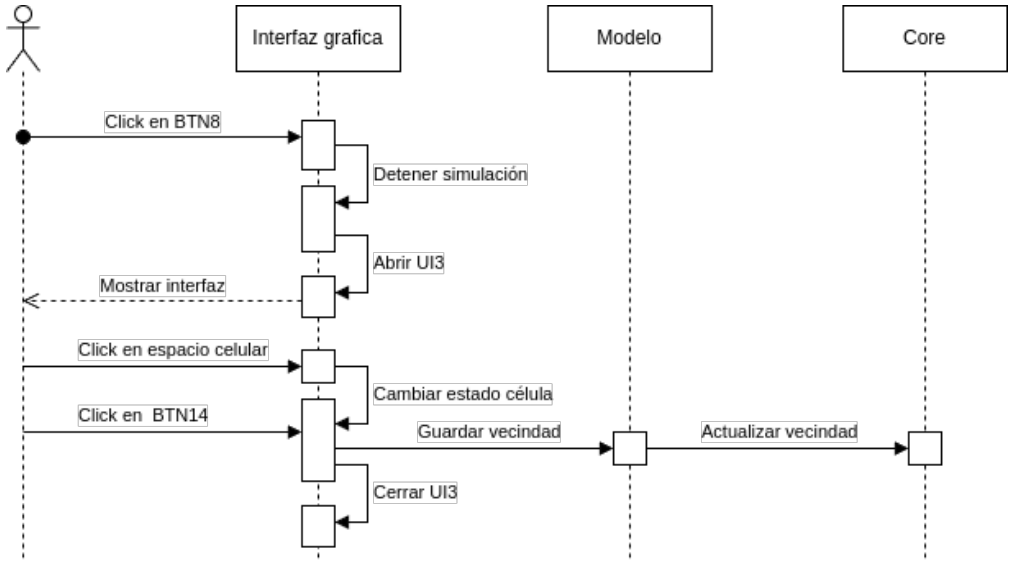


Figura 7.7: Diagrama de secuencia: UC08 Definir vecindad.

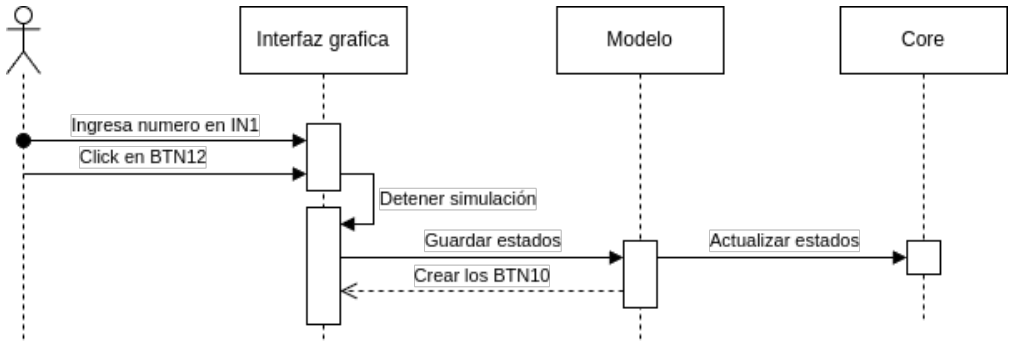


Figura 7.8: Diagrama de secuencia: UC11 Definir numero de estados.

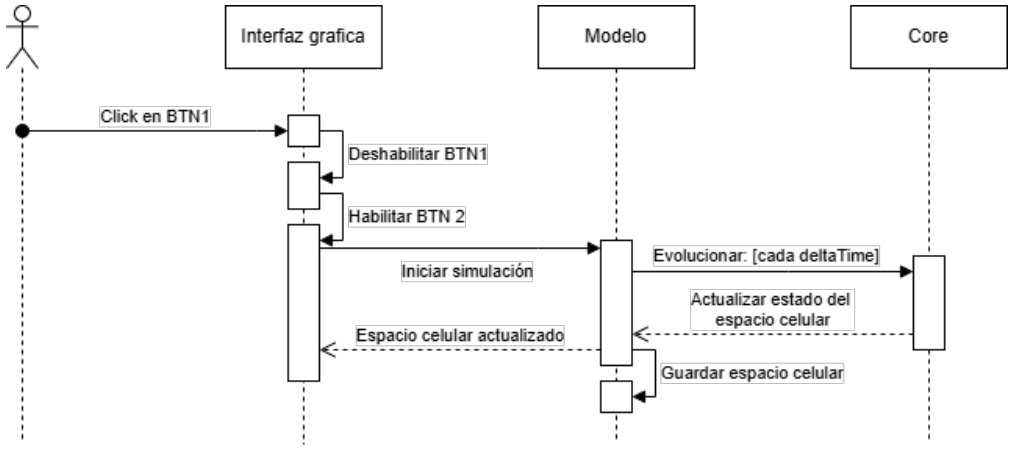


Figura 7.9: Diagrama de secuencia: UC12 Definir función de transición.

Capítulo 8

Diseño de software

8.1. Arquitectura

La arquitectura del sistema se basa en el patrón de diseño modelo vista controlador, el cual resulta adecuado para aplicaciones que combinan lógica de negocio, manipulación de datos y una interfaz gráfica interactiva [153]. Este patrón favorece la separación de responsabilidades, la mantenibilidad y la extensibilidad del sistema.

- **Modelo:** Gestiona los datos y la lógica principal del autómata celular, incluyendo la definición de estados, reglas, vecindad y condiciones de frontera.
- **Vista:** Se encarga de la presentación visual de la información y la interacción con el usuario mediante la interfaz gráfica.
- **Controlador:** Actúa como intermediario entre el modelo y la vista, gestionando los eventos de usuario y actualizando el modelo o la interfaz según corresponda.

8.1.1. Vista

La vista está implementada en *Python* utilizando el framework **PyQt5**. Esta capa proporciona la interfaz gráfica principal del simulador, permitiendo visualizar el espacio celular, modificar parámetros y controlar la ejecución de la simulación (inicio, pausa, reinicio, etc.). Además, se incluyen componentes dinámicos que

permiten la selección de colores, la configuración de reglas y la representación visual del estado del sistema en tiempo real.

8.1.2. Modelo

El modelo encapsula la lógica del autómata celular. Esta capa está implementada en *C++* con el fin de optimizar el rendimiento en las operaciones de actualización de celdas y el procesamiento paralelo. El modelo gestiona la estructura del espacio celular, los estados, la topología de la vecindad, las condiciones de frontera y la regla de transición, garantizando un funcionamiento eficiente y escalable.

8.1.3. Controlador

El **controlador** coordina la comunicación entre la vista y el modelo. Implementado en Python, este componente recibe los eventos generados por la interfaz (por ejemplo, el cambio de número de estados o la selección de una regla), actualiza el modelo y refleja los resultados en la interfaz gráfica. Además, sincroniza los procesos de simulación y representación visual, permitiendo la ejecución paralela y una interacción fluida entre el usuario y la lógica del sistema.

8.2. Diagrama de clases

El diagrama de clases UML sirve para mostrar una vista estática de las clases de un sistema, sus atributos, métodos y relaciones entre objetos. Sirve para proporcionar una información general sobre la aplicación [154].

Como se muestra en la figura 8.1, el simulador está compuesto por siete clases. De estas, únicamente la clase CASimulator se encuentra implementada en C++, por lo que se especifican su encapsulamiento y el tipo de sus variables.

El resto de las clases están desarrolladas en Python y se definen siguiendo la filosofía "pythónica", en la cual "lo simple es mejor" [133].

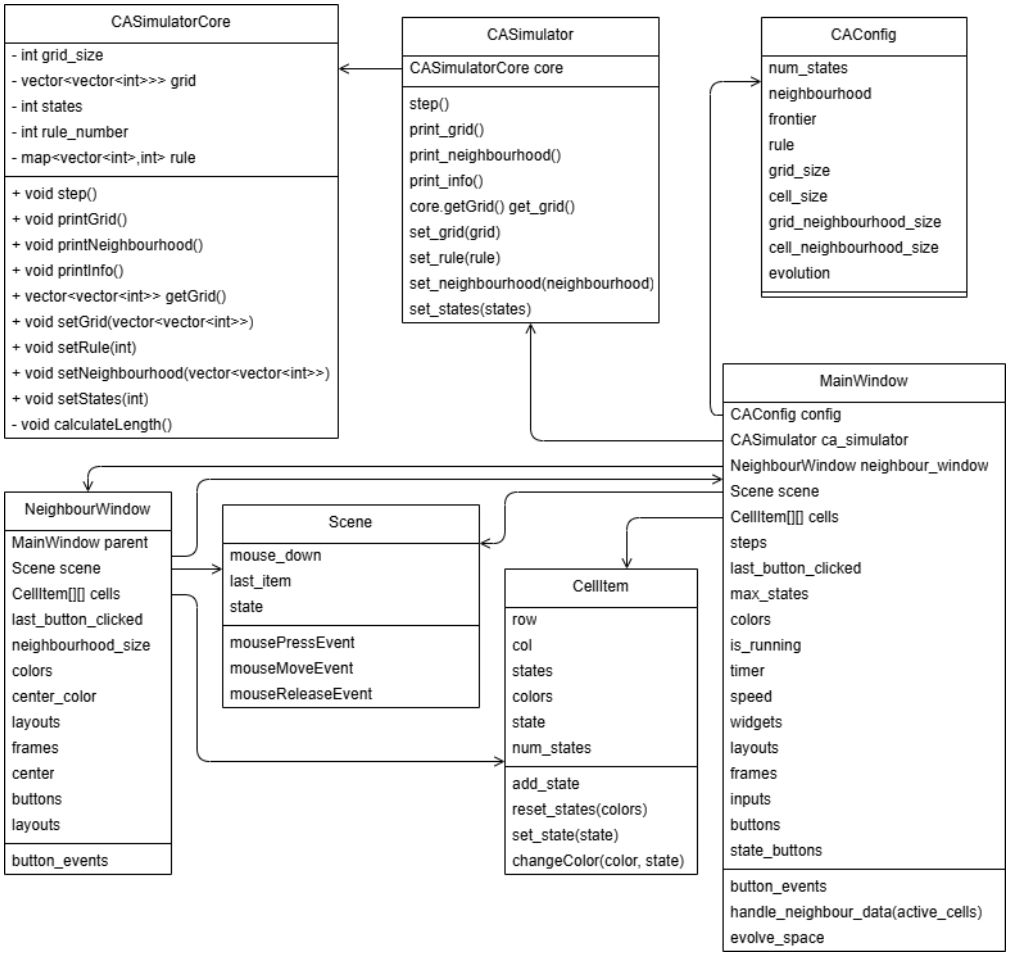


Figura 8.1: Diagrama de clases.

8.3. Interfaces de usuarios

A continuación se muestran las interfaces desarrolladas en PyQt5. Se busca mantener una estructura intuitiva y modular que facilite la configuración, ejecución y visualización del CA.

- **Interfaz principal**

  La interfaz principal se divide en tres secciones: el espacio de evolución, ubicado en la parte superior izquierda, donde se visualiza la cuadrícula en la que evoluciona el autómata celular; el área de controles, situada en la parte inferior izquierda, que permite manipular la simulación (iniciar, pausar,

reiniciar, avanzar o retroceder); y la sección de configuración, ubicada a la derecha, donde se especifican parámetros como la vecindad, las condiciones de frontera, los estados, los colores asociados y la regla de transición (Figura 8.2).

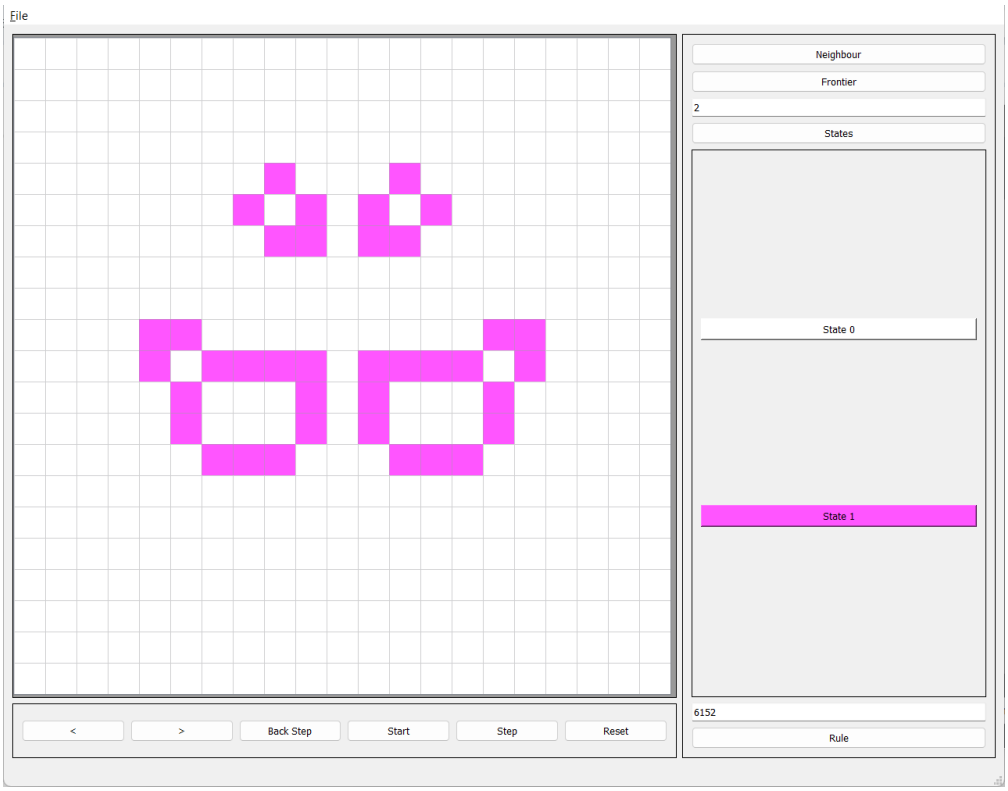


Figura 8.2: Interfaz de usuario principal.

- **Interfaz de vecindad**

  Esta interfaz permite al usuario definir la topología de la vecindad que determinará la dinámica del CA. Para ello, se presenta un espacio celular de menor tamaño en el cual se pueden activar o desactivar las celdas que formarán parte de la vecindad (Figura 8.3).

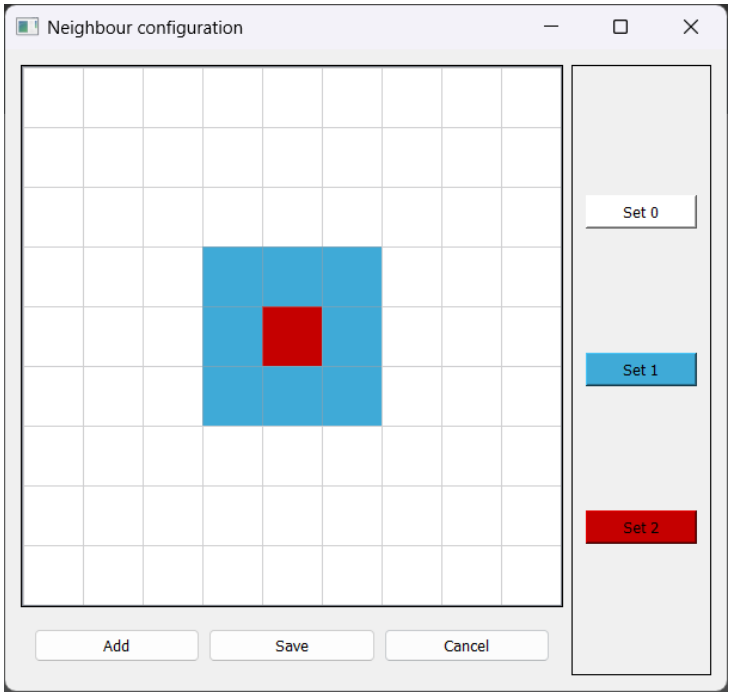


Figura 8.3: Interfaz de usuario que define la vecindad.

- **Interfaz para definir color**

  Esta ventana utiliza un componente nativo de **PyQt5** para la selección de colores. Permite al usuario asignar un color a cada estado del autómata (Figura 8.4).

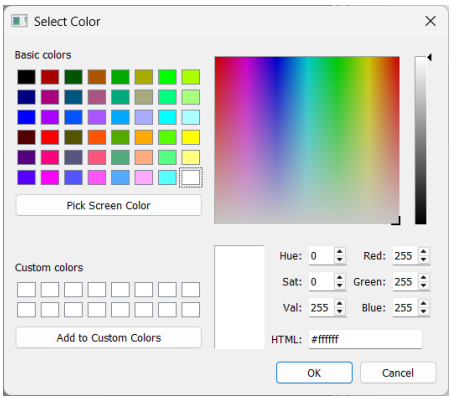


Figura 8.4: Interfaz de usuario para definir el color.

Bibliografía

[1] W. MathWorld, "Cellular automaton," n.d., consultado el 28 de Marzo del 2025. [Online]. Available: https://mathworld.wolfram.com/CellularAutomaton.html

[2] H. Zenil and G. J. Martínez, "Cellular automata," *Scholarpedia*, vol. 19, no. 4, p. 53227, 2024. [Online]. Available: http://www.scholarpedia.org/article/Cellular_automata

[3] D. A. R. Gómez, "Descripción y aplicaciones de los autómatas celulares," 2011, consultado el 27 de Marzo del 2025. [Online]. Available: https://delta.cs.cinvestav.mx/~mcintosh/cellularautomata/Summer_Research_files/Arti_Ver_Inv_2011_DARG.pdf

[4] M. O. Domínguez, "Redes neuronales artificiales," *Ingenio y Conciencia Boletín Científico de la Escuela Superior Ciudad Sahagún*, vol. 12, no. 23, pp. 38–44, 2025. [Online]. Available: https://doi.org/10.29057/escs.v12i23.14132

[5] D. J. Matich, "Redes neuronales: Conceptos básicos y aplicaciones." n.d., consultado el 28 de Marzo del 2025. [Online]. Available: https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf

[6] D. Pub, "Growing cellular automata," 2020, consultado el 27 de Marzo del 2025. [Online]. Available: https://distill.pub/2020/growing-ca/

[7] *Emerging Trends in Applications and Infrastructures for Computational Biology, Bioinformatics, and Systems Biology.* Elsevier, 2016, accedido el 13 de junio de 2025. [Online]. Available: https://doi.org/10.1016/c2015-0-01779-8

[8] S. E. Yacoubi, T. Plénet, S. Dridi, F. Bagnoli, L. Lefèvre, and C. Raïevsky, "Some control and observation issues in cellular automata," *Complex*

Systems, vol. 30, no. 3, pp. 391–413, Sep. 2021, accedido el 14 de junio de 2025. [Online]. Available: https://doi.org/10.25088/complexsystems.30.3.391

[9] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, n.d., consultado el 29 de Marzo del 2025. [Online]. Available: https://journals.aps.org/pre/abstract/10.1103/PhysRevE.100.032402

[10] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[11] ——, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[12] A. O. Cordero. (2025) Introduccion a sistemas dinamicos. CINVESTAV. Accedido el 8 de noviembre de 2025. [Online]. Available: https://umi.cinvestav.mx/Portals/UMI_LAB/Maestria%20material%20de%20apoyo/Seales%20y%20sistemas%201.pdf?ver=JCee2vW9QexvfmWPmQ7KIA==

[13] G. C. Vázquez, E. A. Romero, R. V. Cristóbal, and J. U. Alonso. (2025) Libro complejidad. Complejidad y Economía. Accedido el 8 de noviembre de 2025. [Online]. Available: https://complejidad.iiec.unam.mx/libro/capitulo1/index.php

[14] E. Equipo editorial. (2025) Sistema - características, tipos y ejemplos. Concepto. Accedido el 8 de noviembre de 2025. [Online]. Available: https://concepto.de/sistema

[15] (2025) 2 major types of systems — natural system vs artificial system — board exams simplified - contentxseed. contentXseed. Accedido el 8 de noviembre de 2025. [Online]. Available: https://contentxseed.com/2-major-types-of-systems-natural-vs-artificial

[16] E. D. Rosario. (2025) Señales y sistemas. ESPOL. Accedido el 8 de noviembre de 2025. [Online]. Available: https://blog.espol.edu.ec/telg1001/sistemas-continuos-y-discretos

[17] (2025) Teoría general de sistemas. Accedido el 9 de noviembre de 2025. [Online]. Available: https://8d8a87251d.cbaul-cdnwnd.com/b3abc3d20220b43c1b18cf3768b8f6e1/200000090-2218722189/Sistemas%20Estaticos%20-%20Dinamicos-0.pdf?ph=8d8a87251d

[18] M. S. Rojas. (2018) Determinísticos y probabilísticos. Scribd. Accedido el 8 de noviembre de 2025. [Online]. Available: https://www.scribd.com/document/380696672/Deterministicos-y-Probabilisticos

[19] J. D. M. Ospina. (2023) La dualidad de sistemas: sistemas abiertos y cerrados en la teoría general de sistemas. Universidad de Colombia. Accedido el 8 de noviembre de 2025. [Online]. Available: https://udecolombia.edu.co/wp-content/uploads/2024/12/Monsalve-JD.-2023_TGS-Sistemas-abiertos-y-cerrados.pdf

[20] Y. J. López. (2019) Dominio y propiedades de los sistemas. Universidad Autónoma del Estado de Hidalgo. Accedido el 8 de noviembre de 2025. [Online]. Available: https://www.uaeh.edu.mx/docencia/P_Presentaciones/Sahagun/industrial/2019/Dominio-y-propiedades-de-los-sistemas.pdf

[21] E. V. Alberto, T. H. Eduardo, and E. E. V. Romero. (2021) Exploración de universos discretos con autómatas celulares en dos dimensiones. Accedido el 8 de noviembre de 2025. [Online]. Available: https://www.comunidad.escom.ipn.mx/genaro/Papers/Thesis_files/tesisEricEduardoAlbertoTT2019-B067.pdf

[22] E. Kofman. (2025) Modelado y simulación de sistemas dinámicos: Métodos, algoritmos y herramientas. FCEIA. Accedido el 8 de noviembre de 2025. [Online]. Available: https://www.fceia.unr.edu.ar/~kofman/files/eci_MyS_1.pdf

[23] I. A. Simón and D. R. G. Belinchón. (2025) La teoría del caos. Universidad de Cantabria. Accedido el 8 de noviembre de 2025. [Online]. Available: https://repositorio.unican.es/xmlui/bitstream/handle/10902/30650/AltadillSimonIgnacio.pdf?sequence=1&isAllowed=y

[24] J. Torres. (2025) Introducción a los sistemas complejos. Universidad de Granada. Accedido el 8 de noviembre de 2025. [Online]. Available: https://www.ugr.es/~jtorres/tema_1_sistemas_complejosv2.pdf

[25] I. Zelinka, A. Sanayei, H. Zenil, and O. E. Rössler, Eds., *How Nature Works*. Heidelberg: Springer International Publishing, 2014, accedido el 23 de septiembre de 2025. [Online]. Available: https://doi.org/10.1007/978-3-319-00254-5

[26] I. Zelinka and J. Burguillo, *Complex Systems*, 01 2018, pp. 11–34.

[27] S. Moriello, "Sistemas complejos, caos y vida artificial," https://www.redcientifica.com/doc/doc200303050001.html, n.d., accedido el 23 de septiembre de 2025.

[28] L. R. Izquierdo, J. M. G. Ordax, J. I. Santos, and R. D. O. Martínez, "Modelado de sistemas complejos mediante simulación basada en agentes

y mediante dinámica de sistemas," *Empiria. Revista de Metodología de Ciencias Sociales*, no. 16, p. 85, Oct. 2008, accedido el 24 de septiembre de 2025. [Online]. Available: https://doi.org/10.5944/empiria.16.2008.1391

[29] A. Hoekstra, J. Kroc, and P. Sloot, *Introduction to Modeling of Complex Systems Using Cellular Automata - intro from editors of the book "Simulating Complex Systems by Cellular Automata"*, 05 2010, vol. 2010, pp. 1–16.

[30] J. Damerow. (2025) John von neumann's cellular automata. Accedido el 23 de julio de 2025. [Online]. Available: https://embryo.asu.edu/pages/john-von-neumanns-cellular-automata

[31] E. W. Weisstein, "Cellular automaton," https://mathworld.wolfram.com/CellularAutomaton.html, 2025, from MathWorld–A Wolfram Resource, accedido el 23 de julio de 2025.

[32] S. Wolfram, "Cellular automata as simple self-organizing systems," Caltech, Tech. Rep. CALT-68-938, 1982, general Theoretical Physics, 11 páginas.

[33] F. S. Caparrini, "Autómatas celulares," https://www.cs.us.es/~fsancho/Blog/posts/Automatas_Celulares.md.html, n.d., accedido el 24 de septiembre de 2025.

[34] K. Bhattacharjee, "Cellular automata: Reversibility, semi-reversibility and randomness," Ph.D. dissertation, Indian Institute of Engineering Science and Technology, Howrah, West Bengal, 2019, accedido el 23 de agosto de 2025.

[35] H. Sayama, "11.1: Definition of cellular automata," https://math.libretexts.org/Bookshelves/Scientific_Computing_Simulations_and_Modeling/Introduction_to_the_Modeling_and_Analysis_of_Complex_Systems_(Sayama)/11%3A_Cellular_Automata_I__Modeling/11.01%3A_Definition_of_Cellular_Automata, n.d., accedido el 24 de septiembre de 2025. Licencia CCBY-NC-SA.

[36] A. Ilachinski, *Cellular Automata: A Discrete Universe*. Singapore: World Scientific, 2001, center for Naval Analyses, USA.

[37] J. von Neumann, *Theory of Self-Reproducing Automata*, A. W. Burks, Ed. Urbana, Illinois: University of Illinois Press, 1966.

[38] Y. W. Kwon, *Multiphysics and Multiscale Modeling: Techniques and Applications*, 1st ed. CRC Press, 2015. [Online]. Available: https://doi.org/10.1201/b19098

[39] "Conway's game of life," https://conwaylife.com/wiki/Conway%27s_Game_of_Life, 2024, Última edición el 2 de diciembre de 2024. Accedido el 24 de septiembre de 2025.

[40] S. Wolfram, *A New Kind of Science*. Champaign, Illinois: Wolfram Media, 2002.

[41] T. Tyler, "Cellular automata - the margolus neighbourhood," https://cell-auto.com/neighbourhood/margolus, n.d., accedido el 24 de septiembre de 2025.

[42] O. Ureña Ponce, "Juan josé raygoza panduro," https://riudg.udg.mx/bitstream/20.500.12104/79915/1/MCUCEI01068FT.pdf, 2015, tesis de Maestría en Ciencias en Ingeniería Electrónica y Computación, Universidad de Guadalajara, Centro Universitario de Ciencias Exactas e Ingeniería. [Online]. Available: https://riudg.udg.mx/bitstream/20.500.12104/79915/1/MCUCEI01068FT.pdf

[43] G. J. Martinez, A. Adamatzky, and J. Seck Tuoh Mora, *Some Notes About the Game of Life Cellular Automaton*, 01 2022, pp. 93–104.

[44] E. W. Weisstein, "Totalistic cellular automaton," https://mathworld.wolfram.com/TotalisticCellularAutomaton.html, n.d., from MathWorld–A Wolfram Resource. [Online]. Available: https://mathworld.wolfram.com/TotalisticCellularAutomaton.html

[45] ——, "von neumann neighborhood," *MathWorld – A Wolfram Web Resource*, n.d., accessed 2025-11-12.

[46] J. von Neumann and A. W. Burks, *Theory of Self-Reproducing Automata*. Urbana, IL: University of Illinois Press, 1966.

[47] C. A. W. contributors, "Cellular automaton, history," *Wikipedia*, n.d., accessed 2025-11-12.

[48] Mathematik, Informatik, Naturwissenschaft und Technik am Computer, "John conway's game of life - an introduction to cellular automata," https://beltoforion.de/en/game_of_life, nov 2025, accedido el 11 de noviembre de 2025.

[49] M. Sipper, "Paul rendell: Turing machine universality of the game of life," *Genetic Programming and Evolvable Machines*, vol. 18, no. 1, pp. 115–117, Jan. 2017, accedido el 11 de noviembre de 2025.

[50] E. W. Weisstein. (2025) Rule 30. MathWorld–A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule30.html

[51] ——. (2025) Rule 110. MathWorld–A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule110.html

[52] ——. (2025) Rule 90. MathWorld–A Wolfram Resource. Accedido el 12 de noviembre de 2025. [Online]. Available: https://mathworld.wolfram.com/Rule90.html

[53] G. J. Martinez, "A note on elementary cellular automata classification," 2013. [Online]. Available: https://arxiv.org/abs/1306.5577

[54] G. J. Martínez. (2025) Teoría del campo promedio en autómatas celulares similares a "the game of life". [Online]. Available: https://delta.cs.cinvestav.mx/~mcintosh/comun/tesismaestria/genaro/thesis.html

[55] M. Vispoel, A. Daly, and J. Baetens, "Progress, gaps and obstacles in the classification of cellular automata," *Physica D: Nonlinear Phenomena*, vol. 432, p. 133074, 11 2021.

[56] K. C. II and S. Yu, "Undecidability of ca classification schemes," *Complex Systems*, vol. 2, pp. 177–190, 1988.

[57] ——, "Undecidability of ca classification schemes," *Complex Systems*, vol. 2, pp. 177–190, 1988.

[58] D. Grattarola, L. Livi, and C. Alippi, "Learning graph cellular automata," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 20 983–20 994. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/af87f7cdcda223c41c3f3ef05a3aaeea-Paper.pdf

[59] P. Gács, "Probabilistic cellular automata with andrei toom," *Brazilian Journal of Probability and Statistics*, vol. 38, no. 2, Jun. 2024, accedido el 23 de julio de 2025. [Online]. Available: https://doi.org/10.1214/24-bjps605

[60] H. Hartman, P. Tamayo, and W. Klein, "Inhomogeneous cellular automata and statistical mechanics," *Complex Systems*, vol. 1, no. 2, 1987, massachusetts Institute of Technology and Boston University. [Online]. Available: https://content.wolfram.com/uploads/sites/13/2018/02/01-2-2.pdf

[61] L. Becker. (2025) Continuous cellular automata – hegl. HEGL – Heidelberg Experimental Geometry Lab, accedido el 23 de julio de 2025. [Online]. Available: https://hegl.mathi.uni-heidelberg.de/continuous-cellular-automata

[62] T. de Monterrey, *Data Mining: Módulo 4 - Clasificación, redes neuronales.* México: Tecnológico de Monterrey, 2025, material académico, Escuela de Ingeniería y Ciencias. [Online]. Available: https://tec.mx

[63] C. Manning, "Artificial intelligence definitions," https://hai-production.s3.amazonaws.com/files/2020-09/AI-Definitions-HAI.pdf, Sep. 2020, accedido el 23 de septiembre de 2025.

[64] G. Cloud, "¿qué es la inteligencia artificial o ia? — google cloud," 2025, accedido el 20 de marzo de 2025. [Online]. Available: https://cloud.google.com/learn/what-is-artificial-intelligence?hl=es-419

[65] J. McCarthy, "What is artificial intelligence?" Online, 2007, http://jmc.stanford.edu/articles/whatisai/whatisai.pdf.

[66] Oracle, "¿qué es el aprendizaje automático?" 2025, accedido el 20 de marzo de 2025. [Online]. Available: https://www.oracle.com/mx/artificial-intelligence/machine-learning/what-is-machine-learning/#:~:text=El%20aprendizaje%20automtico%20(ML)%20es,que%20imitan%20la%20inteligencia%20humana

[67] G. Cloud. (s.f.) ¿qué es el aprendizaje automático? tipos y usos. [Accedido: 07-abr-2025]. [Online]. Available: https://cloud.google.com/learn/what-is-machine-learning?hl=es-419

[68] Oracle. (s.f.) ¿qué es el aprendizaje automático? México. [Accedido: 07-abr-2025]. [Online]. Available: https://www.oracle.com/mx/artificial-intelligence/machine-learning/what-is-machine-learning/

[69] IBM. (s.f.) ¿qué es el aprendizaje automático? México. [Accedido: 07-abr-2025]. [Online]. Available: https://www.ibm.com/mx-es/think/topics/machine-learning

[70] A. Main, "What is ai training?" https://www.linkedin.com/pulse/what-ai-training-andy-main-2f4oe/, November 2023, aI Strategy Specialist — Ensuring GDPR & ITSEC Compliance — Transforming Telecom Infrastructure.

[71] Oracle. (2024) ¿Qué es el big data? Accedido el 28 de abril de 2025. [Online]. Available: https://www.oracle.com/mx/big-data/what-is-big-data/

[72] Google Cloud. (s.f.) ¿qué es el aprendizaje no supervisado? [Online]. Available: https://cloud.google.com/discover/what-is-unsupervised-learning?hl=es-419

[73] I. Amazon Web Services. (s.f.) ¿qué es el aprendizaje mediante refuerzo? - explicación del aprendizaje mediante refuerzo. [Online]. Available: https://aws.amazon.com/es/what-is/reinforcement-learning/#:~:text=El%20aprendizaje%20por%20refuerzo%20(RL,utilizan%20para%20lograr%20sus%20objetivos.

[74] Adobe. (s.f.) ¿qué es el aprendizaje automático? [Accedido: 07-abr-2025]. [Online]. Available: https://business.adobe.com/ar/products/real-time-customer-data-platform/what-is-machine-learning.html

[75] Clínica Universidad de Navarra, "Qué es la neurona," https://www.cun.es/diccionario-medico/terminos/neurona, n.d., diccionario médico. Clínica Universidad de Navarra.

[76] P. Bordoli, "La neurona," 11 2023.

[77] brgfx, "Freepik graphic resource," http://www.freepik.com, designed by brgfx / Freepik. Accessed: 2025-09-17.

[78] S. Mhatre. (2020, June) What is the relation between artificial and biological neuron? Medium. [Online]. Available: https://saurabhnativeblog.medium.com/what-is-the-relation-between-artificial-and-biological-neuron-18b05831036

[79] Y. Lao León, A. Rivas Méndez, M. Pérez Pravia, and F. Marrero Delgado, "Procedimiento para el pronóstico de la demanda mediante redes neuronales artificiales / procedure for forecasting demand by using artificial neural networks," *Ciencias Holguin*, vol. 23, pp. 43–59, 01 2017.

[80] Google, "Neural networks: Activation functions," https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions, n.d., google for Developers. [Online]. Available: https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions

[81] AWS, "¿qué es una red neuronal? - explicación de las redes neuronales artificiales," https://aws.amazon.com/es/what-is/neural-network, n.d., amazon Web Services, Inc. [Online]. Available: https://aws.amazon.com/es/what-is/neural-network

[82] Telefónica Tech, "Las matemáticas del machine learning: Redes neuronales (parte i)," https://telefonicatech.com/blog/las-matematicas-del-machine-learning-redes-neuronales-parte-i, noviembre 2019, telefonica Tech. [Online]. Available: https://telefonicatech.com/blog/las-matematicas-del-machine-learning-redes-neuronales-parte-i

[83] C. A. Ruiz and M. S. Basualdo, "Redes neuronales: Conceptos básicos y aplicaciones," https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf, marzo 2001, universidad Tecnológica Nacional, Facultad Regional Rosario. [Online]. Available: https://www.frro.utn.edu.ar/repositorio/catedras/quimica/5_anio/orientadora1/monograias/matich-redesneuronales.pdf

[84] D. Bergmann and C. Stryker, "¿qué es la función de pérdida?" https://www.ibm.com/mx-es/think/topics/loss-function, n.d., iBM. [Online]. Available: https://www.ibm.com/mx-es/think/topics/loss-function

[85] Ultralytics, "Función de pérdida en ia/ml explicada," https://www.ultralytics.com/es/glossary/loss-function, julio 2025, ultralytics — Revolutionizing the World of Vision AI. [Online]. Available: https://www.ultralytics.com/es/glossary/loss-function

[86] A. Amidi and S. Amidi. (2025) Cs 230 - convolutional neural networks cheatsheet. Stanford University. Accedido el 13 de noviembre de 2025. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks

[87] ——. (2025) Cs 230 - recurrent neural networks cheatsheet. Stanford University. Accedido el 13 de noviembre de 2025. [Online]. Available: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

[88] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, vol. 100, no. 3, Sep. 2019, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1103/PhysRevE.100.032402

[89] *Asynchronicity in Neural Cellular Automata*, ser. ALIFE 2022: The 2022 Conference on Artificial Life, vol. ALIFE 2021: The 2021 Conference on Artificial Life, 07 2021. [Online]. Available: https://doi.org/10.1162/isal_a_00461

[90] A. Mordvintsev, E. Randazzo, and C. Fouts, "Growing isotropic neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2205.01681

[91] *Emergent Multiscale Structures and Generative Potential of Isotropic Neural Cellular Automata*, ser. ALIFE 2022: The 2022 Conference on Artificial Life, vol. ALIFE 2024: Proceedings of the 2024 Artificial Life Conference, 07 2024. [Online]. Available: https://doi.org/10.1162/isal_a_00745

[92] E. Randazzo, A. Mordvintsev, and C. Fouts, "Growing steerable neural cellular automata," 2023. [Online]. Available: https://arxiv.org/abs/2302.10197

[93] E. Randazzo, A. Mordvintsev, E. Niklasson, M. Levin, and S. Greydanus, "Self-classifying mnist digits," *Distill*, 2020, https://distill.pub/2020/selforg/mnist.

[94] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, "Adversarial reprogramming of neural cellular automata," *Distill*, 2021, https://distill.pub/selforg/2021/adversarial.

[95] D. Zhang, C. Choi, J. Kim, and Y. M. Kim, "Learning to generate 3d shapes with generative cellular automata," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=rABUmU3ulQh

[96] S. Sudhakaran, D. Grbic, S. Li, A. Katona, E. Najarro, C. Glanois, and S. Risi, "Growing 3d artefacts and functional machines with neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2103.08737

[97] K. Horibe, K. Walker, and S. Risi, "Regenerating soft robots through neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2102.02579

[98] E. Niklasson, A. Mordvintsev, E. Randazzo, and M. Levin, "Self-organising textures," *Distill*, 2021, https://distill.pub/selforg/2021/textures.

[99] E. R. Alexander Mordvintsev, Eyvind Niklasson, "Texture generation with neural cellular automata," *CoRR*, vol. abs/2105.07299, 2021. [Online]. Available: https://arxiv.org/abs/2105.07299

[100] A. Mordvintsev and E. Niklasson, "μnca: Texture generation with ultra-compact neural cellular automata," *CoRR*, vol. abs/2111.13545, 2021. [Online]. Available: https://arxiv.org/abs/2111.13545

[101] E. Pajouheshgar, Y. Xu, A. Mordvintsev, E. Niklasson, T. Zhang, and S. Süsstrunk, "Mesh neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2311.02820

[102] E. Pajouheshgar, Y. Xu, A. Abbasi, A. Mordvintsev, W. Jakob, and S. Süsstrunk, "Neural cellular automata: From cells to pixels," 2025. [Online]. Available: https://arxiv.org/abs/2506.22899

[103] A. H. Ruiz, A. Vilalta, and F. Moreno-Noguer, "Neural cellular automata manifold," 2021. [Online]. Available: https://arxiv.org/abs/2006.12155

[104] K. Frans, "Stampca: Growing emoji with conditional neural cellular automata," *kvfrans.com*, 2021. [Online]. Available: http://kvfrans.com/stampca-conditional-neural-cellular-automata/

[105] M. Chen and Z. Wang, "Image generation with neural cellular automatas," 2020. [Online]. Available: https://arxiv.org/abs/2010.04949

[106] R. B. Palm, M. González-Duque, S. Sudhakaran, and S. Risi, "Variational neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2201.12360

[107] M. Tesfaldet, D. Nowrouzezahrai, and C. Pal, "Attention-based neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2211.01233

[108] J. Tang, S. Kumar, L. De Lorenzis, and E. Hosseini, "Neural cellular automata for solidification microstructure modelling," *Computer Methods in Applied Mechanics and Engineering*, vol. 414, p. 116197, Sep. 2023. [Online]. Available: http://dx.doi.org/10.1016/j.cma.2023.116197

[109] G. Li, C. Leefmans, J. Williams *et al.*, "Deep learning with photonic neural cellular automata," *Light: Science & Applications*, vol. 13, no. 283, 2024, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1038/s41377-024-01651-7

[110] C. Grasso and J. Bongard, "Empowered neural cellular automata," ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 108–111. [Online]. Available: https://doi.org/10.1145/3520304.3529067

[111] A. D. Richardson, T. Antal, R. A. Blythe, and L. J. Schumacher, "Learning spatio-temporal patterns with neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2310.14809

[112] A. E. Azuero Azuero, "Significatividad del marco metodológico en el desarrollo de proyectos de investigación," *Revista Arbitrada Interdisciplinaria Koinonía*, vol. 4, no. 8, pp. 110–127, julio 2019, jul. - Dic. 2019. [En línea]. Disponible en: Dialnet. [Accedido: 21 abr., 2025].

[113] UNAM, "Los diseños de investigación en la psicología," https://enlinea.iztacala.unam.mx/resources/modules/UAPAS/Los%20diseos%20de%20investigacin%20en%20la%20psicologa/#:~:text=Los%20diseos%20experimentales,%20son%20aquellos,puros,%20cuasiexperimentos%20y%20pre%20experimentos., 2025, accedido el 23 de abril de 2025.

[114] B. Capili and J. K. Anastasi, "An introduction to types of quasi-experimental designs," *American Journal of Nursing*, vol. 124, no. 11, pp. 50–52, november 2024, epub 2024 Oct 24.

[115] "Metodología cuantitativa," https://web.ujaen.es/investiga/tics_tfg/enfo_cuanti.html, n.d., universidad de Jaén. Enlace inaccesible (403 Forbidden) al momento de consulta. [Online]. Available: https://web.ujaen.es/investiga/tics_tfg/enfo_cuanti.html

[116] T. C. Luza, O. J. Córdova, J. A. Gonzáles, F. A. F. Limo, and C. A. B. Flores, *Métodos mixtos de investigación para principiantes*, primera edición digital ed. Puno, Perú: Instituto Universitario de Innovación Ciencia y Tecnología Inudi Perú S.A.C., 2023, depósito legal en la Biblioteca Nacional del Perú N° 2023-06193. [Online]. Available: https://editorial.inudi.edu.pe

[117] J. Arias-Gómez, M. Ángel Villasís-Keever, and M. G. Miranda-Novales, "El protocolo de investigación iii: la población de estudio," *Revista de Alergia de México*, vol. 63, no. 2, p. 201, May 2016, accedido el 23 de abril de 2025. [Online]. Available: https://doi.org/10.29262/ram.v63i2.181

[118] M. E. D. Hernandez, "Población, muestra y muestreo," https://www.uaeh.edu.mx/docencia/P_Presentaciones/huejutla/enfermeria/2017/Poblacion_Muestra_Muestreo.pdf, 2017, universidad Autónoma del Estado de Hidalgo :: UAEH. Accedido el 23 de abril de 2025.

[119] Universidad Veracruzana, "Introducción a la investigación: guía interactiva — tipos de investigación," https://www.uv.mx/apps/bdh/investigacion/unidad1/investigacion-tipos.html, n.d., material didáctico interactivo sobre clasificación de investigaciones: básica, aplicada, documental, de campo, experimental, cualitativa, cuantitativa, entre otras. [Online]. Available: https://www.uv.mx/apps/bdh/investigacion/unidad1/investigacion-tipos.html

[120] Q. Team, "Estadística descriptiva: Qué es y cómo utilizarla," https://www.questionpro.com/blog/es/estadistica-descriptiva/, 2023, questionPro. Accedido el 15 de mayo de 2025.

[121] UAPA, "Lenguajes de Programación," https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/2655/mod_resource/content/

1/UAPA-Lenguajes-Programacion/index.html, 2025, unidad de Apoyo para el Aprendizaje, UNAM.

[122] ClickPanda. (2023) La Importancia de seleccionar el entorno y el lenguaje de base de datos correcto. LinkedIn. [Online]. Available: https://www.linkedin.com/pulse/la-importancia-de-seleccionar-el-entorno-y-lenguaje-base-2pnme/

[123] Universidad de Valladolid, "El lenguaje c++ — fundamentos de programación en c++," https://www2.eii.uva.es/fund_inf/cpp/temas/1_introduccion/introduccion.html, n.d., material didáctico sobre programación imperativa, orientada a objetos y genérica en C++. [Online]. Available: https://www2.eii.uva.es/fund_inf/cpp/temas/1_introduccion/introduccion.html

[124] MSMK University, "¿qué es c++ en programación?" https://msmk.university/que-es-c-en-programacion, mayo 2025, publicado el 21 de mayo de 2025. Revisión conceptual del lenguaje C++: historia, paradigmas, eficiencia, aplicaciones y evolución. [Online]. Available: https://msmk.university/que-es-c-en-programacion

[125] Pybind11 Developers, "pybind11 — seamless operability between c++11 and python," https://pybind11.readthedocs.io/en/stable/index.html, n.d., documentación oficial de pybind11. [Online]. Available: https://pybind11.readthedocs.io/en/stable/index.html

[126] Microsoft Ignite. (2025) Microsoft learn: Build skills that open doors in your career. Microsoft Learn. Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/iostream?view=msvc-170

[127] (2025) Clase vector - microsoft learn. Microsoft Learn. Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/vector-class?view=msvc-170

[128] (2025) cmath - microsoft learn. Microsoft Learn. Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/cmath?view=msvc-170

[129] W3Schools.com. (2025) C++ maps. W3Schools Online Web Tutorials. Accedido el 11 de noviembre de 2025. [Online]. Available: https://www.w3schools.com/cpp/cpp_maps.asp

[130] (2025) utility - microsoft learn. Microsoft Learn. Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/utility?view=msvc-170

[131] (2025) algorithm - microsoft learn. Microsoft Learn. Accedido el 11 de noviembre de 2025. [Online]. Available: https://learn.microsoft.com/es-es/cpp/standard-library/algorithm?view=msvc-170

[132] Supermicro, "¿qué es cuda?" https://www.supermicro.com/es/glossary/cuda, n.d., artículo técnico sobre CUDA como plataforma de cómputo paralelo desarrollada por NVIDIA. Explica su arquitectura, ventajas, aplicaciones en IA, HPC, procesamiento de video, y diferencias con la CPU. [Online]. Available: https://www.supermicro.com/es/glossary/cuda

[133] Python Software Foundation. (2025) El tutorial de Python — documentación de Python - 3.13.4. Python.org. [Online]. Available: https://docs.python.org/es/3.13/tutorial/index.html

[134] Amazon Web Services, Inc. (2025) ¿Qué es Python? - Explicación del lenguaje Python. AWS. [Online]. Available: https://aws.amazon.com/es/what-is/python/

[135] Python Software Foundation. (2025) os — Interfaces misceláneas del sistema operativo — documentación de Python - 3.10.17. Python.org. [Online]. Available: https://docs.python.org/es/3.10/library/os.html

[136] A. Alfaro. (2025) La librería numpy. https://aprendeconalf.es/docencia/python/manual/numpy/. Accedido: 11-jun-2025. [Online]. Available: https://aprendeconalf.es/docencia/python/manual/numpy/

[137] CuPy, "Cupy," https://cupy.dev, n.d., sitio oficial de CuPy. [Online]. Available: https://cupy.dev

[138] PyTorch, "Pytorch documentation — pytorch 2.8 documentation," https://docs.pytorch.org/docs/stable/index.html, n.d., documentación oficial del tensor library optimizado para deep learning en GPU y CPU. [Online]. Available: https://docs.pytorch.org/docs/stable/index.html

[139] D. Bergmann and C. Stryker, "¿qué es pytorch?" https://www.ibm.com/mx-es/think/topics/pytorch, n.d., iBM. [Online]. Available: https://www.ibm.com/mx-es/think/topics/pytorch

[140] Riverbank Computing Limited, "Pyqt5 — comprehensive python bindings for qt v5," https://pypi.org/project/PyQt5/, julio 2024, pyPI. [Online]. Available: https://pypi.org/project/PyQt5/

[141] M. Fitzpatrick, "Pyqt6 vs pyside6: What's the difference between the two python qt libraries?" https://www.pythonguis.com/faq/pyqt6-vs-pyside6, n.d., python GUIs. [Online]. Available: https://www.pythonguis.com/faq/pyqt6-vs-pyside6

[142] h5py Project, "h5py — pythonic interface to hdf5," https://www.h5py.org, n.d., sitio oficial del proyecto h5py. [Online]. Available: https://www.h5py.org

[143] A. Alfaro. (2025) La librería pandas. https://aprendeconalf.es/docencia/python/manual/pandas/. Accedido: 11-jun-2025. [Online]. Available: https://aprendeconalf.es/docencia/python/manual/pandas/

[144] A. Alférez. (2025) La librería matplotlib. Accedido: 12-jun-2025. [Online]. Available: https://aprendeconalf.es/docencia/python/manual/matplotlib/

[145] Master Data Scientist. (2025) Scikit-learn, herramienta básica para el data science en python. https://www.master-data-scientist.com/scikit-learn-data-science/. Accedido: 11-jun-2025. [Online]. Available: https://www.master-data-scientist.com/scikit-learn-data-science/

[146] Itop Consulting. (2025) Scikit-learn: biblioteca de python para el aprendizaje automático. https://www.itop.es/soluciones-tecnologicas/business-analytics-business-intelligence/scikit-learn.html. Accedido: 11-jun-2025. [Online]. Available: https://www.itop.es/soluciones-tecnologicas/business-analytics-business-intelligence/scikit-learn.html

[147] G. C. Cuadrado. (2024) Visual studio code: Editor de código para desarrolladores. [En línea]. Disponible en: https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/ [Accedido: 12-jun-2025]. [Online]. Available: https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/

[148] F. G. de Zúñiga. (2024) ¿qué es visual studio code y cuáles son sus ventajas? [En línea]. Disponible en: https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas. [Online]. Available: https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas

[149] GitHub. (2025) Acerca de github y git. [En línea]. Disponible en: https://docs.github.com/es/get-started/start-your-journey/about-github-and-git [Accedido: 12-jun-2025]. [Online]. Available: https://docs.github.com/es/get-started/start-your-journey/about-github-and-git

[150] Canva. (2025) Acerca de canva. [En línea]. Disponible en: https://www.canva.com/es_es/about/ [Accedido: 12-jun-2025]. [Online]. Available: https://www.canva.com/es_es/about/

[151] B. van der Zander, "Texstudio - a latex editor," https://www.texstudio.org/#about, n.d., teXstudio - A LaTeX editor. [Online]. Available: https://www.texstudio.org/#about

[152] Pymes y Calidad 2.0, "¿cómo se hace una matriz de riesgo? - ventajas y desventajas," https://www.pymesycalidad20.com/como-se-hace-una-matriz-de-riesgo-ventajas-y-desventajas.html, oct 2025, accedido el 28 de octubre de 2025.

[153] MDN Web Docs, "Mvc - glosario de mdn web docs," https://developer.mozilla.org/es/docs/Glossary/MVC, oct 2025, accedido el 28 de octubre de 2025.

[154] Microsoft Support, "Crear un diagrama de clase uml - soporte técnico de microsoft," https://support.microsoft.com/es-es/topic/crear-un-diagrama-de-clase-uml-de6be927-8a7b-4a79-ae63-90da8f1a8a6b, nov 2025, accedido el 11 de noviembre de 2025.